

Un lenguaje de dominio específico para describir situaciones
tabulares y materializarlas en múltiples arquitecturas de salida

Trabajo de diploma

Dayan Cabrera

septiembre de 2026

Resumen

Los procesos de planificación de una facultad —el horario docente, los tribunales de tesis, la gestión del departamento— se materializan hoy en libros de cálculo contruidos a medida. Cada uno mezcla la lógica del problema con la mecánica del destino, y esa mezcla es lo que impide reutilizar el trabajo cuando cambia el formato de salida.

Este trabajo diseña e implementa un lenguaje de dominio específico, interno en Common Lisp, para describir situaciones tabulares con derivaciones vivas, marcado por significado y entrada validada; y un protocolo de extensión que permite incorporar una arquitectura de salida nueva sin modificar el núcleo. A diferencia de los trabajos previos de la línea, la independencia de la arquitectura no se afirma: se hace cumplir por construcción y se comprueba con pruebas automáticas, y se somete a prueba materializando las mismas descripciones en tres destinos que no comparten nada entre sí.

El sistema incorpora un evaluador de referencia que fija la semántica del lenguaje con independencia de todo destino y sirve de oráculo de validación. La verificación cierra el ciclo pasando los libros generados por LibreOffice en modo headless y comprobando que cada valor calculado coincide con el del evaluador.

Índice general

1. Introducción	6
1.1. Problema científico	7
1.2. Pregunta científica	7
1.3. Hipótesis	8
1.4. Objetivo general	8
1.5. Objetivos específicos	8
1.6. Qué se hace distinto respecto a los trabajos previos de la línea	9
1.6.1. (a) Subir la representación intermedia por encima del backend	9
1.6.2. (b) Definir y probar el mecanismo de extensión	10
1.6.3. (c) Acertar el núcleo semántico	11
1.7. Metodología y fuentes de evidencia	12
1.8. Estructura del documento	12
1.9. Declaración sobre el uso de inteligencia artificial	13
2. Preliminares teóricos	15
2.1. Lenguajes de dominio específico	15
2.1.1. Internos y externos	15
2.1.2. Cuándo merece la pena construir uno	16
2.2. Common Lisp como lenguaje anfitrión	16
2.3. Árboles de sintaxis abstracta	17
2.3.1. Por qué en un DSL interno no hace falta analizador	18
2.4. Funciones genéricas y despacho múltiple	18
2.4.1. Función genérica y método no son lo mismo	18
2.4.2. Por qué los métodos se pueden añadir desde fuera	19
2.4.3. Despachar sobre más de un parámetro	19
2.5. Herencia múltiple y composición de propiedades	20
2.5.1. El ejemplo clásico de los lenguajes de salida	21
2.5.2. A quién pertenece la idea	21
2.5.3. Lo que este trabajo hace con ella	21
2.6. El modelo de evaluación de las hojas de cálculo	22
2.6.1. Primera pieza: cada celda es una expresión pura sobre otras celdas	22
2.6.2. Segunda pieza: un motor de dependencias recalcula	22
2.6.3. Tercera pieza: una expresión, muchas filas	22
2.6.4. Por qué ese modelo no existe en otros destinos	23
2.6.5. Por qué apoyarse en el modelo sin declararlo contamina la representación	23
2.7. El formato OOXML y la materialización de libros	24
2.8. Hojas de cálculo como desarrollo por usuarios finales	24
3. Estado del arte	26

3.1.	La línea de investigación de la facultad	26
3.2.	Automatizar la creación de Generadores Multilenguajes (2017)	27
3.2.1.	La metodología de los seis pasos	27
3.2.2.	<code>defnode</code>	28
3.2.3.	<code>gcode</code> y <code>recognize-pattern</code>	28
3.2.4.	La jerarquía de lenguajes, y a quién pertenece	29
3.2.5.	Qué se hereda y qué no aporta	29
3.3.	LDMAG (2025)	30
3.3.1.	El elemento consulta	30
3.3.2.	<code>def-entidad</code> y la lectura en español	30
3.3.3.	El modelo visual no tiene tabla	31
3.3.4.	Los operadores no alcanzan para las preguntas del capítulo 2	31
3.3.5.	La conexión consulta-vista quedó sin cerrar	31
3.3.6.	El código generado	32
3.3.7.	No hay capítulo de experimentación ni de validación	32
3.3.8.	La lección	32
3.4.	Un DSL para la descripción declarativa de estructuras tabulares (2026)	33
3.4.1.	Qué hace	33
3.4.2.	Qué se hereda	33
3.4.3.	El mecanismo de extensión no cubre el cálculo	34
3.4.4.	El árbol no tiene jerarquía	36
3.4.5.	El JSON de traspaso ya es Excel	36
3.4.6.	Conceptos de rejilla dentro del árbol	37
3.4.7.	<code>expr-counta</code>	37
3.4.8.	La regla de marcado perdió el color	37
3.4.9.	La afirmación central nunca se sometió a prueba	39
3.5.	Comparación entre los tres trabajos	39
3.6.	El punto ciego compartido: el modelo de evaluación	40
3.7.	Herramientas externas de generación dirigida por modelos	42
3.8.	Por qué un DSL y no un modelo de lenguaje	43
3.9.	Síntesis: qué se hereda y qué se rehace	44
4.	El dominio de las situaciones tabulares	47
4.1.	El corpus	47
4.2.	Qué es una situación tabular	49
4.3.	Las cuatro patas del núcleo semántico	50
4.3.1.	Datos	50
4.3.2.	Derivaciones vivas	50
4.3.3.	Marcado con significado	50
4.3.4.	Entrada validada	51
4.4.	Análisis del corpus	51
4.4.1.	El método de conteo, declarado	51
4.4.2.	Funciones de Excel presentes en el corpus	52
4.4.3.	Características de <code>openpyxl</code> presentes en el corpus	53
4.5.	Construcciones que la taxonomía de 2026 no cubre	53
4.5.1.	Búsqueda por clave: el <i>join</i> que falta	54
4.5.2.	Clave compuesta	54
4.5.3.	Entrada validada	55
4.5.4.	Rol de campo: qué se escribe y qué se calcula	55
4.5.5.	Capacidad y reserva: el documento que crece	56

4.5.6.	Relación uno a muchos	57
4.5.7.	Conteo de distintos	58
4.5.8.	Navegación	58
4.6.	Lo que ya nombra bien el corpus	59
4.7.	La frontera: qué entra en el lenguaje y qué no	60
4.8.	Método de verificación disponible	61
5.	El lenguaje	62
5.1.	Visión de conjunto	62
5.2.	Las formas de declaración	64
5.2.1.	defsituacion	64
5.2.2.	coleccion	64
5.2.3.	campo	65
5.2.4.	marca	66
5.2.5.	vista	67
5.2.6.	parametro	67
5.3.	El rol de campo	67
5.4.	El álgebra de expresión	69
5.4.1.	Por qué un solo nodo para los operadores	70
5.5.	Ámbitos y la semántica de fila	71
5.6.	La decisión de lectura del :DONDE	72
5.7.	La marca declara significado	73
5.8.	Lo que el lenguaje NO dice	74
5.9.	Una descripción completa, comentada	75
6.	La arquitectura del sistema	79
6.1.	Criterios de diseño	79
6.2.	Las seis fases de la compilación	81
6.3.	El evaluador de referencia	83
6.3.1.	La coerción, y por qué está en un solo sitio	84
6.4.	El protocolo	85
6.4.1.	Las operaciones	85
6.4.2.	La macro que hace cumplir el criterio	87
6.4.3.	La tabla de contraste con 2026	88
6.4.4.	Ámbito frente a plan	89
6.5.	La jerarquía de capacidades	90
6.5.1.	Qué cumple cada arquitectura	91
6.6.	El mecanismo de degradación	92
6.6.1.	Las cuatro respuestas	92
6.6.2.	La frontera entre degradar y rechazar	93
6.6.3.	El sistema de condiciones, y dónde vive la política	93
6.6.4.	El informe de conformidad	94
6.7.	Estructura de sistemas	95
6.8.	Registro de decisiones	96
7.	Implementación	99
7.1.	Panorama	99
7.2.	La arquitectura de texto plano	101
7.2.1.	Por qué existe	101
7.2.2.	Su resolución de carencias, entera	101
7.2.3.	Cómo emite	102

7.3.	La arquitectura de hoja de cálculo	103
7.3.1.	El plan y el direccionamiento	103
7.3.2.	La emisión de expresiones	104
7.3.3.	Fórmulas realmente emitidas	106
7.3.4.	Cómo se emula el crecimiento	107
7.3.5.	El plano de libro y las tres reglas	108
7.4.	El materializador en Python	109
7.4.1.	Qué hace	109
7.4.2.	Qué no hace	109
7.4.3.	Por qué existe	109
7.4.4.	Por qué esto no es el error de 2026	109
7.5.	La arquitectura web	110
7.5.1.	Por qué se elige	110
7.5.2.	La emisión a JavaScript	111
7.5.3.	La misma expresión en los dos destinos	111
7.5.4.	Las dos implementaciones de la coerción	112
7.5.5.	El resto de la página	113
7.6.	La inversión de capacidades	114
7.7.	Cómo se añade una arquitectura nueva	115
7.8.	Dos correcciones al documento de diseño	116
8.	Experimentación y validación	117
8.1.	Qué hay que demostrar	117
8.2.	Invariantes arquitectónicos	118
8.2.1.	I1 — Léxico del núcleo y del protocolo	118
8.2.2.	Cómo se comprobó que I1 muere	119
8.2.3.	I2 — Firma del protocolo	120
8.2.4.	Cómo se comprobó que I2 muere	120
8.2.5.	I3 — Grafo de dependencias	121
8.3.	La misma descripción en tres arquitecturas	121
8.3.1.	El artefacto de texto plano	122
8.3.2.	El plano de libro	122
8.3.3.	El JavaScript generado	122
8.4.	Los informes de conformidad	123
8.4.1.	Qué dice esta tabla	124
8.5.	El <i>round-trip</i> por LibreOffice	125
8.5.1.	Por qué una batería de pruebas de Python no basta	125
8.5.2.	El procedimiento	125
8.5.3.	El resultado	126
8.5.4.	Qué demuestra	126
8.5.5.	Qué se puede afirmar frente a los antecedentes	126
8.6.	El caso de la figura 6.2	127
8.7.	Lo que NO está validado	128
9.	Conclusiones	130
9.1.	Las tres contribuciones	131
10.	Recomendaciones	132
11.	El alcance del lenguaje	134
11.1.	La pregunta y por qué no se puede contestar razonando	134

11.2. El método	135
11.3. Los cinco dominios	136
11.3.1. Control de inventario de un almacén — CABE-FORZADO	136
11.3.2. Evaluación de un curso — CABE-A-MEDIAS	137
11.3.3. Presupuesto y ejecución de un proyecto — CABE-A-MEDIAS	137
11.3.4. Torneo con eliminatorias — CABE-A-MEDIAS	138
11.3.5. Cuadrante de turnos de un equipo — CABE-A-MEDIAS	139
11.4. Lo que el experimento rompió	139
11.5. Los nueve huecos y cómo se cerraron	141
11.6. La tabla cruzada: de caro a barato	142
11.7. Qué sigue sin caber, y por qué se deja fuera	145
11.8. Cómo enunciar el alcance en una defensa	146
12. Apéndice: el sistema después del experimento de alcance	148
12.1. Por qué existe este apéndice	148
12.2. Recuentos	148
12.3. Construcciones del álgebra que los capítulos no recogen	150
12.4. Capacidades nuevas	150
12.5. La vista cruzada	151
12.6. Lo que se corrigió en el código	151
12.7. Lo que sigue sin estar	152
Referencias bibliográficas	154

Capítulo 1

Introducción

En la Facultad de Matemática y Computación de la Universidad de La Habana hay procesos de planificación que se repiten cada curso: el horario docente, los tribunales de tesis y la gestión del departamento. Los tres tienen la misma forma: datos que ya se conocen, decisiones que alguien toma a mano, valores que se calculan a partir de los anteriores, y situaciones que no se pueden permitir —un profesor citado en dos locales a la misma hora, una asignatura sin profesores suficientes— que el documento debe señalar cuando ocurren.

Los tres se materializan hoy en libros de Excel hechos a medida, producidos siempre con el mismo ciclo: el tutor describe la situación en lenguaje natural; el diplomante escribe un programa en Python que, con `openpyxl`, fabrica el libro; el libro se entrega y se refina por iteración hasta que sirve. Ha funcionado tres veces. Pero no escala, y de ese diagnóstico sale el trabajo.

No escala por dos razones. La primera es de volumen: cada situación nueva es un programa nuevo. La segunda, la que importa, es que en ese programa se mezclan dos cosas sin relación entre sí: **la lógica del problema** —quién tiene conflicto de horario, cuántas defensas acumula un profesor— y **la mecánica del destino** —que la columna del tutor es la C, que la primera fila de datos es la 4, que una búsqueda se escribe `VLOOKUP(...,FALSE)`.

El ejemplo más nítido está en `tribunales/hoja_dia.py:40-43`, cuatro líneas que rellenan las columnas del tribunal de una defensa:

```
ws[f"{col}-{f}"] = (  
    f'=IF({L.COL_ESTUDIANTE}-{f}="", "", '  
    f'IFERROR(VLOOKUP({L.COL_ESTUDIANTE}-{f},TesisTribunal,{idx},FALSE),"")'  
)
```

Lo que esas líneas *quieren decir* son tres frases simples del dominio. Si esta fila todavía no tiene estudiante, no muestres nada: es el `IF(...="", "", ...)`. Si lo tiene, trae de la tabla de tesis el profesor que ocupa este rol: es el `VLOOKUP`. Y si ese estudiante no aparece en la tabla de tesis, tampoco muestres nada: es el `IFERROR`, rama alcanzable porque el desplegable de estudiantes de esa hoja avisa pero no bloquea (`tribunales/hoja_dia.py:113-116`). Lo que esas líneas *dicen*, en cambio, es comillas, símbolos de dólar, índices de columna, el nombre

de un rango y un FALSE. Si mañana hay que hacer lo mismo en una página web, no sirven para nada, aunque las tres frases del dominio sean idénticas.

No es un caso aislado. Un recuento de los generadores, hecho el 2026-09-20 con coincidencia por límite de palabra y sin incluir las pruebas, da la medida del acoplamiento: 45 apariciones de IF, 13 de COUNTIF, 11 de VLOOKUP, 9 de IFERROR y 5 de SUMIF, más 24 rangos nombrados, 17 reglas de formato condicional y 9 validaciones de datos, en 4 355 líneas de generadores.

La situación no es particular de la facultad. Las hojas de cálculo son programas escritos por quienes no se consideran programadores, sin las prácticas de verificación que se aplicarían a cualquier otro programa (Ko et al. 2011), y la tasa de celdas incorrectas en libros de uso real es alta y persistente (Panko 1998). Los libros del repositorio mitigan parte de ese riesgo, pero fijan la mitigación en el lenguaje de Excel, junto con la lógica.

La pregunta de este trabajo no es, por tanto, cómo escribir mejores libros de Excel, sino si la descripción puede separarse del sistema que la materializa.

1.1. Problema científico

Las descripciones de situaciones tabulares con cálculo, marcado y entrada validada no admiten hoy una representación reutilizable: la lógica del dominio queda fijada en el vocabulario del sistema que la materializa —coordenadas, rangos, nombres de función de hoja de cálculo—, de modo que cambiar de destino obliga a reescribir la descripción completa. Los intentos previos de esta línea formularon su punto de extensión en términos del primer destino implementado, con lo que la independencia respecto de la arquitectura de salida se afirma pero no se sostiene, y el mecanismo por el cual un tercero incorporaría una arquitectura nueva no llega a definirse ni a probarse.

1.2. Pregunta científica

¿Es posible definir una representación intermedia de las situaciones tabulares que sea independiente del sistema que las materializa, **y un protocolo que permita a un tercero incorporar una arquitectura de salida nueva sin modificar el núcleo del lenguaje?**

La pregunta tiene dos mitades, y las dos son falsables por separado. La **primera** se refuta produciendo una construcción del dominio cuya representación intermedia no pueda materializarse en alguna arquitectura sin introducir en ella vocabulario propio de otra: si ahí aparece una letra de columna, una coordenada, un número de fila o el nombre de una función de hoja de cálculo, la independencia no se cumple. La **segunda** se refuta si, entregados el protocolo y el juego de conformidad, incorporar una arquitectura nueva exige modificar el núcleo, el protocolo o alguna arquitectura ya escrita; admite además un experimento con sujeto externo: que el tutor implemente la arquitectura de Emacs por su cuenta.

El contraste con el antecedente directo es explícito. La pregunta de (Morales Lazo 2026) era: «¿Es posible, mediante el diseño de un lenguaje de dominio específico, separar la descripción

semántica de una estructura tabular del mecanismo concreto de su generación, de modo que el mismo programa declarativo permanezca válido con independencia del backend de salida?» (Introducción, recuadro). Tiene solo la primera mitad, y no se puso a prueba: aquel trabajo implementó una sola arquitectura, la de Excel (objetivo específico 4 y capítulo 5), de modo que el experimento que habría respondido su propia pregunta nunca se ejecutó. Una pregunta sobre independencia respondida con un único destino solo puede recibir una respuesta afirmativa por defecto.

1.3. Hipótesis

Si la representación intermedia se define sobre un modelo de evaluación declarado explícitamente —valores con nombre, derivaciones puras sobre esos valores y recálculo ante cambios—, y si el punto de extensión se formula como un protocolo de funciones genéricas que despachan sobre la arquitectura de salida y transportan un plan opaco fabricado por ella misma, entonces: (1) la misma descripción puede materializarse en arquitecturas de naturaleza distinta sin que ninguna decisión de direccionamiento suba a la representación intermedia; y (2) un tercero puede incorporar una arquitectura nueva escribiendo solo métodos propios sobre el protocolo, sin tocar el núcleo, el protocolo ni ninguna otra arquitectura. La hipótesis queda refutada si alguna de las dos afirmaciones falla sobre las tres situaciones del corpus.

1.4. Objetivo general

Diseñar, implementar y validar un lenguaje de dominio específico interno en Common Lisp para la descripción de situaciones tabulares con derivaciones vivas, marcado con significado y entrada validada, junto con un protocolo de extensión que permita a terceros incorporar arquitecturas de salida sin modificar el núcleo del lenguaje, y demostrar empíricamente ambas cosas sobre tres documentos reales en uso en la Facultad de Matemática y Computación.

1.5. Objetivos específicos

1. **Caracterizar el dominio** a partir de los tres libros en uso, con un inventario de construcciones y evidencia de archivo y línea. *Verificable*: toda construcción del lenguaje tiene un uso documentado en el corpus, y toda construcción del corpus tiene forma en el lenguaje o queda fuera de alcance con su razón declarada.
2. **Declarar el modelo de evaluación** y la clasificación de las arquitecturas por capacidades. *Verificable*: cada capacidad es una clase del sistema y tiene al menos un caso de conformidad.
3. **Diseñar el lenguaje y su representación intermedia**, sin que ninguna decisión de direccionamiento aparezca por encima del punto de extensión. *Verificable*: una prueba recorre el núcleo y la serialización de las tres situaciones resueltas y falla si encuentra vocabulario de direccionamiento o nombres de función de hoja de cálculo.
4. **Especificar el protocolo de arquitectura** como contrato de extensión publicable,

documentado para que un tercero lo use sin acceso al núcleo. *Verificable*: una prueba comprueba que todas sus funciones genéricas se especializan sobre la arquitectura. Es lo que el antecedente no supera.

5. **Implementar arquitecturas de naturaleza deliberadamente distinta** —texto plano, Excel, web y SQL—, sin que ninguna dependa de otra ni del lenguaje. *Verificable*: el grafo de dependencias lo impide por construcción, y las cuatro pasan el juego de conformidad en lo que declaran cumplir.
6. **Definir el mecanismo de degradación** —cumplir, emular, degradar, rechazar— y el informe de conformidad que lo documenta. *Verificable*: compilar los tres libros contra las cuatro arquitecturas bajo tres políticas da una tabla de resultados reproducible.
7. **Validar contra el corpus y contra un tercero**. *Verificable*: (a) el libro generado es semánticamente equivalente al que produce hoy el generador Python; (b) las fórmulas se evalúan, con verificación reproducible de ida y vuelta por LibreOffice; (c) el tutor implementa la arquitectura de Emacs usando solo el protocolo y el juego de conformidad.

1.6. Qué se hace distinto respecto a los trabajos previos de la línea

Los tres trabajos previos de la línea —(Arcia Corcho 2017), la herramienta base para generadores multilinguaje; (Cápiro Colomar 2025), el primer intento del lenguaje; y (Morales Lazo 2026), el antecesor directo— se analizan en el capítulo 2. Aquí se enuncian las tres contribuciones que justifican que éste sea un trabajo nuevo y no una continuación.

1.6.1. (a) Subir la representación intermedia por encima del backend

En (Morales Lazo 2026) la separación entre lenguaje y destino existe para la estructura del documento y no para el cálculo, que es donde está la sustancia. La operación que traduce la estructura, `generate-code`, recibe un parámetro que identifica la arquitectura de salida (sección 5.1.1). La que traduce las expresiones —columnas calculadas, conteos, el cuantificador existencial— se llama `compile-excel-formula` y no lo recibe: sus parámetros son un diccionario de símbolo de columna a **letra de Excel** más tres números de fila (listado 5.1). El punto de extensión está formulado en el vocabulario del primer destino, y no hay dónde enganchar un segundo, porque una página web no tiene letras de columna ni números de fila. *Verificado contra el PDF; pendiente de contraste contra su código fuente, aún no obtenido*. Ese trabajo afirma además que un generador de tablas HTML podría reutilizar su JSON intermedio; eso vale para los encabezados y no para lo que tiene cálculo o marcado, porque ese JSON no lleva la descripción del cálculo sino el cálculo ya compilado en rangos absolutos (listado 5.14).

Aquí la representación intermedia se define por encima del punto de extensión, y el criterio de diseño lo verifica una máquina: **si ahí aparece una letra de columna, una coordenada, un número de fila o el nombre de una función de hoja de cálculo, el diseño está**

mal. La lista prohibida es de *direccionamiento*, no de estructura: «fila» es palabra legítima del dominio; lo prohibido es su dirección. El diccionario de columnas del antecedente no desaparece: baja al plan de la arquitectura de Excel. Lo que desaparece es que esté en la firma del punto de extensión.

1.6.2. (b) Definir y probar el mecanismo de extensión

Ninguno de los tres trabajos previos define qué significa «ser una arquitectura de salida», y por eso «cualquier arquitectura» es en ellos un deseo y no una condición comprobable. Este trabajo lo convierte en cuatro piezas.

Un protocolo explícito: funciones genéricas, todas especializadas sobre la arquitectura, que separan la planificación —donde cada arquitectura decide su direccionamiento— de la emisión, donde escribe su artefacto consultando lo que ya decidió. Por eso emitir una expresión no requiere números de fila. Se apoya en el despacho múltiple y en la extensibilidad externa de las funciones genéricas de CLOS (Keene 1989; Kiczales, Rivières, y Bobrow 1991): los métodos se añaden desde fuera sin tocar los existentes.

Una jerarquía de capacidades por composición. En lugar de enumerar destinos se nombran propiedades —rejilla, recálculo vivo, entrada, dominio de entrada, orden declarado, marcado visual, marcado textual, navegación— y cada arquitectura se declara como composición de ellas. Así un método puede especializarse en la propiedad y ser compartido por dos arquitecturas que no se conocen entre sí. La idea no es original: (Arcia Corcho 2017) la describe y la atribuye a trabajos anteriores de la facultad (ADOL* y GEMURAL, sección 3.3.2 y figura 3.1). Lo nuevo es implementarla: allí se describe en prosa y en una figura, y lo único implementado es una propiedad suelta, la indentación (sección 4.2). Ni (Cápiro Colomar 2025) ni (Morales Lazo 2026) la usan; en este último todas las clases del árbol declaran superclases vacías.

Un juego de pruebas de conformidad: descripciones mínimas, una por capacidad, con las aserciones que una arquitectura debe satisfacer para declararse conforme. Se entrega al tercero junto con el protocolo.

Un mecanismo de degradación. Cuando una descripción pide algo que una arquitectura no cumple de forma nativa, esa arquitectura —no el núcleo, y no el usuario— responde una de cuatro cosas: **cumple**, **emula** (lo fabrica, declarando a qué coste), **degrada** (una versión más pobre que sirve para lo mismo) o **rechaza**. La frontera entre degradar y rechazar hay que defenderla: se degrada cuando el resultado sigue cumpliendo el propósito por otra vía; se rechaza cuando *parecería* correcto sin serlo. Cada compilación produce un informe de conformidad con esa clasificación, que es la evidencia que ninguno de los tres trabajos previos tenía.

En el experimento con sujeto externo, **los dos resultados son publicables**: si el tutor logra su arquitectura de Emacs, el mecanismo funciona; si no, el trabajo documenta dónde se rompió, que es más honesto que afirmar la extensibilidad sin probarla.

1.6.3. (c) Acertar el núcleo semántico

Ésta es la contribución menos visible y probablemente la más determinante. El núcleo del dominio no es «una tabla»: son cuatro cosas.

Concepto	Ejemplo en el libro de tribunales
Datos	La lista de profesores, estudiantes, locales y tesis
Derivaciones vivas	El tribunal que se autocompleta al elegir estudiante
Marcado con significado	El señalamiento del conflicto de horario
Entrada validada	El desplegable que avisa si se escribe un estudiante inexistente

Los libros que se entregan hoy no son fotografías de una situación: son programas pequeños y vivos. Al escribir el nombre de un estudiante, las celdas de al lado se rellenan solas; si dos defensas coinciden en momento y comparten profesor, se señalan mientras el usuario escribe. Ese comportamiento es el modelo de evaluación de las hojas de cálculo —un grafo de dependencias entre celdas que un motor recalcula cuando algo cambia (Sestoft 2014)—, y es lo que las tres tesis previas dieron por supuesto sin declararlo; por eso se les fue colando el modelo de Excel dentro del árbol. Aquí se declara arriba y se separa de la replicación por filas, que sí es propia de Excel. De ahí sale la clasificación de arquitecturas, y que el nivel de recálculo —vivo, bajo demanda, congelado— sea una capacidad declarada por cada arquitectura.

El **marcado declara significado, no color**: la regla dice *insuficiente*, y cada arquitectura decide cómo se ve. La hoja de cobertura del departamento ya no pinta el estado, lo escribe, y concuerda el plural. En (Morales Lazo 2026) la regla de marcado no tiene color en el árbol; el color aparece por primera vez en el JSON del backend, y las tres reglas del caso del horario se distinguen solo por comentarios (listado 6.4). *Su figura 6.2 sugiere que los colores se asignan por orden de aparición de las reglas y no por su significado; es lectura propia de una imagen y queda pendiente de confirmar contra el código.*

La **entrada validada** aporta una distinción que ningún trabajo previo recoge: hay restricciones que invalidan y restricciones que solo advierten. El corpus ya la tiene, con desplegables de aviso no bloqueante en los tres libros (`tribunales/hoja_dia.py:113-116`, `horarios/hoja_docencia.py:87`, `departamento/hoja_asignacion.py:140`). Es decisión del dominio, no de Excel.

Hay, en fin, una asimetría entre los antecedentes cercanos: (Cápiro Colomar 2025) identifica el elemento consulta, pero no tiene tabla ni agregación, de modo que las preguntas de conteo de su capítulo 2 no son expresables en su capítulo 3; (Morales Lazo 2026) tiene tablas y agregación, pero no consultas ni búsqueda por clave. Cada uno tiene la mitad de lo que hace falta.

1.7. Metodología y fuentes de evidencia

Este trabajo dispone de algo que los tres anteriores no tuvieron: **tres documentos reales, en uso, refinados por iteración con el usuario final**. Los tres diseñaron su lenguaje contra ejemplos descritos en prosa, y el efecto se ve en los resultados: sin un caso real que obligue a cerrar cada cabo, el diseño se detiene justo antes de la parte difícil. Es lo que explica que (Cápiro Colomar 2025) no conecte sus consultas con sus vistas, y que sus páginas de resultados lleven un comentario donde deberían ir los datos (sección 3.3).

El corpus cumple dos funciones. **Como vocabulario**, cada construcción del lenguaje se justifica contra un uso existente, con archivo y línea: la búsqueda por clave sale de los 11 usos de `VLOOKUP`; los agregados con filtro, de los de `COUNTIF`, `SUMIF`, `COUNTIFS` y `SUMIFS` en la hoja de cobertura del departamento; la concordancia de plural, de los mensajes de estado de esa hoja. El criterio de admisión es explícito: una construcción entra si el corpus la necesita y se rechaza si es mecanismo del destino y no hecho del dominio. **Como oráculo**, el éxito no es producir un `.xlsx`, sino que el archivo generado sea equivalente al que hoy produce el código Python, juzgado por las 5 782 líneas de pruebas existentes.

A esto se añade una verificación que ninguno de los tres trabajos previos tiene: comprobar de forma **reproducible y automatizada** que las fórmulas generadas se evalúan. El procedimiento genera el libro, lo rellena como lo haría un usuario, lo convierte con LibreOffice en modo desatendido y relea los valores calculados; ya está resuelto aquí y ha cazado tres fallos que la batería de pruebas daba por buenos. El contraste debe ser preciso: (Morales Lazo 2026) **sí** afirma en sus conclusiones que los archivos generados «se evalúan correctamente en Excel y LibreOffice Calc», y lo respalda con capturas; lo que no hay en ninguno de los tres es un procedimiento que cualquiera pueda volver a ejecutar.

El resto del método es el habitual en el diseño de lenguajes de dominio específico (Mernik, Heering, y Sloane 2005; Fowler 2010), con una particularidad heredada de (Arcia Corcho 2017): al ser un lenguaje **interno** en Common Lisp, en el sentido de (Hudak 1996), el paso de construir analizadores léxico y sintáctico desaparece —lo hace el intérprete del anfitrión— y el papel de compilador lo cumplen las macros (Graham 1993; Seibel 2005). El costo se declara: el usuario escribe sintaxis de Common Lisp, ventaja para el usuario real de este trabajo y limitación para cualquier otro; una sintaxis externa queda como trabajo futuro. Queda fuera del alcance toda resolución automática: la facultad tiene una línea larga dedicada a *resolver* problemas de planificación; ésta es la de *representarlos*.

1.8. Estructura del documento

El **capítulo 1, Preliminares teóricos**, presenta los fundamentos: el modelo de evaluación de las hojas de cálculo, los lenguajes de dominio específico y la distinción entre internos y externos, y las características de Common Lisp en que se apoya la implementación: macros, CLOS con despacho múltiple, y condiciones y reinicios.

El **capítulo 2, Estado del arte**, revisa la investigación sobre hojas de cálculo como artefactos de software, las herramientas de ingeniería dirigida por modelos y de generación a

múltiples destinos, y —con más detalle— los tres trabajos previos de esta línea.

El **capítulo 3, El dominio de las situaciones tabulares**, presenta los tres documentos del corpus, extrae el inventario de construcciones con evidencia de archivo y línea, y declara el modelo de evaluación del que se deriva la clasificación de arquitecturas.

El **capítulo 4, El lenguaje**, presenta el modelo de situación —parámetros, colecciones, campos con su rol, marcas y vistas—, el álgebra de expresión, los ámbitos y variables de fila, y lo que queda fuera.

El **capítulo 5, La arquitectura del sistema**, describe las fases de compilación, el protocolo con el contraste frente al punto de extensión del antecedente, la jerarquía de capacidades, el mecanismo de degradación y los invariantes que convierten las afirmaciones de diseño en pruebas.

El **capítulo 6, Implementación**, documenta el núcleo, el análisis estático, el evaluador de referencia y las cuatro arquitecturas de salida, y justifica la frontera entre el trabajo semántico, íntegro en Common Lisp, y la escritura del `.xlsx`, delegada en un materializador por razones mecánicas del formato (*ECMA-376: Office Open XML File Formats* 2016).

El **capítulo 7, Experimentación y validación**, presenta la equivalencia con el corpus, la verificación de las fórmulas, la tabla de conformidad de las cuatro arquitecturas bajo las tres políticas, y el experimento de extensión por un tercero. Cierran el documento las **Conclusiones** y las **Recomendaciones**.

1.9. Declaración sobre el uso de inteligencia artificial

Durante el desarrollo de este trabajo se utilizaron herramientas de inteligencia artificial generativa. Se declara aquí en qué se usaron y en qué no.

Se usaron para cuatro tareas. Primera, la revisión sistemática de los tres trabajos previos: localización de listados, secciones y citas textuales, y contraste entre lo que el análisis afirmaba y lo que las fuentes decían; ese contraste está registrado y produjo trece correcciones de exactitud verificadas contra la fuente, entre ellas la retirada de una afirmación demasiado fuerte sobre la evidencia de evaluación de fórmulas. Segunda, el recuento del corpus, con método de conteo declarado. Tercera, borradores iniciales de secciones, revisados y reescritos. Cuarta, asistencia en la escritura de código y pruebas.

No se usaron para las decisiones que constituyen el aporte: el diagnóstico de los trabajos previos, el diseño del lenguaje y de su representación intermedia, el protocolo, las capacidades, la degradación y los criterios de validación son decisiones propias. Tampoco sustituyen a la verificación: toda afirmación sobre un trabajo previo se comprobó contra su documento, y toda afirmación sobre el corpus contra el código, con archivo y línea. Lo que sigue siendo lectura propia de una figura o inferencia no verificada está marcado como tal.

([Morales Lazo 2026](#)) incluye una declaración equivalente en su introducción, con cuatro usos —extracción de datos desde documentos en PDF, borradores de secciones, versiones iniciales del código y un borrador de la opinión del tutor—, y hace constar que las decisiones de diseño

y la validación se realizaron de forma independiente. Hay por tanto un precedente explícito en la línea, que este trabajo sigue y amplía: declara no solo las tareas, sino el procedimiento por el cual lo producido con asistencia se verificó contra las fuentes.

Capítulo 2

Preliminares teóricos

Este capítulo fija el vocabulario técnico con el que se lee el resto del documento. No supone conocimiento previo de Common Lisp ni de la construcción de lenguajes. Las cuatro primeras secciones explican qué es un lenguaje de dominio específico y qué ofrece el lenguaje anfitrión elegido; la quinta y la sexta sostienen el argumento central del trabajo —la composición de propiedades por herencia y el modelo de evaluación de las hojas de cálculo—; las dos últimas sitúan el formato de salida y la naturaleza del artefacto que se genera. Los antecedentes concretos de esta línea de investigación y el diseño del sistema propuesto se tratan en capítulos posteriores.

2.1. Lenguajes de dominio específico

Un **lenguaje de dominio específico** (DSL, por *domain-specific language*) es un lenguaje de programación de expresividad limitada, dirigido a un dominio particular (Fowler 2010). La limitación no es un defecto: es el rasgo que lo define. Un lenguaje de propósito general aspira a expresar cualquier cómputo; un DSL aspira a expresar bien un conjunto acotado de cosas y a no poder expresar el resto. SQL describe consultas sobre relaciones y no sabe abrir un socket.

La ganancia tiene dos partes. La primera es de notación: el programa se escribe con los términos del dominio, de modo que quien conoce el dominio puede leerlo y, en el mejor de los casos, escribirlo. La segunda es de análisis: como el lenguaje sabe de qué habla, puede comprobar lo que un lenguaje de propósito general no podría, y generar código especializado a partir de un enunciado breve (Mernik, Heering, y Sloane 2005; Czarnecki y Eisenecker 2000).

2.1.1. Internos y externos

Fowler distingue dos formas de construir un DSL (Fowler 2010). Un **DSL externo** tiene sintaxis propia. Hay que escribir un analizador léxico, que parte el texto en piezas, y un analizador sintáctico, que comprueba que esas piezas forman frases válidas y construye una representación interna. A cambio, la sintaxis es exactamente la que convenga al dominio.

SQL es un DSL externo. Las herramientas de ingeniería dirigida por modelos —Xtext sobre EMF, por ejemplo— automatizan casi todo ese trabajo y añaden editor y comprobación en tiempo de escritura (Eysholdt y Behrens 2010; Steinberg et al. 2008; Kelly y Tolvanen 2008).

Un **DSL interno** —Hudak lo llama *domain-specific embedded language* (Hudak 1996)— no tiene sintaxis propia: se escribe dentro de un lenguaje anfitrión y con la sintaxis de ese anfitrión, de modo que el programa del DSL es, visto desde fuera, un programa del anfitrión. Mernik y otros lo describen como el patrón de implementación por *embebido*, y lo contraponen al preprocesado y al compilador o intérprete independientes (Mernik, Heering, y Sloane 2005).

El compromiso es directo. **A favor:** no hay que escribir analizador léxico ni sintáctico, porque los hace el anfitrión, ni hay que construir herramientas; el depurador, el editor, el sistema de módulos y las bibliotecas del anfitrión están disponibles desde el primer día. **En contra:** el usuario queda obligado a la sintaxis del anfitrión, le guste o no —si el anfitrión exige paréntesis, el usuario escribe paréntesis—; los mensajes de error son los del anfitrión y hablan de la expansión, no de lo que el usuario escribió; y la frontera del lenguaje es porosa, porque nada impide, salvo disciplina, mezclar construcciones del DSL con código arbitrario del anfitrión.

Ese último punto afecta directamente a lo que este trabajo promete. Un DSL interno mal construido es una biblioteca con nombres bonitos: se ejecuta, pero no deja ninguna estructura que inspeccionar, y por tanto no hay nada que traducir a un segundo destino. Un DSL interno bien construido no ejecuta la descripción: la **convierte en datos** y después trabaja sobre esos datos. Esa diferencia decide si la independencia del destino es siquiera posible.

2.1.2. Cuándo merece la pena construir uno

Mernik y otros sistematizan la decisión (Mernik, Heering, y Sloane 2005). Los indicios favorables son: un dominio reconocible con vocabulario propio; programas parecidos entre sí que se escriben una y otra vez; conocimiento experto que hoy vive en la cabeza de alguien y se pierde en cada implementación; y un mismo enunciado del que deban salir varios artefactos distintos. Voelter añade el análisis de costo: el lenguaje es una inversión que solo se amortiza si se escriben en él suficientes programas y si el dominio es estable (Voelter 2013). Los indicios contrarios son los simétricos: un dominio todavía desconocido, un solo programa a escribir, o un lenguaje que crece hasta volverse de propósito general.

2.2. Common Lisp como lenguaje anfitrión

Tres propiedades de Common Lisp explican por qué un DSL interno en ese lenguaje es cualitativamente distinto de un DSL interno en casi cualquier otro. Las tres están en el estándar (*American National Standard for Information Systems. Programming Language Common Lisp* 1994) y descritas con detalle en la literatura clásica (Steele 1990; Graham 1993; Seibel 2005).

Homoiconicidad. El código de Lisp se escribe con la misma notación que los datos de Lisp. `(+ 5 (* y 2))` es a la vez una expresión que suma y una lista de tres elementos cuyo tercero

es otra lista. El componente que convierte el texto en esas listas se llama el *lector*, y es una pieza del lenguaje disponible para cualquiera. La consecuencia es la que interesa aquí: **el análisis léxico y sintáctico ya está hecho**. Escribir un DSL interno en Lisp no consiste en imitar un lenguaje con llamadas encadenadas, sino en recibir el programa del usuario ya convertido en una estructura de datos.

Macros. Una macro es un programa que recibe código y devuelve código (Graham 1993). Se ejecuta antes que el programa, en lo que se llama *tiempo de expansión*, y lo que devuelve es lo que finalmente se compila. En un DSL interno las macros cumplen el papel del compilador del lenguaje: toman la forma breve que escribe el usuario y la convierten en las llamadas que construyen la representación interna.

Conviene distinguirla de una función. Una función recibe valores ya calculados. Una macro recibe el programa sin evaluar, y puede decidir qué se evalúa, cuándo y en qué orden; puede también rechazar formas mal escritas antes de que el programa llegue a ejecutarse. Eso es lo que permite que un DSL interno en Lisp tenga comprobaciones propias, que es justamente lo que le falta a un DSL interno construido como biblioteca.

CLOS. El sistema de objetos de Common Lisp, incorporado al estándar (*American National Standard for Information Systems. Programming Language Common Lisp* 1994), ofrece clases con herencia múltiple, funciones genéricas con despacho sobre varios argumentos y combinación de métodos (Keene 1989). Es el mecanismo con el que se representa el árbol del DSL y, sobre todo, el mecanismo con el que se traduce ese árbol a varios destinos. Las dos secciones siguientes lo desarrollan, porque de ahí depende el argumento central del trabajo.

Una consecuencia de las tres juntas: la metodología habitual para construir un DSL —determinar el dominio, identificar sus elementos, diseñar el lenguaje, implementar la jerarquía de nodos, construir los analizadores, implementar la generación de código para cada destino— pierde un paso completo al elegir Lisp. El de los analizadores desaparece, porque lo hace el lector; y el diseño del lenguaje se reduce, en buena medida, a elegir bien los nombres de los constructores.

2.3. Árboles de sintaxis abstracta

Cuando un programa lee una descripción, no se queda con el texto: lo convierte en una estructura de piezas que se contienen unas a otras. Cada pieza es un **nodo**; el conjunto es un **árbol de sintaxis abstracta** (AST, por *abstract syntax tree*). Un nodo tiene un tipo —qué clase de cosa es— y huecos con nombre, cada uno de los cuales contiene un valor o bien otro nodo.

La expresión `(+ 5 (* y 2))` se representa así:

```
      suma
    /   \
literal producto
(5)    /   \
      variable literal
```

(y) (2)

El nodo raíz es una suma. Su hueco izquierdo contiene un literal de valor 5; su hueco derecho, un producto cuyos huecos contienen una variable y otro literal. El árbol es *abstracto* porque no guarda nada del texto original: ni los paréntesis, ni los espacios, ni el orden en que se escribieron el operador y sus operandos. Guarda la estructura, y la estructura es lo único que hace falta para traducir. El mismo árbol produce $5 + y * 2$ en notación infija y $(+ 5 (* y 2))$ en prefija; la diferencia está en el traductor, no en el árbol.

Escrito con constructores explícitos, el árbol anterior es esto:

```
(nodo-suma
  :izquierda (nodo-literal :valor 5)
  :derecha   (nodo-producto
    :izquierda (nodo-variable :nombre 'y)
    :derecha   (nodo-literal :valor 2)))
```

2.3.1. Por qué en un DSL interno no hace falta analizador

En un DSL externo, el camino del texto al árbol lo recorre un analizador que hay que escribir: el léxico parte $(+ 5 (* y 2))$ en símbolos y el sintáctico reconoce que eso es una suma de un literal y un producto, y construye los nodos.

En un DSL interno en Lisp ese camino está partido en dos tramos y ninguno hay que escribirlo desde cero. El lector convierte el texto en listas anidadas: ese es el tramo léxico y sintáctico, y viene dado. Después, **las macros convierten esas listas en nodos**, en tiempo de expansión, antes de que el programa se ejecute.

Esa segunda conversión no es un trámite. Una lista anidada ya es un árbol, pero es un árbol sin tipos: todos sus nodos son listas, y no hay forma de preguntarle a una lista qué representa sin mirar su primer elemento y comparar símbolos. Los nodos del AST, en cambio, son instancias de clases distintas, y esa distinción es lo que hace posible el mecanismo de la sección siguiente: escribir una regla de traducción por cada tipo de nodo y dejar que el sistema elija cuál aplicar. Sobre listas no se puede; sobre clases, sí.

En resumen: el DSL interno ahorra el analizador, pero no ahorra el árbol.

2.4. Funciones genéricas y despacho múltiple

Esta sección es la más importante del capítulo, porque el argumento central del trabajo depende de que se entienda con precisión.

2.4.1. Función genérica y método no son lo mismo

En la programación orientada a objetos más difundida, un método *pertenece* a una clase: se escribe dentro de ella y se invoca sobre un objeto. En CLOS no es así ([Keene 1989](#)). Hay dos entidades distintas:

Una **función genérica** es el nombre y la forma de una operación. Declara cómo se llama, cuántos argumentos recibe y qué significa. No contiene código: es un contrato.

Un **método** es una implementación concreta de esa operación para unas clases de argumentos determinadas. Contiene el código. No pertenece a ninguna clase: es una entidad independiente que se *añade* a la función genérica.

Al llamar a una función genérica, el sistema mira las clases de los argumentos reales, selecciona el más específico de los métodos aplicables y lo ejecuta. Esa selección se llama **despacho**.

2.4.2. Por qué los métodos se pueden añadir desde fuera

De esa separación se sigue la propiedad que sostiene toda la extensibilidad de este trabajo: **un método puede definirse en otro archivo, en otro paquete, y cargarse mucho después, sin modificar ni la función genérica ni ninguno de los métodos que ya existen.**

La comparación aclara lo que está en juego. En un sistema donde el método vive dentro de la clase, añadir una operación nueva obliga a abrir todas las clases afectadas y escribir en ellas; quien no sea dueño de ese código, sencillamente no puede. En CLOS, añadir una operación es definir una función genérica nueva y sus métodos, y añadir un caso a una operación existente es definir un método más: ninguna de las dos cosas toca lo ajeno.

Aplicado a lo que aquí interesa: si la operación “*traducir un nodo a una arquitectura*” está planteada como función genérica y recibe la arquitectura como argumento, entonces **incorporar una arquitectura nueva consiste en escribir métodos nuevos, y nada más.** Si esa operación no recibe la arquitectura como argumento, la extensión es imposible sin cambiar la forma de la función, y cambiar la forma de una función obliga a revisar todo lo que ya se escribió con ella.

2.4.3. Despachar sobre más de un parámetro

El **despacho múltiple** es la capacidad de elegir el método mirando las clases de varios argumentos a la vez, y no solo la del primero (Keene 1989; Kiczales, Rivières, y Bobrow 1991). Es lo que distingue a CLOS de los sistemas de objetos de despacho simple.

La diferencia importa porque el problema de este trabajo es de dos dimensiones. Hay N construcciones del lenguaje y M arquitecturas de salida, y hay que decidir qué hacer en cada una de las $N \times M$ combinaciones. Con despacho simple esa matriz se representa a mano: o una cadena de condicionales sobre el destino dentro de cada construcción, o una clase visitante por destino que enumere todas las construcciones. En ambos casos, añadir un destino obliga a tocar código existente. Con despacho múltiple, la matriz *es* el conjunto de métodos, y cada casilla se rellena por separado.

El ejemplo siguiente está simplificado: su propósito es enseñar el mecanismo, no anticipar la interfaz del sistema.

```

;; La funcion generica solo declara la forma de la operacion. No tiene
;; cuerpo: dice que existe una operacion llamada `emitir-marca', que recibe
;; tres argumentos, y nada mas.
(defgeneric emitir-marca (arquitectura marca destino))

;; Primer metodo. Se especializa en DOS argumentos a la vez: la
;; arquitectura tiene que ser de clase `excel' y la marca de clase
;; `marca-problema'. El tercer argumento no esta especializado: acepta
;; cualquier cosa.
(defmethod emitir-marca ((a excel) (m marca-problema) destino)
  (pintar-relleno destino (color-de a :problema)))

;; Segundo metodo. Misma clase de marca, otra arquitectura. Puede vivir en
;; otro archivo y cargarse despues; no hay que tocar nada de lo anterior.
(defmethod emitir-marca ((a web) (m marca-problema) destino)
  (anadir-clase destino "problema"))

;; Tercer metodo. Misma arquitectura que el primero, otra clase de marca.
;; En Excel, una marca meramente informativa no se pinta: se escribe.
(defmethod emitir-marca ((a excel) (m marca-informativa) destino)
  (escribir-texto destino (explicacion-de m)))

```

Tres observaciones sobre ese código:

1. **Ningún método sabe que los otros existen.** El de web no menciona a Excel ni al revés. No hay condicional que enumere los destinos, y por tanto no hay ningún sitio que editar al añadir el cuarto.
2. **La especialización ocurre en los dos primeros parámetros a la vez.** Añadir una clase de marca y añadir una arquitectura son operaciones independientes: cada una rellena una fila o una columna de la matriz sin interferir con la otra.
3. **El tercer parámetro no está especializado.** A la función genérica le da igual qué sea el destino donde se escribe. Solo se despacha sobre lo que discrimina.

CLOS ofrece además la **combinación de métodos**: un método puede declararse como `:before`, `:after` o `:around` respecto del principal, e invocar al siguiente más general con `call-next-method` (Keene 1989). Eso permite que una arquitectura reutilice el comportamiento heredado y le añada un matiz en lugar de reescribirlo. El protocolo de metaobjetos describe cómo se calcula el conjunto de métodos aplicables y en qué orden se combinan (Kiczales, Rivières, y Bobrow 1991).

2.5. Herencia múltiple y composición de propiedades

La **herencia múltiple** permite que una clase tenga varias superclases y reúna lo que cada una aporta (Keene 1989). CLOS resuelve los conflictos con una *lista de precedencia de clases*: un orden lineal de todas las superclases que determina qué método se prefiere y en qué orden se recorren los heredados (*American National Standard for Information Systems. Programming Language Common Lisp* 1994; Kiczales, Rivières, y Bobrow 1991).

Lo interesante no es el mecanismo, que es estándar, sino un uso concreto de él: que las clases no nombren **cosas** sino **propiedades**, y que las cosas se definan como la composición de las propiedades que tienen.

2.5.1. El ejemplo clásico de los lenguajes de salida

El caso que ilustra la idea en la literatura de esta línea de investigación es el de un generador que produce código en varios lenguajes de programación (Arcia Corcho 2017). La forma ingenua de soportar cinco lenguajes de salida es escribir, para cada construcción del DSL, cinco reglas de traducción. El trabajo se multiplica por cinco, y cada construcción nueva vuelve a multiplicarse.

La forma buena parte de observar que los lenguajes de salida **comparten propiedades**. Entonces se declaran clases para las propiedades, no para los lenguajes:

- *lenguaje de notación infija*: escribe `2 + 3`, no `(+ 2 3)`;
- *lenguaje indentado*: los bloques se delimitan con espacios al margen;
- *lenguaje separado por símbolo*: las instrucciones terminan en un separador, típicamente `;;`;
- *lenguaje con operaciones básicas*: dispone de `+`, `*`, `/`, `<`, `>` con esos mismos nombres.

Y después cada lenguaje se define como una composición. Un lenguaje de la familia de C es infijo, separado por símbolo y con operaciones básicas; Java y C# heredan de esa familia y solo declaran en qué se apartan; Python es infijo, indentado y con operaciones básicas, pero no separado por símbolo. Añadir un lenguaje deja de ser escribir todas las reglas otra vez: es declarar de qué hereda y escribir solo lo que hace distinto.

2.5.2. A quién pertenece la idea

Esta precisión de atribución se hace aquí para no repetirla después. La idea de representar las propiedades sintácticas de los lenguajes como clases y componerlas por herencia **no es de** (Arcia Corcho 2017): su propio texto la atribuye expresamente a los trabajos anteriores ADOL* y GEMURAL, y la presenta como un subconjunto de propiedades ya representado en ADOL*. Arcia la expone en prosa y en una figura y la usa como marco, pero en su trabajo **no aparece el código de la composición**: no hay ninguna clase de lenguaje declarada con varias superclases. Lo que sí implementa es una de esas propiedades por separado —la indentación—, con su nodo y sus macros de apoyo: muestra cómo automatizar una propiedad, no cómo componer varias. Esa distinción entre la idea enunciada y la idea demostrada se retoma en el capítulo de antecedentes.

2.5.3. Lo que este trabajo hace con ella

La aportación de este trabajo en este punto es de traslado de dominio: aplicar la composición por propiedades **no a los lenguajes de programación de salida, sino a las arquitecturas de materialización**. En lugar de enumerar Excel, web, SQL y Emacs y escribirlo todo cuatro veces, se declaran propiedades —tener rejilla, recalcular ante cambios, admitir entrada del usuario, tener navegación entre secciones— y cada arquitectura se define por las

que reúne. De ahí sale el mecanismo que permite saber qué se le puede pedir a cada destino y qué no. El desarrollo corresponde al capítulo de diseño.

2.6. El modelo de evaluación de las hojas de cálculo

Las hojas de cálculo tienen un modelo de evaluación muy particular, tan cómodo y tan interiorizado que se usa sin nombrarlo. De ahí viene el riesgo que esta sección quiere hacer visible. Consta de tres piezas, y la tercera es la traicionera.

2.6.1. Primera pieza: cada celda es una expresión pura sobre otras celdas

Una celda contiene un valor literal o una fórmula. La fórmula solo puede leer otras celdas y no puede modificar nada: no asigna, no tiene efectos laterales, y su resultado depende exclusivamente de los valores que lee. Por eso se ha descrito la hoja de cálculo como el lenguaje funcional de primer orden más utilizado del mundo (Peyton Jones, Blackwell, y Burnett 2003). Sestoft formaliza ese núcleo —celdas, expresiones, referencias— al describir la implementación de una hoja de cálculo (Sestoft 2014). La pureza es lo que permite lo demás: si una fórmula pudiera escribir en otra celda, el orden de evaluación importaría y no habría recálculo posible.

2.6.2. Segunda pieza: un motor de dependencias recalcula

Las referencias entre celdas forman un grafo dirigido. Cuando el usuario cambia una celda, el sistema no reevalúa la hoja entera: recorre el grafo y recalcula solo las celdas que dependen de la que cambió, directa o transitivamente, en un orden compatible con esas dependencias (Sestoft 2014). Un ciclo es un error que el sistema detecta y reporta. Esta es la pieza que mantiene *vivo* al documento: el usuario escribe y el resto se ajusta solo, sin que nadie ordene recalcular.

2.6.3. Tercera pieza: una expresión, muchas filas

Una referencia puede ser **absoluta** o **relativa**. Una referencia absoluta señala siempre la misma celda. Una relativa señala una celda *en relación con la posición de la fórmula*: no dice “la celda B4”, dice “la celda que está dos columnas a la izquierda y en mi misma fila”. Sestoft trata esa distinción como parte del núcleo del modelo y no como una comodidad de la interfaz (Sestoft 2014).

La consecuencia es que **una sola expresión, escrita una vez, puede aplicarse a un rango entero y significar algo distinto en cada fila**. No hay copia ni duplicación conceptual: es la misma regla, reinterpretada en cada posición.

En el corpus de este trabajo eso aparece de forma literal. En `departamento/hoja_cobertura.py`:184–196 se declara un formato condicional para todo el rango de datos con una sola condición, anclada en la primera fila:

```
rango = f"A{f}:{COL_ULTIMA}{fila_fin}"
ws.conditional_formatting.add(
```

```
rango, estilos.regla_formula(f'${COL_ESTADO}-{f}="{ESTADO_COMPLETA}"',
                             estilos.COLOR_COMPLETA))
```

La columna está fijada con \$ y la fila no. La regla se escribe una vez y Excel la evalúa en cada fila del rango ajustando el número de fila solo. El mismo patrón está en `horarios/hoja_docencia.py:92-102`, y la validación de datos se comporta igual: en `tribunales/hoja_dia.py:110-117` un único objeto de validación se asigna a un conjunto de celdas y rige en todas.

El contraste está en el otro patrón del corpus. En `tribunales/hoja_dia.py:40-43` y en `horarios/hoja_grupo.py:288-292` la fórmula se escribe **una vez por celda**, dentro de un bucle del generador en Python, con el número de fila interpolado en el texto. Ahí la iteración la hace el generador y es visible; en el formato condicional la hace Excel y es invisible.

El problema está en la iteración invisible. Cuando la hoja la aporta, quien diseña el lenguaje nunca se ve obligado a decir *sobre qué* se itera. La información —“esta regla se aplica a cada asignatura del plan”— no llega a escribirse, porque la geometría de la hoja la suple. Y lo que no está escrito no se puede traducir.

2.6.4. Por qué ese modelo no existe en otros destinos

HTML. Un documento HTML es marcado estático. No hay expresiones, no hay referencias entre nodos y no hay motor que recalculé nada. Reproducir las tres piezas exige escribir el programa que falta: una representación de los datos, un recorrido explícito sobre ellos y un mecanismo que lo vuelva a ejecutar cuando algo cambie. Ninguna se obtiene del formato.

Emacs Lisp. Como lenguaje de programación, elisp no tiene grafo de dependencias ni recálculo: si un valor depende de otro, hay que volver a calcularlo a mano. Cabe una salvedad, porque el modo `org-table` de Org sí ofrece fórmulas de celda y de columna, y estas últimas se aplican a todas las filas de la tabla. La coincidencia es parcial: el recálculo es bajo demanda, no automático, y la fórmula de columna rige sobre una columna entera, no sobre una posición relativa arbitraria. *(Esta comparación con org-table no se ha verificado contra una instalación de Emacs: en la máquina de trabajo no hay ninguna. Queda pendiente de comprobación.)*

SQL. Es el caso más cercano y el más instructivo. Una vista es una expresión pura sobre relaciones y se reevalúa al consultarla; eso cubre en parte las dos primeras piezas. Pero la unidad no es la celda sino la relación entera; el recálculo no ocurre al editar sino al consultar; y no hay nada equivalente a la tercera pieza. La iteración sobre filas es siempre explícita —está en el `FROM` y en el `WHERE`— y para hablar de filas vecinas hay que declarar un orden y usar una función de ventana. No se puede decir “la celda de arriba” porque no hay arriba.

2.6.5. Por qué apoyarse en el modelo sin declararlo contamina la representación

Una representación intermedia que aspire a ser independiente del destino tiene que contener toda la información que cualquiera de los destinos vaya a necesitar. La tercera pieza del

modelo hace que una parte de esa información —sobre qué colección se itera, en qué orden, con qué fila ligada— sea innecesaria *en Excel*, porque la geometría la aporta gratis. Si el diseñador del lenguaje solo tiene Excel delante, esa información no se echa en falta y no se escribe.

El resultado es una representación que parece completa y no lo es, y la carencia no se manifiesta hasta que aparece el segundo destino, cuando ya hay escrito un cuerpo de código que la da por supuesta. La contaminación es doble: por omisión, porque falta lo que Excel no obligó a declarar; y por presencia, porque el vocabulario de la hoja —fila, columna, rango, celda anterior— se filtra hacia la representación como si fuera vocabulario del dominio.

De ahí se sigue el criterio que este trabajo adopta y que se justifica en el capítulo de diseño: **la iteración es siempre explícita**, y no existe la noción de “la fila de arriba”; solo se puede hablar de la fila anterior cuando la colección declara por qué campo está ordenada, porque entonces “anterior” significa lo mismo en todos los destinos.

2.7. El formato OOXML y la materialización de libros

Un archivo `.xlsx` no es un formato binario opaco. Es un contenedor comprimido —un ZIP, en los términos del empaquetado de documentos abiertos— que agrupa un conjunto de partes XML: el libro, cada una de sus hojas, la tabla de cadenas compartidas, los estilos, los nombres definidos. El formato está especificado por ECMA-376 (*ECMA-376: Office Open XML File Formats* 2016).

Dentro de esas partes, todo es texto estructurado. Una fórmula se guarda como texto en un elemento de la celda, junto al último valor calculado que la aplicación dejó en caché. El formato condicional, la validación de datos, los rangos nombrados y la protección de hoja son, todos, elementos XML con sus atributos.

La consecuencia para este trabajo es una sola, y es de delimitación: **escribir un libro es un problema mecánico, no semántico**. Toda la dificultad interesante está en decidir qué dice cada celda; una vez decidido, serializarlo es empaquetar XML. Esto justifica que la escritura del archivo se delegue a una biblioteca existente sin que eso comprometa ninguna afirmación del trabajo: lo delegado no contiene ninguna decisión de significado. (*La afirmación complementaria —que no existe ninguna biblioteca de Common Lisp capaz de escribir .xlsx con fórmulas, formato condicional, validación, rangos nombrados y protección— fue comprobada en septiembre de 2026 y se documenta en el capítulo de implementación.*)

2.8. Hojas de cálculo como desarrollo por usuarios finales

La última noción que hace falta es sobre la naturaleza del artefacto generado.

Se llama **desarrollo por usuarios finales** a la actividad de quienes programan sin ser programadores de oficio, como medio para otra cosa y no como fin (Ko et al. 2011). La hoja de cálculo es su instrumento más extendido: quien escribe una fórmula está programando, aunque no lo llame así y no tenga formación para ello.

Esa literatura documenta dos hechos relevantes aquí. El primero es que las hojas de cálculo en uso real contienen errores con una frecuencia alta, y que las auditorías de campo los encuentran de forma sistemática (Panko 1998). El segundo es que esos errores tienen estructura reconocible y son detectables: se han propuesto técnicas para inferir la plantilla implícita en una hoja concreta (Abraham y Erwig 2006) y para localizar indicios de mal diseño en las relaciones entre hojas de un mismo libro (Hermans, Pinzger, y Deursen 2012).

Lo que esto aporta al planteamiento del trabajo es un encuadre. El documento que se genera **no es un informe**: es un programa, y además un programa que su destinatario va a modificar. Si fuera un informe —una foto de un estado de cosas— bastaría con volcar valores y la tarea sería de formateo. Como es un programa, lo que el generador entrega es un artefacto ejecutable que quedará en manos de alguien sin formación en programación, que lo editará durante meses.

De ahí que ciertas decisiones que parecen cosméticas no lo sean. Proteger las celdas calculadas evita que el usuario borre una fórmula sin advertirlo. Restringir el dominio de una celda de entrada evita que escriba un nombre que no existe. Y distinguir entre una restricción que impide y una que solo advierte es una decisión del dominio, no del formato: en el corpus, el desplegable de estudiantes se emite con `errorStyle="information"`, es decir, avisa y deja seguir (tribunales/hoja_dia.py:114-115). Todas ellas son defensas contra la clase de error que la literatura documenta, y todas ellas tienen que poder decirse en el lenguaje, porque forman parte de lo que el documento *es*.

Capítulo 3

Estado del arte

Este capítulo revisa el trabajo previo sobre el que se levanta esta tesis. Tiene dos mitades desiguales. La primera, y la más extensa, examina las tres tesis de diploma que en la Facultad de Matemática y Computación de la Universidad de La Habana abordaron el mismo problema: describir una situación en un lenguaje propio y generar a partir de esa descripción el producto en varios formatos. La segunda sitúa el trabajo frente al estado del arte externo —ingeniería dirigida por modelos, programación generativa, plataformas de bajo código— y frente a la alternativa que hoy cualquier tribunal plantea de entrada: usar un modelo de lenguaje en lugar de diseñar un lenguaje.

Cada uno de los tres trabajos anteriores se examina en dos tiempos: primero qué hace y qué aporta, con evidencia; después dónde está su límite, también con evidencia, señalando sección, definición o listado del documento correspondiente. El orden no es cortesía. Una crítica que no reconoce el mérito previo es una crítica sin credibilidad, y además sería falsa: los tres trabajos aportan piezas que esta tesis hereda y usa. Lo que se argumenta no es que estuvieran equivocados, sino que ninguno llegó a someter a prueba su afirmación central, y que en el caso del más reciente —el antecesor directo— la afirmación es demostrablemente falsa tal como quedó implementada.

Una advertencia de método. Todo el análisis de las tres tesis se ha hecho sobre los documentos publicados, no sobre sus repositorios de código, a los que no se ha tenido acceso. Cuando una afirmación es lectura directa de un listado o de una cita, se indica el listado o la sección. Cuando es interpretación propia —muy especialmente en el caso de la figura 6.2 de ([Morales Lazo 2026](#))— se dice expresamente en el texto que es interpretación y que queda pendiente de confirmar contra el código.

3.1. La línea de investigación de la facultad

Las tres tesis que se revisan a continuación comparten esqueleto técnico: lenguaje de dominio específico interno en Common Lisp, árbol de sintaxis abstracta construido con clases CLOS, y generación de salida mediante funciones genéricas con despacho sobre el nodo y, en el mejor de los casos, sobre el formato destino. Esa coincidencia no es casual: las tres heredan,

de forma declarada o no, la misma tradición de trabajo en la facultad, y las tres aprovechan la propiedad que hace de Common Lisp un anfitrión natural para esta clase de problema, a saber, que el propio intérprete se encarga del análisis léxico y sintáctico y que las macros funcionan como compilador del lenguaje (Graham 1993; Seibel 2005).

Se diferencian en el dominio que describen y en hasta dónde llegaron. La de 2017 no tiene dominio propio: construye la herramienta con la que se construyen estos lenguajes. La de 2025 describe microaplicaciones de gestión. La de 2026 describe estructuras tabulares, que es exactamente el dominio de este trabajo.

Conviene distinguir esta línea de otra línea de la facultad, mucho más larga y mucho más poblada, dedicada a la **generación automática de horarios**: dado un conjunto de restricciones, calcular una asignación válida de asignaturas, profesores, grupos, locales y turnos. Esa línea acumula varias tesis a lo largo de años y es un problema de optimización combinatoria. La línea en la que se inscribe este trabajo es otra: no calcula la solución, la **representa**. El tutor lo estableció de forma explícita al fijar el alcance, y queda por tanto fuera de este trabajo cualquier solucionador, optimizador o motor de restricciones. Sin esa distinción, un lector que conozca la facultad supondrá que se trata de la enésima tesis de horarios. Una línea *resuelve* la situación; la otra la *describe*, para que una persona la resuelva a mano y el documento la ayude a no equivocarse.

3.2. Automatizar la creación de Generadores Multilenguajes (2017)

El trabajo de (Arcia Corcho 2017) define la noción de **Generador Multilenguaje**: una aplicación que define un lenguaje de dominio específico y permite representar en múltiples formatos los programas escritos en él. Su objeto no es un dominio concreto sino la herramienta —que llama MGA— con la que se construyen estos generadores. Su ejemplo de trabajo son operaciones aritméticas, deliberadamente trivial: lo que quiere enseñar es el andamiaje, no el edificio.

3.2.1. La metodología de los seis pasos

El trabajo identifica seis pasos para construir un generador multilenguaje. Se citan literalmente porque son el marco metodológico que esta tesis adopta:

1. Determinar el dominio que se quiere representar en el DSL.
2. Identificar los elementos que se desean representar en el DSL.
3. Diseñar el lenguaje del DSL.
4. Implementar una jerarquía de clases que represente cada elemento como nodo de un árbol.
5. Construir el analizador lexicográfico y el sintáctico.
6. Implementar la generación de código de cada nodo, en cada lenguaje de salida.

Y observa lo que justifica toda la decisión técnica de esta línea de trabajo: si el DSL es **interno** en Common Lisp, el paso 5 desaparece —lo hace el intérprete—, el paso 3 se reduce

a elegir bien los nombres de los constructores, y los pasos 4 y 6 se pueden automatizar con macros. De seis pasos quedan tres, y dos de los que quedan son de pensar, no de teclear. Esta observación coincide con la caracterización clásica del DSL embebido (Hudak 1996) y con el análisis de costes de decidir entre sintaxis propia y sintaxis anfitriona (Mernik, Heering, y Sloane 2005; Fowler 2010).

3.2.2. defnode

La primera de las dos macros de MGA automatiza el paso 4. Sin ella, declarar un nodo de suma con dos operandos exige escribir la clase y su constructor a mano:

```
(defclass sum-node ()
  ((left-hand :accessor left-hand :initarg :left-hand)
   (right-hand :accessor right-hand :initarg :right-hand)))

(defun sum-node (left-hand right-hand)
  (make-instance 'sum-node :left-hand left-hand :right-hand right-hand))
```

Con la macro, la misma declaración cuesta una línea:

```
(defnode sum-node () (left-hand right-hand))
```

`defnode` rellena por defecto `:accessor`, `:initarg`, `:initform` y `:documentation`, y admite sobrescribirlos con `:func-name`, `:lambda-list` y `:ctr-body`. El efecto práctico es que declarar un concepto nuevo del dominio cuesta una línea en lugar de un bloque de clase más su constructor. Las tesis de 2025 y 2026 usan una macro equivalente, que ambas llaman `defclass*`. Conviene precisar que **ninguna de las dos cita a (Arcia Corcho 2017)**: la continuidad es funcional, no declarada, y si se quiere usar la genealogía como argumento hay que establecerla, no darla por hecha.

3.2.3. gcode y recognize-pattern

La segunda macro automatiza el paso 6. `gcode` encapsula el patrón repetido de escribir a mano un `defmethod generate-code` (nodo lenguaje stream). En lugar del método entero, se declara una plantilla:

```
(gcode sum-node csharp "~a+~a" left-hand right-hand)
```

que se lee: *un nodo de suma, en C#, se escribe como el operando izquierdo, un signo de más, y el operando derecho*. El trabajo presenta dos versiones de la macro: la heredada, que solo sirve cuando la traducción encaja exactamente en ese patrón de plantilla, y la propia de MGA, generalizada para admitir los casos que no encajan. La función `recognize-pattern` completa el mecanismo: permite registrar símbolos que `gcode` sustituye por código en todas las traducciones futuras, y es así como se automatiza, por ejemplo, la indentación obligatoria de Python sin tocar ninguna regla ya escrita. Al mismo fin sirven los métodos auxiliares `:before`, `:after` y `:around` de CLOS (Keene 1989), que el trabajo usa para características transversales: se marca un nodo con una propiedad y un método aparte se ocupa de ella en

todos los casos.

El propio trabajo es honesto sobre el límite del mecanismo: el uso de `gcode` deja de ser factible cuando la generación del código de un nodo no cumple exactamente el patrón de plantilla. Queda entonces la válvula de escape de escribir el método a mano.

3.2.4. La jerarquía de lenguajes, y a quién pertenece

La idea más valiosa que aparece en este trabajo es la **jerarquía de lenguajes por herencia múltiple** (sección 3.3.2 y figura 3.1). La forma ingenua de soportar cinco lenguajes de salida es escribir, para cada concepto, cinco reglas. La forma buena es advertir que los lenguajes comparten propiedades, y nombrar las propiedades en vez de los lenguajes: `Infix-Language` escribe `2 + 3` y no `(+ 2 3)`; `Indent-Language` marca los bloques con espacios; `Symbol-Separated-Language` termina las instrucciones con un símbolo; `Basic-Operation-Language` dispone de las operaciones aritméticas y relacionales habituales. Entonces un lenguaje concreto se declara como composición: `C-like` hereda de las cuatro, y añadir Java o `C#` consiste en heredar de `C-like` y escribir solo lo que se hace distinto.

Dos precisiones de atribución que importan, y mucho, para la defensa de esta tesis.

La idea no es de (Arcia Corcho 2017). Su propio documento la atribuye expresamente a `ADOL*` y `GEMURAL`, dos trabajos anteriores de la facultad. La formulación del documento es que un subconjunto de las propiedades sintácticas de los lenguajes se representa *en ADOL** con las clases que a continuación enumera. Citar esta jerarquía como aporte de 2017 sería un error de atribución, y en una defensa un error así es un regalo para el tribunal.

Y el trabajo no muestra el código de la composición. En todo el documento no aparece ni una sola clase de lenguaje con varias superclases: ni `C-like`, ni `Infix-Language`, ni `Basic-Operation-Language` implementadas. La jerarquía está descrita en prosa y dibujada en la figura 3.1. Lo que sí implementa, en la sección 4.2, es **una** de esas propiedades por separado, la indentación:

```
(defnode indentable-language () (indentation indent-increment))
```

más la macro `mark-node-as-indentable` y un método `:before` especializado en el par (nodo, lenguaje indentable). Es decir: el trabajo enseña cómo automatizar una propiedad, no cómo componer varias por herencia múltiple. Esa composición está sin demostrar en toda la línea de investigación. Esta tesis la hereda, la aplica a un problema distinto —clasificar arquitecturas de salida por lo que saben hacer— y, sobre todo, la implementa.

3.2.5. Qué se hereda y qué no aporta

Se heredan la metodología de los seis pasos como marco, `defnode` en su versión moderna, `gcode` y `recognize-pattern` con los métodos auxiliares para reglas transversales, y la jerarquía por herencia múltiple, que es el fundamento del mecanismo de capacidades que este trabajo construye.

Lo que no aporta hay que decirlo con la misma claridad: **no tiene dominio propio, no**

tiene casos de uso reales y no tiene validación. Es una herramienta, no una aplicación. Como tal es cimiento y no competencia: nada de lo que hace compite con lo que esta tesis propone, y buena parte de lo que hace se reutiliza.

3.3. LDMAg (2025)

(Cápiro Colomar 2025) define **microaplicación de gestión** (MAG) como aplicación informática destinada a gestionar procesos en los cuales toda la información relevante cabe en una cantidad pequeña de unidades de almacenamiento, e identifica tres componentes: el almacenamiento de la información, la definición de las consultas y la representación visual. El lenguaje LDMAg describe los tres y exporta a tres formatos: consola de Python, web con Django y MySQL. Se apoya en cinco procesos de ejemplo: tres de la facultad —asignación de alumnos ayudantes, planificación de defensas de tesis, asignación de la docencia— y dos de ámbito general —control de acontecimientos relevantes y administración de notas de libros—.

3.3.1. El elemento consulta

El aporte real del trabajo, y el que esta tesis hereda, es el **elemento consulta**. Es el reconocimiento de que un documento de gestión no es un volcado de datos, sino los datos **más las preguntas sobre esos datos**. “¿Cuántas defensas tiene asignadas este profesor?” no es algo que se consulte aparte del documento: es parte del documento, tan parte como la lista de defensas. Un lenguaje que solo sabe describir la lista describe medio documento.

La descomposición que el trabajo hace de una consulta sigue valiendo entera: selección de resultados, acceso a atributos, iteraciones sobre entidades, operadores lógicos, operadores de comparación y valores atómicos. Esta tesis la hereda y la amplía por donde a LDMAg le faltó, que se detalla más abajo.

3.3.2. def-entidad y la lectura en español

La macro **def-entidad** genera automáticamente las funciones de acceso a los atributos de una entidad. De

```
(def-entidad defensa (estudiante tutor oponente))
```

salen accesorios por cada atributo, y no uno sino **dos por atributo**, con las dos variantes gramaticales:

```
(estudiante-del-defensa d)
(estudiante-de-la-defensa d)
```

Además, el lenguaje sabe pluralizar por sí solo el nombre de la entidad. El propósito es que la descripción **se lea como español**, y es un propósito bien visto: decide si el usuario del lenguaje puede leer una descripción y juzgar si es correcta sin que se la expliquen. Ése es el criterio de calidad que esta tesis hereda, y ninguna prueba automática lo puede sustituir.

Conviene señalar, sin exagerar su importancia, que la macro genera las dos variantes a ciegas, sin conocer el género del sustantivo. Como “defensa” es femenino, la primera de las dos formas —“del defensa”— está mal escrita. Es un descuido menor: la forma correcta existe y es la que se usa. Lo que se hereda es el propósito, no el descuido.

El trabajo añade otra comodidad barata en un DSL interno y de efecto grande en la lectura: **sinónimos de constructor**. **se-cumple-simultaneamente** y **al-mismo-tiempo** construyen ambos el mismo nodo de conjunción. En un lenguaje cuya sintaxis es la del anfitrión, registrar un nombre más cuesta una macro y mejora de forma apreciable la legibilidad de las descripciones.

3.3.3. El modelo visual no tiene tabla

Los elementos visuales del lenguaje son `pagina`, `titulo`, `texto`, `lista`, `item` e `input` (secciones 3.2.1 a 3.2.5). **No hay tabla**. Para un lenguaje de microaplicaciones de gestión cuyos cinco ejemplos del capítulo 2 se presentan todos como tablas —cuadros 2.1 a 2.5— eso no es un olvido de detalle: es una omisión estructural. El lenguaje no puede representar la forma en que sus propios casos de uso se presentan.

3.3.4. Los operadores no alcanzan para las preguntas del capítulo 2

Éste es el hallazgo más instructivo del trabajo, y el que más condiciona el diseño de esta tesis. El capítulo 2 de (Cápiro Colomar 2025) plantea, como consultas objetivo de sus casos de uso, preguntas de esta forma:

¿Cuántos alumnos ayudantes tiene asignados un profesor determinado?

¿Cuántas defensas tiene asignado un profesor como miembro del tribunal?

Y el lenguaje del capítulo 3 dispone exactamente de estos operadores (secciones 3.2.9 y 3.2.10): igualdad, desigualdad, conjunción, disyunción, negación e iteración sobre entidades. **No hay conteo. No hay suma. No hay agrupamiento**. A esas tres operaciones —resumir muchas filas en un solo valor— se las llama *agregación*, y son la mitad de cualquier documento de gestión. Sin agregación, ninguna de las dos preguntas citadas es expresable en el lenguaje.

El requisito está enunciado en el capítulo 2, el lenguaje del capítulo 3 no lo cumple, y el documento no lo señala en ninguna parte.

3.3.5. La conexión consulta-vista quedó sin cerrar

El trabajo identifica tres componentes —almacenamiento, consultas, representación visual— y nunca conecta los dos últimos. Las páginas de resultado de la sección 3.3 están así en el documento, citadas completas:

```
(page mostrar-estudiantes
  (title "Listado de estudiantes y sus grupos")
  (text "A continuación se muestran todos los estudiantes junto a
        su grupo.")
```

```
(lista
    ;; Aquí se mostraría el resultado de la consulta
    ;; mostrar-estudiantes
))
```

El comentario no es un descuido de una página: se repite en las tres páginas de consulta de esa sección. No existe ninguna forma, en el lenguaje, de decir “*esta lista se llena con el resultado de esta consulta*”. El nudo central del sistema —unir los tres componentes que el propio trabajo identificó— quedó sin atar.

3.3.6. El código generado

En la exportación a Django (sección 4.2.2) el modelo generado declara el atributo como clave foránea:

```
class asignaciones(models.Model):
    estudiante = models.ForeignKey(estudiantes, on_delete=models.CASCADE)
```

y la vista generada hace lo siguiente:

```
def listar_asignaturas_impartidas_por_un_estudiante(request):
    estudiante_arg = request.GET.get('estudiante_arg')
    result = []
    asignaciones__ = asignaciones.objects.all()
    for asignacion in asignaciones__:
        if asignacion.estudiante == estudiante_arg:
            result.append((asignacion.asignatura))
```

Hay dos problemas independientes. `asignacion.estudiante` es una instancia de modelo y `estudiante_arg` es una cadena venida de la cadena de consulta: la comparación es siempre falsa. Y el recorrido se hace sobre `objects.all()` en Python en lugar de filtrar con el ORM, con lo que se pierde justamente lo que aportaría usar Django.

3.3.7. No hay capítulo de experimentación ni de validación

El índice del documento va del capítulo 5 —detalles de implementación— directo a las conclusiones. Las conclusiones afirman que se comprobó la utilidad práctica del lenguaje mediante la generación automática de aplicaciones en distintos formatos, sin presentar la comprobación: no hay caso completo, ni captura, ni métrica, ni traza de ejecución. Las propias recomendaciones del trabajo piden realizar una validación práctica del lenguaje.

3.3.8. La lección

La lección que esta tesis extrae de (Cápiro Colomar 2025) es una sola, y es la que más condiciona su método de trabajo: **el lenguaje se diseñó contra ejemplos escritos en prosa, no contra artefactos reales**. Sin un caso real completo que obligue a cerrar cada cabo, el diseño se detiene justo antes de la parte difícil: las consultas nunca se enganchan a las vistas porque ningún artefacto obligó a engancharlas; la agregación nunca se añade

porque ninguna tabla real hubo que reproducir. El antídoto del que este trabajo dispone, y que ninguna de las tres tesis anteriores tuvo, es un corpus de documentos en uso que sirve a la vez de fuente de vocabulario y de oráculo de validación.

3.4. Un DSL para la descripción declarativa de estructuras tabulares (2026)

([Morales Lazo 2026](#)) es el antecesor directo de este trabajo y, con diferencia, el más sólido de los tres. Conviene decirlo con claridad antes de criticarlo, porque es cierto y porque una crítica que no lo reconozca no merece crédito: es el único que tiene el dominio correcto, el único que presenta casos de uso reales de la facultad, el único que llega a generar artefactos que se abren y funcionan, y el que formula la pregunta científica que esta tesis recoge y cierra.

3.4.1. Qué hace

El trabajo aporta cuatro cosas. Una **taxonomía** del dominio de la planificación tabular en el capítulo 3: tabla, plantilla, fila, columna, celda, columna calculada, rango de columnas, referencia cruzada, parámetro, regla de marcado condicional, cuantificador existencial, región horizontal, posicionamiento relativo, hoja y libro, más los nodos de navegación y de referencia cruzada entre hojas. Un **árbol de sintaxis abstracta** implementado con clases CLOS. **Tres casos de uso reales**: el horario docente de la facultad, una parrilla de programación televisiva y el horario de defensas de diploma. Y una **arquitectura de salida**: Excel.

Sobre el número de conceptos conviene un apunte de rigor, porque el documento se cita a sí mismo. Las conclusiones hablan de 17 conceptos identificados; el capítulo 3 numera 13 definiciones, de la 3.1 (Tabla) a la 3.13 (Libro), y describe otros 4 solo en prosa. La cuenta cuadra si se suman ambos grupos, pero si el número se cita hay que decir cómo se cuenta.

La arquitectura de generación está en **dos fases**: un generador escrito en Lisp recorre el árbol y escribe un archivo JSON; un módulo en Python (`hoja_con_formulas.py`) lee ese JSON y llama a `openpyxl` para fabricar el `.xlsx`. Retener ese JSON es imprescindible para entender la crítica que sigue.

3.4.2. Qué se hereda

La taxonomía del dominio. Es un buen punto de partida y se hereda, se amplía y se corrige donde haga falta. No se reinventa.

El cuantificador existencial. Es lo mejor de las tres tesis. Captura de forma declarativa la clase entera de problema “detectar conflictos”, que es el corazón de todos estos documentos. El trabajo presenta dos variantes: una que examina otra tabla —variante de tabla cruzada— y otra que examina la misma tabla. La que sigue es la **variante de tabla cruzada**, escrita con la macro `exists` (listados 4.18 y 4.19):

```
(def-table profesores-table
  ((nombre "") (defensas ""))
  :render
```

```
(conditional-rendering
  :condition
    (exists (r :rows-of defensas-table)
      :self-in (tutor presidente secretario vocal oponente)
      :matching (dia hora))
  :target-columns (nombre)))
```

La expresión vive dentro de una regla que se evalúa una vez por cada fila de la tabla de profesores, de modo que “yo” es la fila que se está evaluando. Pieza a pieza: `(exists (r :rows-of defensas-table))` busca entre las filas de la tabla de defensas alguna fila `r`; `:matching (dia hora)` exige que coincida conmigo en día y hora; `:self-in (tutor presidente secretario vocal oponente)` exige que yo aparezca en cualquiera de esos cinco roles. Es decir: *marca a este profesor si está citado en dos defensas a la vez. Y no menciona Excel por ningún lado.*

La fórmula de Excel que esta variante produce (sección 5.2.3, listado 5.6) es, abreviando los sumandos como los abrevia el propio documento:

```
SUMPRODUCT( ((D$4:D$16=$K5)+...+(H$4:H$16=$K5))
  * ((COUNTIFS($G$4:$G$16,$G$4:$G$16,
    $H$4:$H$16,$H$4:$H$16,
    $D$4:$D$16,$K5) + ...) > 1) ) > 0
```

Puestas una al lado de la otra, las dos expresiones dicen lo mismo y solo una de las dos se puede leer. Ése es el listón que esta tesis se pone para todo su lenguaje, y la pieza concreta que hereda.

La plantilla instanciable. Declarar la estructura una vez e instanciarla con datos distintos, con parámetros de instancia (`:inst-params (dia)`). Es lo que evita repetir diecisiete veces la misma declaración para diecisiete grupos.

La separación entre identificador interno y etiqueta visual. La columna se llama `tutor` por dentro y muestra "Tutor" por fuera (`col-def` con `name` y `header`). Correcto y necesario.

La arquitectura en dos fases, como idea. Lo que está mal no es que haya una frontera entre el trabajo semántico y el mecánico; es dónde se puso esa frontera.

Lo que sigue son los límites, uno por apartado, con la evidencia de cada uno.

3.4.3. El mecanismo de extensión no cubre el cálculo

Éste es el límite más importante y el que justifica una tesis nueva en lugar de una continuación. El trabajo define dos funciones genéricas de traducción (sección 5.1.1, listados 5.1 y 5.2). Se copian literalmente:

```
(defgeneric compile-excel-formula
  (nodo col-map data-names fila primera-fila ultima-fila)
  (:documentation
```

```

"Traduce un nodo de expresion a la cadena de notacion Excel.
  col-map      : alist (simbolo-columna . letra-excel)
  fila        : numero de fila que se esta generando
  primera-fila : primera fila de datos de la tabla actual
  ultima-fila  : ultima fila de datos de la tabla actual"))

(defgeneric generate-code (nodo lang stream)
  (:documentation
   "Serializa un nodo estructural como JSON en el flujo indicado.
    lang  : instancia de out que identifica el formato destino
    stream : flujo de salida al que se escribe el JSON"))

```

Obsérvese la asimetría: `generate-code` recibe `lang`; `compile-excel-formula` no. El documento explica que la clase `out` es lo que CLOS usa para seleccionar los métodos correctos cuando existe más de un backend. Pero eso solo se aplica a `generate-code`, es decir, a los nodos **estructurales**: libro, hoja, región, tabla. Los nodos de **expresión** —que son toda la sustancia del lenguaje: columnas calculadas, comparaciones, agregaciones, referencias cruzadas y el cuantificador existencial que se acaba de elogiar— se traducen mediante una función genérica que no despacha sobre el formato destino y que se llama, literalmente, “compilar fórmula de Excel”.

De ahí salen dos consecuencias, y las dos son graves.

Primera: añadir una arquitectura de salida obliga a tocar lo ya escrito. La sección 5.1.1 afirma que solo se implementó el backend Excel, pero que *la estructura admite añadir otro —por ejemplo, un generador de tablas HTML— definiendo una nueva clase y nuevos métodos especializados en ella, sin modificar los existentes*. Para los nodos estructurales, la afirmación es cierta. Para los nodos de expresión, no: habría que añadir a `compile-excel-formula` el parámetro del destino, lo que cambia su lista de parámetros y obliga a revisar todos sus métodos ya escritos. Es exactamente lo que la arquitectura prometía evitar.

Segunda, y más profunda: el punto de extensión está formulado en términos de Excel. `col-map` es una lista de asociación de símbolo de columna a **letra de Excel**; `fila`, `primera-fila` y `ultima-fila` son **números de fila**. La interfaz por la que un tercero añadiría su arquitectura presupone una rejilla bidimensional con direccionamiento A1. No es que el backend de Excel sea el único implementado —eso se arreglaría con tiempo—; es que el formulario que hay que rellenar para entrar solo tiene casillas de hoja de cálculo. Una arquitectura web no tiene letras de columna ni números de fila, y no hay nada sensato que pueda escribir en esos parámetros.

Esto convierte el diagnóstico fácil —“implementó un solo backend, así que la extensibilidad está sin probar”, objeción a la que siempre se puede responder que faltó tiempo— en otro mucho más fuerte: **la extensibilidad está demostrablemente mal puesta**, y eso es una base firme para justificar un trabajo nuevo.

Queda pendiente de verificación contra el repositorio de código del trabajo la firma real del `defgeneric`, que aquí se cita del documento publicado.

3.4.4. El árbol no tiene jerarquía

La idea de ADOL* que ([Arcia Corcho 2017](#)) transmite —componer destinos por herencia múltiple de propiedades— no se usa en absoluto en este trabajo, ni para los nodos ni para las arquitecturas. Todas las clases del árbol declaran **superclases vacías**: los `defclass*` de los listados 4.1, 4.14 y siguientes abren con `()`. No hay clase base común. Sin clase base común no hay métodos por defecto, no hay comportamiento heredado y no hay forma barata de decir “todos los nodos de comparación se traducen así salvo estos dos”.

En el mismo sentido, **la clase que identifica el backend nunca se define**. La clase `out`, que según la sección 5.1.1 es la que permite a CLOS seleccionar los métodos correctos cuando hay más de un destino, se describe en prosa y se instancia una vez; su definición no aparece en el documento.

Y el efecto acumulado se ve en una cifra: **hay un solo `defmethod` concreto en todo el trabajo**, y es precisamente el que busca la letra de columna en el diccionario y le pega el número de fila. La maquinaria de despacho múltiple de CLOS ([Keene 1989](#); [Kiczales, Rivières, y Bobrow 1991](#)), que es la razón técnica de haber elegido Common Lisp, queda prácticamente sin usar.

Como en el apartado anterior, esto se ha leído del documento publicado y hay que confirmarlo contra el código.

3.4.5. El JSON de traspaso ya es Excel

La frontera entre la fase Lisp y la fase Python es un archivo JSON. Lo que el generador entrega al módulo de Python, para una regla de coloreado, es esto (listado 5.14, reproducido completo):

```
{
  "range": "B4:I16",
  "formula": 「
    ↪ "SUMPRODUCT(($G$4:$G$16=$G4)*($H$4:$H$16=$H4)*(ROW($A$4:$A$16)<>ROW($A4))*(($B$4:$B$16=$B4)
    ↪ ,
  "style": {"font_color": "#FF0000"}
}
```

Rango absoluto en notación A1, letras de columna y sintaxis de fórmula de Excel. Eso no es una descripción del cálculo: es el cálculo ya compilado. Y sin embargo la sección 5.4.4 afirma:

Si se quisiera generar tablas HTML en lugar de hojas de cálculo, bastaría escribir un nuevo módulo que leyera el mismo JSON y produjera etiquetas `<table>`, sin modificar una línea del generador Lisp.

La afirmación es cierta para los encabezados y las celdas de datos literales, y falsa para todo lo que tenga columna calculada o marcado condicional, que es lo único interesante. Un módulo HTML que leyera ese JSON recibiría una cadena de caracteres que solo un motor de hoja de cálculo sabe interpretar, y tendría que escribir un intérprete de fórmulas de Excel para hacer algo con ella.

El problema no es que existan dos fases. Es **dónde está la frontera**. Esta tesis hereda la idea de partir el trabajo en dos y mueve la frontera de sitio.

3.4.6. Conceptos de rejilla dentro del árbol

En el árbol —que se presenta como independiente del destino— viven **region**, **nav**, **anchor**, **cell-height** y **cell-width**. Los cinco solo significan algo si hay una rejilla de filas y columnas con coordenadas. **region** delimita un bloque rectangular; **anchor** fija dónde empieza; **nav** coloca un elemento de navegación contando filas; **cell-height** y **cell-width** son dimensiones de celda.

El propio trabajo lo concede. Al definir el posicionamiento relativo, la sección 3.2.10 dice, tras la definición 3.11:

Este concepto supone que el backend puede organizar sus elementos en coordenadas bien definidas, como ocurre en una hoja de cálculo. Un backend cuyo formato de salida no admita ese tipo de posicionamiento podría simplemente prescindir de esta funcionalidad.

Reconocer la dependencia es honesto y hay que apuntarlo a favor del trabajo. Pero reconocerla no la elimina, y “prescindir de la funcionalidad” no es un mecanismo de degradación: es una nota al pie. Una arquitectura que prescinde de la navegación entrega un documento sin navegación, y nadie queda informado de que eso ocurrió.

Conviene distinguir aquí, porque la distinción es fina y esta tesis la explota: **region** y **anchor** están formulados como rejilla y salen del lenguaje. La navegación **como concepto de dominio** —que un documento tiene secciones, un punto de entrada y una forma de moverse entre ellas— sí es un hecho del problema y entra, rehecha; lo que no entra es el nodo que coloca un elemento contando filas desde el final de una columna.

3.4.7. `expr-counta`

El árbol incluye **expr-counta**, cuya semántica es **contar celdas no vacías**. Eso es una noción de hoja de cálculo, no del dominio. En el dominio la pregunta es *cuántas filas hay o cuántas filas cumplen tal condición*, y una fila existe o no existe. “Celda vacía” solo significa algo donde hay una rejilla preasignada con huecos, y esa rejilla es una invención del destino, no un hecho del problema que se describe.

Es el caso de estudio perfecto de un concepto mal levantado, y sirve de prueba de fuego para cualquier vocabulario que se proponga: si al nombrar un concepto hay que mencionar una celda, el concepto pertenece al destino y no al dominio. Nótese que la noción contraria —*este campo de entrada todavía no se ha rellenado*— sí es un hecho del dominio y sí tiene que poder expresarse; lo que no lo es, es contar huecos de una rejilla.

3.4.8. La regla de marcado perdió el color

La **definición 3.8** establece que una regla de marcado condicional tiene dos partes: una condición booleana y un **color de fondo**. El nodo del árbol, listado 4.14, es este:

```
(defclass* style-rule ()
  (rule-condition      ;; nodo de expresion booleana
   target-columns      ;; columnas cuyas celdas se colorean
  ))
```

No hay color. Y la macro `conditional-rendering` (listado 4.16) solo recibe `:condition` y `:target-columns`. En el caso de uso real del horario docente (listado 6.4) las tres reglas quedan distinguidas únicamente por **comentarios** y por su orden:

```
:renders
((conditional-rendering
  :condition (gt (col asignadas) (col frec))
  :target-columns (asig))      ; excedida -> naranja
 (conditional-rendering
  :condition (equals (col asignadas) (col frec))
  :target-columns (asig))      ; exacta -> verde
 (conditional-rendering
  :condition (lt (col asignadas) (col frec))
  :target-columns (asig)))      ; falta -> rojo
```

El color aparece por primera vez aguas abajo, dentro del JSON que produce el generador (`"style": {"font_color": "#FF0000"}`), y el documento **no explica en ninguna parte de dónde sale ese color ni cómo se asocia a cada regla**. Hay además una contradicción interna: en el listado 4.25 sí aparece un `:color verde` dentro de `def-table`.

Esto importa más de lo que parece. El marcado *es* la visualización: es lo único que convierte una tabla en un instrumento de detección de problemas. Si el color no está en el lenguaje, el lenguaje describe media situación.

Y la propia figura de validación del trabajo parece enseñar el fallo ocurriendo.

En la figura 6.2 la asignación de colores está corrida entera:

- La fila AM I (Frec 2, Asignadas 3, Faltan -1) sale **roja**, cuando la regla a la que el comentario asigna el rojo es `asignadas < frec`, que con 3 y 2 es falsa. Por los comentarios debería ser naranja.
- Las filas ICD y F (Frec 2, Asignadas 1, Faltan 1), que sí cumplen la regla del rojo, salen **azules**: un color que no aparece en ninguna de las tres reglas.
- Solo el verde de la regla de igualdad cae donde le corresponde.

La lectura natural de ese patrón es que los colores se aplican **en el orden de las reglas** —primera, rojo; segunda, verde; tercera, azul— y no según el significado que los comentarios les atribuyen. **Se declara expresamente que esto último es interpretación propia de una imagen, no una afirmación del documento de (Morales Lazo 2026)**, y que **queda pendiente de confirmar regenerando el libro y examinando el archivo, no la figura del PDF**. Si el código lo confirma, el argumento se sostiene solo; si no lo confirma, se retira sin que el resto del apartado se resienta, porque la ausencia del color en el nodo es un hecho del listado 4.14 y no depende de ninguna lectura de imagen.

El diseño que este episodio sugiere es mejor que el de 2026 y mejor incluso que el de los documentos que hoy se entregan a mano: **la regla no declara un color, declara un significado**. Que cada arquitectura decida cómo se ve: en Excel un relleno, en una web un color más un icono, en Emacs una *face*, y en una impresión en blanco y negro **necesariamente texto**, o no se ve nada. El color no sobrevive al cambio de arquitectura; el significado sí.

3.4.9. La afirmación central nunca se sometió a prueba

La pregunta científica de (Morales Lazo 2026) es, textualmente:

¿Es posible, mediante el diseño de un lenguaje de dominio específico, separar la descripción semántica de una estructura tabular del mecanismo concreto de su generación, de modo que el mismo programa declarativo permanezca válido con independencia del backend de salida?

Se implementó **una** arquitectura de salida. El experimento que respondería esa pregunta —materializar la misma descripción en dos destinos con modelos de evaluación distintos— no se ejecutó nunca. Y por lo descrito en los apartados anteriores, de haberse ejecutado la respuesta habría sido negativa: el punto de extensión de las expresiones no admite un segundo destino sin cambiar su firma, y lo que cruza la frontera de las dos fases ya es Excel.

Esto no desmerece el trabajo. Es un buen trabajo que dejó abierta exactamente la pregunta que ahora hay que cerrar, y sus propias recomendaciones piden explícitamente arquitecturas de HTML y de SQL y una herramienta de validación estática del árbol. Hay continuidad declarada, no ruptura.

3.5. Comparación entre los tres trabajos

	2017 (Arcia Corcho 2017)	2025 (Cápiro Colomar 2025)	2026 (Morales Lazo 2026)
Objeto	La herramienta para construir generadores	Microaplicaciones de gestión	Estructuras tabulares
Dominio propio	No	Sí, en prosa	Sí, con casos reales
Tabla en el lenguaje	No aplica	No	Sí
Consultas sobre los datos	No aplica	Sí, su aporte	No como elemento
Agregación (contar, sumar)	No aplica	No	Sí
Marcado con significado	No aplica	No	Parcial: sin color en el nodo
Entrada validada	No aplica	input sin dominio	No
Jerarquía de clases	Descrita, de ADOL*, no compuesta	No	No: superclases vacías

	2017 (Arcia Corcho 2017)	2025 (Cápiro Colomar 2025)	2026 (Morales Lazo 2026)
Destinos implementados	Varios (ejemplo trivial)	Tres (consola, Django, MySQL)	Uno (Excel)
Punto de extensión del cálculo	gcode genérico	No separado	Formulado en Excel
Capítulo de validación	No	No	Sí, con capturas
Verificación reproducible	No	No	No

La observación que resume el cuadro es ésta: **2025 tiene consultas sin tablas ni agregación; 2026 tiene tablas y agregación sin consultas.** Cada una dispone de la mitad de lo que hace falta para describir un documento de gestión real, y nadie juntó las dos mitades. Un documento de gestión es una tabla —o varias— y las preguntas que se le hacen a esa tabla; separar ambas cosas produce, en un caso, un lenguaje que no puede dibujar sus propios ejemplos, y en el otro, un lenguaje que dibuja la tabla pero no puede decir cuántas filas cumplen una condición sin bajar al nivel de la fórmula. A esto se suma una carencia que no es una casilla del cuadro sino una ausencia compartida: ninguna de las tres declara qué tiene que saber hacer un destino para que la descripción siga siendo válida en él. Ése es el asunto del apartado siguiente.

3.6. El punto ciego compartido: el modelo de evaluación

Ninguna de las tres tesis define su **modelo de evaluación**, y es precisamente lo que hace falta para que “cualquier arquitectura” signifique algo preciso en lugar de ser una aspiración.

Una hoja de cálculo tiene un modelo de evaluación muy concreto y bastante particular ([Sestoft 2014](#)). Tres piezas:

1. Cada celda es una **expresión** sobre otras celdas.
2. Un **motor de dependencias** recalcula las celdas afectadas cuando algo cambia.
3. Una **única expresión con referencias relativas** se aplica a todas las filas de un rango, y el motor ajusta las referencias fila a fila.

La tercera pieza es la traicionera: permite escribir una sola vez `J4>=$B$1` y que el motor evalúe `J5>=$B$1` en la fila siguiente, sin que nadie haya declarado nunca respecto a qué está ordenada la tabla ni qué significa “la fila siguiente”. Las tres tesis se apoyaron en ese comportamiento **sin declararlo**, y ésa es la explicación de por qué el árbol se les fue llenando de conceptos de hoja de cálculo sin que mediara ninguna decisión consciente: `col-map` con letras de columna, `region` y `anchor`, `expr-counta`, el JSON con rangos A1. No es descuido de sus autores; es lo que ocurre cuando el modelo de evaluación del único destino disponible se da por supuesto.

Declarar el modelo arriba —**valores con nombre, valores derivados de otros, recálculo**

ante cambios— convierte “ser una arquitectura válida” en una condición verificable, y permite clasificar honestamente qué cumple cada destino:

Arquitectura	Rejilla	Recálculo	Entrada	Dominio de entrada	Orden de declaración	Uno a muchos	Distintos	Navegación
Excel / Calc	sí	vivo	sí	sí, avi-sando	por columna	emulado	emulado	pestañas y enlaces
Web	no	vivo	sí	sí	sí	sí	sí	rutas
Emacs (org-table)	sí	bajo demanda	sí	sí	por columna	emulado	emulado	buffers
SQL	no	al consultar	parcial	parcial	sí	sí	sí	no
PDF / HTML estático	no	congelado	no	no	sí	sí	sí	enlaces

Este cuadro se presenta como **contribución teórica** del trabajo, y conviene justificar por qué merece ese nombre. Primero, porque no describe herramientas sino propiedades: cada columna es una propiedad del modelo de evaluación y cada fila es una composición de propiedades, que es exactamente la forma de la jerarquía por herencia múltiple heredada de ADOL* vía (Arcia Corcho 2017). Segundo, porque de él se deriva directamente un mecanismo de degradación: si una arquitectura no cumple una propiedad, el sistema sabe de antemano que va a tener que emular, degradar o rechazar, y puede dejarlo por escrito en vez de producir un artefacto silenciosamente incompleto. Y tercero, porque es falsable: cada casilla es una afirmación comprobable sobre un destino concreto, y una casilla mal puesta se detecta implementando ese destino.

La fila del PDF explica además por qué la pregunta “¿las derivaciones son vivas o congeladas?” no es un detalle de implementación: es lo que determina si un formato sin recálculo es una arquitectura de primera clase o un caso degradado declarado como tal.

El interés de declarar el modelo no es solo arquitectónico. La literatura sobre errores en hojas de cálculo es abundante y consistente en una conclusión: los documentos de cálculo contruidos a mano contienen errores en proporción alta y esos errores son difíciles de detectar por inspección (Panko 1998), las dependencias entre hojas generan estructuras que se degradan con el tiempo (Hermans, Pinzger, y Deursen 2012), y la estructura implícita de un libro—qué es plantilla y qué es dato— puede inferirse pero rara vez está declarada (Abraham y Erwig 2006). Generar el documento a partir de una descripción declarativa, en lugar de editarlo a mano, ataca la causa de buena parte de esos errores: la descripción se revisa una vez y las diecisiete instancias salen iguales.

3.7. Herramientas externas de generación dirigida por modelos

Ninguna de las tres tesis anteriores incluye una sección de estado del arte externo. Es una omisión que conviene no repetir, porque el campo de la generación de código dirigida por modelos lleva décadas produciendo herramientas que, vistas desde lejos, hacen algo muy parecido a lo que aquí se propone: describir una vez y generar varios destinos.

Xtext (Eysholdt y Behrens 2010) es un entorno para definir lenguajes de dominio específico **externos**: se escribe una gramática, y la herramienta deriva el analizador, el metamodelo, el editor con resaltado y completado, el validador incremental y el andamiaje de generación. **EMF** (Steinberg et al. 2008) es la capa de modelado sobre la que Xtext se apoya: define el metamodelo, genera el código de las clases del modelo, y proporciona serialización, notificación de cambios y persistencia. El **modelado de dominio específico** (Kelly y Tolvanen 2008) sistematiza la práctica: construir un metamodelo del dominio, una notación para él y un generador completo, con el objetivo declarado de generación total en lugar de esqueletos que después se completan a mano. La **programación generativa** (Czarnecki y Eisenecker 2000) formula el problema en términos de familias de productos y espacios de configuración: un modelo de características describe qué variantes existen, y un generador produce el miembro concreto de la familia. A todo ello hay que sumar la ingeniería de DSL como disciplina, que documenta con detalle las decisiones de diseño y sus compromisos (Voelter 2013; Mernik, Heering, y Sloane 2005; Fowler 2010).

Por otra parte, existe una familia de plataformas comerciales de **bajo código y sin código construidas alrededor de la metáfora de la hoja de cálculo en la nube**: se describe una estructura tabular en una interfaz visual, se declaran columnas calculadas, validaciones y reglas de formato condicional, y la plataforma genera de forma automática vistas web, formularios de captura y aplicaciones móviles a partir de la misma base. Son, sin duda, el competidor comercial más cercano a lo que aquí se propone. Se describen de forma genérica, sin citar producto concreto, porque no se ha realizado un estudio comparativo con productos específicos y afirmar detalles sobre ellos sin respaldo sería irresponsable en un documento que se defiende.

Frente a todo esto hay tres diferencias defendibles. Ninguna de las tres es de superioridad: son diferencias de naturaleza, y como tales acotan el aporte en vez de inflarlo.

Primera: DSL interno frente a DSL externo, y el coste de añadir un concepto.

Xtext y EMF exigen metamodelo, gramática, generación del andamiaje y un ciclo de regeneración. Ese coste compra cosas valiosas —editor, validación incremental, interoperabilidad— pero lo compra por adelantado y lo cobra en cada cambio. En un DSL interno en Common Lisp no hay metamodelo ni gramática: las formas del lenguaje son listas y las macros son el compilador (Hudak 1996; Graham 1993). El coste de añadir un concepto del dominio es escribir una macro y unos métodos, no regenerar un metamodelo y su tooling. Para un lenguaje que se espera que crezca durante años a medida que aparecen situaciones nuevas —que es el escenario real aquí— esa diferencia de coste marginal es la que decide si el lenguaje se mantiene vivo. El compromiso inverso hay que declararlo y no disimularlo: quien escribe

la descripción escribe paréntesis de Lisp, y no hay editor gráfico ni validación mientras se teclea.

Segunda: el dominio no es CRUD. Las herramientas de modelado de dominio específico y las plataformas de bajo código están afinadas para aplicaciones de gestión con entidades, relaciones y pantallas de alta, baja, modificación y consulta. El dominio de este trabajo es más estrecho y más profundo: la **situación tabular con derivaciones vivas y marcado por significado**. Los documentos que hay que reproducir no son pantallas de mantenimiento de datos; son instrumentos que calculan mientras se escriben, que avisan de conflictos en el momento en que aparecen y que distinguen entre una restricción que invalida y una restricción que solo advierte. El corpus de este trabajo contiene esa distinción de forma explícita: los desplegables de los tres libros se generan con `errorStyle="information"`, es decir, **avisan sin bloquear** (`tribunales/hoja_dia.py:113-116`), y el comentario del código lo dice con esas palabras. Esa es una decisión del dominio, no del formato, y un generador de CRUD no tiene dónde ponerla.

Tercera: el mecanismo de extensión por terceros. Es la diferencia genuinamente nueva y la que sostiene el trabajo. En las herramientas citadas, añadir un destino significa escribir plantillas dentro de su arquitectura, conforme a sus convenciones, normalmente por parte de quien ya conoce la herramienta por dentro. Lo que aquí se propone es un **protocolo publicado** —una lista cerrada de funciones genéricas con su contrato— acompañado de un **juego de pruebas de conformidad** que una arquitectura nueva debe pasar para declararse conforme, y de un **informe de conformidad** que, en cada compilación, deja por escrito qué se cumplió de forma nativa, qué se emuló, qué se degradó y qué se rechazó. Y, sobre todo, se propone **probarlo con un tercero real**: que el tutor implemente la arquitectura de Emacs usando solo el protocolo, sin tocar el núcleo y sin ayuda. Esa prueba tiene la propiedad, poco común, de que sus dos resultados son publicables: si lo logra, el mecanismo funciona; si no lo logra, el trabajo documenta por qué falló, lo cual es un resultado más honesto y más útil que afirmar la extensibilidad sin someterla a prueba, que es lo que ocurrió tres veces seguidas.

Un apunte sobre el formato de salida principal. Que el destino más trabajado sea la hoja de cálculo no es una elección estética: es donde está el usuario. La investigación sobre programación por usuarios finales lleva décadas documentando que la hoja de cálculo es, por volumen, el entorno de programación más usado del mundo, y que sus usuarios no se consideran programadores (Peyton Jones, Blackwell, y Burnett 2003; Ko et al. 2011). El formato en sí está normalizado (*ECMA-376: Office Open XML File Formats* 2016), lo que hace que el destino sea estable y que la generación sea verificable contra una especificación pública.

3.8. Por qué un DSL y no un modelo de lenguaje

Ésta es la pregunta con más filo que se le puede hacer a este trabajo, y conviene formularla en su versión más fuerte antes de responderla. El flujo que el tutor describe —“te describo la situación y me devuelves el documento”— es literalmente lo que ya está ocurriendo: durante los meses previos a este trabajo, la situación se describió en lenguaje natural y el documento se produjo escribiendo código a medida, con asistencia de modelos de lenguaje en buena

parte del proceso. Si un modelo ya hace el trabajo, ¿para qué un lenguaje?

La respuesta no es que el modelo no sirva. Sirve, y muy bien. La respuesta tiene cinco partes, y la última es la importante.

Determinismo y reproducibilidad. La misma descripción produce siempre el mismo producto. Un modelo de lenguaje, no: dos ejecuciones sobre el mismo enunciado pueden diferir, y las diferencias aparecen justamente en los detalles que más caro cuesta revisar —un rango que empieza una fila más abajo, una comparación que cambia de sentido, una guarda que desaparece—. En un documento que después usan personas para tomar decisiones sobre horarios y tribunales, ese tipo de variación no es aceptable.

La descripción es el artefacto. Un programa escrito en el lenguaje es un archivo de texto que se versiona, se revisa en un control de cambios, se compara entre semestres y sobrevive al modelo que lo escribió. Una conversación con un modelo y su respuesta no cumplen ninguna de esas cuatro propiedades. Cuando dentro de dos cursos haya que averiguar por qué el documento marca en rojo un caso concreto, la respuesta está en la descripción, leyéndola; no en recordar qué se le pidió a qué modelo.

Coste marginal cero al regenerar. Los diecisiete grupos del horario se regeneran cada semestre. Con una descripción, regenerarlos cuesta ejecutar un comando. Con un modelo, cuesta una interacción por documento y una revisión por documento, porque no hay garantía de que la salida de esta vez coincida con la de la anterior.

El protocolo de extensión. Cuando un tercero añade una arquitectura nueva conforme al protocolo, **todas las descripciones que ya existen la ganan de golpe**. Ese efecto no tiene equivalente en el flujo con un modelo: allí, añadir un formato nuevo significa volver a pedir cada documento en el formato nuevo, uno por uno, con la revisión correspondiente.

Y el argumento fuerte, que además es verdad: un modelo de lenguaje es muy bueno escribiendo la descripción en el DSL, y no la sustituye. Los dos tramos del problema son distintos. El tramo que va del lenguaje natural a la descripción formal es ambiguo, tolera la conversación, admite preguntas de aclaración y se beneficia enormemente de un modelo; el tramo que va de la descripción al producto tiene que ser determinista, porque lo que se genera después nadie lo lee entero. No son alternativas: son las dos mitades del mismo recorrido, y la existencia de modelos capaces de cubrir bien la primera mitad **aumenta** el valor de tener bien definida la segunda, porque hace que escribir la descripción deje de ser una barrera de entrada. Conviene decirlo antes de que lo pregunten, y conviene no fingir lo contrario: en un futuro próximo, el usuario más probable de este lenguaje no es una persona tecleando paréntesis, sino una persona dictando la situación y revisando la descripción que un modelo escribió por ella.

3.9. Síntesis: qué se hereda y qué se rehace

Elemento	Origen	Decisión
Metodología de los seis pasos	(Arcia Corcho 2017)	Heredar como marco

Elemento	Origen	Decisión
defnode / defclass*	(Arcia Corcho 2017), vía (Cápiro Colomar 2025; Morales Lazo 2026)	Heredar
Jerarquía por herencia múltiple	ADOL*, conocida a través de (Arcia Corcho 2017)	Heredar y, por primera vez, implementar: es la base del mecanismo de capacidades
gcode, recognize-pattern, métodos :before/:around	(Arcia Corcho 2017)	Heredar para reglas transversales
Elemento consulta y su descomposición	(Cápiro Colomar 2025)	Heredar, ampliado con agregación y búsqueda por clave
def-entidad con accesorios gramaticales	(Cápiro Colomar 2025)	Heredar
Sinónimos de constructor	(Cápiro Colomar 2025)	Heredar
Modelo visual	(Cápiro Colomar 2025)	Descartar: no tiene tabla
pagina/lista/item		
Taxonomía del dominio tabular	(Morales Lazo 2026)	Heredar como punto de partida y ampliar
Cuantificador existencial	(Morales Lazo 2026)	Heredar; es lo mejor de las tres
Plantilla instanciable con parámetros	(Morales Lazo 2026)	Heredar
Identificador interno frente a etiqueta visual	(Morales Lazo 2026)	Heredar
Arquitectura en dos fases, como idea	(Morales Lazo 2026)	Heredar la idea, mover la frontera
region, anchor,	(Morales Lazo 2026)	Rehacer: son rejilla, salen del lenguaje
cell-height, cell-width		
nav	(Morales Lazo 2026)	Rehacer: la navegación entra como concepto de dominio, no como colocación en rejilla
expr-counta	(Morales Lazo 2026)	Rehacer en términos de filas, no de celdas vacías
style-rule sin color	(Morales Lazo 2026)	Rehacer: la regla declara significado, no color
compile-excel-formula	(Morales Lazo 2026)	Rehacer: es el punto de extensión y está formulado en Excel
JSON con fórmulas y rangos A1 como frontera	(Morales Lazo 2026)	Rehacer: la frontera está en el sitio equivocado

Elemento	Origen	Decisión
Modelo de evaluación declarado	ninguna	Nuevo
Protocolo de arquitectura y juego de pruebas de conformidad	ninguna	Nuevo
Mecanismo de degradación e informe de conformidad	ninguna	Nuevo
Verificación reproducible de que las fórmulas se evalúan	ninguna	Nuevo

Dos precisiones para cerrar, porque son las que separan lo que este trabajo puede afirmar de lo que no.

La primera es sobre la evidencia de evaluación. Sería excesivo decir que ninguna de las tres tesis presenta evidencia de que sus fórmulas se evalúen: las conclusiones de ([Morales Lazo 2026](#)) afirman que los archivos `.xlsx` generados se evalúan correctamente en Excel y en LibreOffice Calc, y lo respaldan con capturas de valores ya calculados. Lo que no hay en ninguna de las tres es **verificación reproducible y automatizada**: un procedimiento que cualquiera pueda volver a ejecutar y que falle por sí solo cuando algo se rompa. Ésa es la diferencia que este trabajo puede sostener, y es más modesta y más defendible que la afirmación fuerte.

La segunda es sobre el origen del vocabulario. Las tres tesis anteriores diseñaron su lenguaje contra ejemplos escritos en prosa. Este trabajo dispone de tres documentos reales en uso — horario docente, tribunales de defensa y gestión del departamento—, refinados por iteración con el usuario final, que cumplen dos funciones a la vez: son la fuente del vocabulario y son el oráculo de validación. De ahí salen construcciones que ninguna de las tres taxonomías anteriores contiene —búsqueda por clave, entrada validada que avisa sin bloquear, rol de campo, capacidad de crecimiento, navegación, relación uno a muchos y conteo de valores distintos— y de ahí sale el criterio de éxito, que no es “se produce un archivo” sino “el archivo producido desde la descripción es equivalente al que hoy produce el código escrito a mano”. El capítulo siguiente caracteriza ese corpus y extrae de él el vocabulario del lenguaje.

Capítulo 4

El dominio de las situaciones tabulares

Todo lenguaje de dominio específico descansa sobre un análisis del dominio, y la calidad de ese análisis decide la del lenguaje. La literatura lo dice sin rodeos: la fase de análisis es la que determina qué conceptos existen y cuáles no, y equivocarse ahí no se arregla después con sintaxis (Mernik, Heering, y Sloane 2005; Kelly y Tolvanen 2008). El problema práctico es de dónde sale el material. Un análisis hecho sobre ejemplos escritos en prosa produce un vocabulario que se detiene justo antes de la parte difícil, porque la prosa no obliga a cerrar ningún cabo.

Este capítulo levanta el vocabulario del lenguaje a partir de artefactos reales. No se parte de una descripción imaginada de lo que sería un horario: se parte de tres generadores de libros de Excel que están en uso, que se refinaron por iteración con el usuario final y que hoy producen los documentos con los que se planifica la docencia, las defensas de tesis y la carga del departamento. Cada concepto que se propone aquí se sostiene sobre un fragmento de código concreto, con archivo y línea. Cuando algo es lectura o interpretación propia, se dice.

4.1. El corpus

El corpus son tres generadores escritos en Python sobre `openpyxl`, más un paquete común que los tres comparten. Cada generador lee un archivo YAML con los datos de la situación y escribe un libro `.xlsx`.

- `generar.py` y el paquete `horarios/` producen el **horario docente de la facultad**: una hoja por grupo con la rejilla de días y turnos, una hoja de ocupación de aulas, los listados de aulas, asignaturas, grupos, estructura, profesores y docencia, y una hoja auxiliar oculta con las tablas derivadas (`horarios/generador.py:63-122`).
- `generar_tribunales.py` y el paquete `tribunales/` producen la **planificación de las defensas de tesis**: una hoja por día, con un bloque por local y una fila por momento, más las hojas de profesores, estudiantes, locales, días, tribunales y una hoja **Localizar**

(tribunales/generador.py:21-60).

- `generar_departamento.py` y el paquete `departamento/` producen la **gestión del departamento**: el claustro, el plan de asignaturas del semestre, la hoja de asignación donde se reparte la docencia, el reporte de carga por profesor y la cobertura por asignatura (`departamento/generador.py:27-79`).

Los tamaños, contados el 2026-09-20 sobre los archivos `.py` de cada paquete:

Paquete	Archivos	Líneas	Qué es
<code>comun/</code>	11	631	formato, leyenda, protección, rangos, portada, impresión, vista, validación
<code>horarios/</code>	15	1 428	generador del horario docente
<code>tribunales/</code>	14	848	generador de defensas de tesis
<code>departamento/</code>	13	1 448	generador de gestión del departamento
Generadores	53	4 355	el vocabulario sale de aquí
<code>extraccion/</code>	6	967	importadores de PDF y de Excel existentes
<code>tests/</code>	51	5 782	batería de pruebas

Conviene subrayar qué es y qué no es este material. **No es un ejercicio.** No se escribió para ilustrar una tesis ni para probar un lenguaje. Se escribió porque alguien necesitaba los documentos, los usó, encontró lo que no servía y lo pidió de otra manera. Buena parte del detalle que aparece en el código no se habría diseñado nunca en un escritorio: existe porque un caso real lo rompió. El docstring de `comun/portada.py:4-5` lo dice sin adornos: la portada existe *“porque el tutor tuvo que preguntar dónde se entraban los datos de la docencia: el libro no lo decía por ninguna parte”*.

El corpus cumple aquí **dos funciones distintas**, y conviene no confundirlas.

1. **Fuente del vocabulario.** Los conceptos del lenguaje se extraen de lo que los tres generadores hacen de verdad, no de lo que parecería razonable que hicieran. Si una construcción aparece once veces en tres dominios independientes, es del dominio; si aparece una vez y en un solo archivo, es sospechosa de ser un accidente.
2. **Oráculo de validación.** Los libros ya existen y sus 5 782 líneas de pruebas también. El criterio de éxito del lenguaje no es “sale un `.xlsx`”, sino que el `.xlsx` generado

desde la descripción declarativa sea equivalente al que hoy produce el código Python. Esa es una vara que ninguna de las tres tesis anteriores tuvo disponible.

Esta segunda función es la que convierte la ventaja en método. Un lenguaje diseñado contra un oráculo tiene una condición de parada objetiva; uno diseñado contra ejemplos en prosa termina cuando el autor se cansa.

4.2. Qué es una situación tabular

Definición 3.1 (situación tabular). Una *situación tabular* es un conjunto finito de entidades con atributos, organizado en una o más tablas relacionadas por claves, sobre el cual se define:

- un conjunto de **derivaciones**, que son valores calculados a partir de otros valores de la misma situación;
- un conjunto de **marcas**, que asocian un estado con significado a las filas o celdas que cumplen una condición;
- un conjunto de **puntos de entrada**, que son las posiciones que una persona completa a mano, cada una con el conjunto de valores que admite; y
- una **navegación**, que es la manera de recorrer la situación por secciones.

Una situación tabular no es una tabla. Una tabla es su soporte. Lo que hace que la situación sea interesante —y lo que justifica un lenguaje— son las cuatro cosas de arriba, que son precisamente las que no caben en un volcado de datos.

La observación central de este capítulo es la siguiente, y de ella se sigue casi todo lo demás:

Los tres libros del corpus **no son documentos estáticos. Son programas pequeños y reactivos.**

El Excel generado no *muestra* la situación resuelta. La **mantiene consistente mientras el usuario la edita**. En la hoja de un día del libro de tribunales, la única columna que se escribe es la del estudiante; las cinco columnas del tribunal se rellenan solas, y se vuelven a rellena cada vez que se cambia el estudiante (`tribunales/hoja_dia.py:38-43`). En la hoja de un grupo del horario, al cambiar la asignatura de un turno el profesor la sigue (`horarios/hoja_grupo.py:269-292`). En la hoja de cobertura del departamento, la columna Estado dice qué falta, y lo dice otra vez en cuanto alguien crea una fila de carga (`departamento/hoja_cobertura.py:112-133`).

Que una hoja de cálculo sea un lenguaje de programación no es una observación nueva: es la base del trabajo sobre implementación de hojas de cálculo (Sestoft 2014) y del análisis de Excel como lenguaje funcional de primer orden (Peyton Jones, Blackwell, y Burnett 2003). Lo que sí importa aquí es la consecuencia para el diseño del lenguaje: **si el producto es reactivo, la descripción tiene que describir la reactividad, no su resultado congelado en un instante**. Una derivación no es un valor: es una regla que sigue vigente después de generar. Y “ser una arquitectura válida” deja de ser una etiqueta y pasa a ser una condición precisa: la arquitectura tiene que poder sostener las derivaciones vivas, o declarar explícitamente que no puede y degradarlas.

4.3. Las cuatro patas del núcleo semántico

Del análisis del corpus salen cuatro categorías, y no una. El error de pensar que el dominio es “una tabla” consiste justamente en quedarse con la primera.

4.3.1. Datos

Las entidades de la situación y sus atributos, tal como llegan del YAML: los profesores, las aulas, las asignaturas con su carrera, su año y su frecuencia, los grupos, los locales, los momentos del día. En el corpus viven en hojas de listado visibles, no en el código: el claustro está en la hoja **Profesores** y el plan del semestre en la hoja **Asignaturas**, y cada una declara sus propios rangos nombrados (`departamento/hoja_datos.py:3-5`).

Esta pata es la que cualquier análisis del dominio encuentra primero y la única que las tres tesis anteriores cubren sin discusión. También es la menos interesante: un volcado de datos no necesita un lenguaje, necesita un formato.

4.3.2. Derivaciones vivas

Valores que no se escriben, se calculan, **y se recalculan**. El corpus está lleno de ellas y son de varios tipos: aritméticas (`faltan = frecuencia - asignadas`), agregaciones (`COUNTIF`, `SUMIF`, `SUMIFS`) y recuperaciones por clave (`VLOOKUP`).

El ejemplo canónico está en `tribunales/hoja_dia.py:40-43`:

```
ws[f"{col}{f}"] = (
    f'=IF($L.COL_ESTUDIANTE){f}="", "", '
    f'IFERROR(VLOOKUP($L.COL_ESTUDIANTE){f},TesisTribunal,{idx},FALSE), "")'
)
```

Lo que esa línea quiere decir son tres frases del dominio: si esta fila todavía no tiene estudiante, no muestres nada; si lo tiene, trae de la tabla de tesis el profesor que ocupa este rol; y si ese estudiante no aparece en la tabla de tesis, tampoco muestres nada. Lo que la línea *dice* es Excel puro. La distancia entre las dos cosas es el problema que el lenguaje viene a resolver.

Un detalle que solo aparece cuando el documento se usa de verdad: la guarda `IF(...="", "", ...)` no es decoración. `departamento/hoja_datos.py:76-87` la documenta con precisión: en Excel y en Calc una referencia directa a una celda **vacía** se muestra como 0, no en blanco, y sin la guarda una línea del bloque de carga mostraría un 0 donde no hay nada. Es el tipo de detalle que separa un prototipo de algo usable, y es también evidencia de que el dominio tiene una noción propia de “todavía no hay nada aquí” que no coincide con el cero.

4.3.3. Marcado con significado

Reglas que asocian un estado a las filas o celdas que cumplen una condición. El corpus tiene 17 reglas de formato condicional. Un ejemplo, en `horarios/hoja_grupo.py:240-250`,

detecta al profesor citado en dos grupos a la misma hora sumando un COUNTIF por cada hoja de grupo:

```
conteos = "+".join(
    f"COUNTIF({quote_sheetname(g.id)}!{primera_prof},{primera_prof})"
    for g in facultad.grupos)
ws.conditional_formatting.add(
    sq_prof,
    estilos.regla_formula(
        f'AND({primera_prof}<>"",({conteos})>1)',
        estilos.COLOR_CONFLICTO))
```

Lo decisivo no es la fórmula, es el nombre de la constante. El corpus no escribe un código de color: escribe COLOR_CONFLICTO, COLOR_SOBRECARGA, COLOR_ASIG_DESCONOCIDA, COLOR_AULA_SIN_ASIG, COLOR_COMPLETA, COLOR_INCOMPLETA. Es decir, **el corpus ya nombra el significado de la marca y usa el color como una consecuencia**, aunque los dos vivan todavía en la misma constante (horarios/estilos.py:15-25, departamento/estilos.py:13-18, tribunales/estilos.py:8-10).

4.3.4. Entrada validada

Las posiciones que una persona completa a mano, con el conjunto de valores que admiten. El corpus tiene 9 validaciones de datos, todas de tipo lista y casi siempre alimentadas por un rango con nombre. Pero la entrada no es solo el desplegable: es también la declaración de qué celdas se pueden tocar. La hoja del día de tribunales deja editable exactamente la columna del estudiante y bloquea las cinco del tribunal (tribunales/hoja_dia.py:53-56):

```
# Solo la columna Estudiante se elige a mano; C..G son BUSCARV del tribunal.
editables = [proteccion.celda_unica(c)
              for c in L.celdas_estudiante(len(facultad.locales), n_mom)]
proteccion.proteger_hoja(ws, editables=editables)
```

Esta cuarta pata es la que se cae si se decide que el alcance del trabajo es “solo visualizar”. Los tres libros del corpus son instrumentos de captura: sin entrada, el lenguaje no puede describir ni uno solo de ellos, y con ello se perdería el oráculo. La lectura que se sostiene en este trabajo es que “hacer lo mismo que se hace ahora con los Excel” incluye la entrada por definición, no como ampliación de alcance.

4.4. Análisis del corpus

4.4.1. El método de conteo, declarado

Los conteos que siguen se obtuvieron así, y el procedimiento se declara porque de él dependen las cifras:

1. **Solo sobre los cuatro paquetes de generadores:** comun/, horarios/, tribunales/ y departamento/. Se excluye tests/ y se excluye extraccion/, que son importadores y no generan documento.

2. **Con límite de palabra** (`\bTERMINO\b`), no por subcadena. Así `COUNTIF` no captura las apariciones de `COUNTIFS`.
3. **Excluyendo comentarios y docstrings**: solo se cuentan las apariciones en el código que construye la fórmula o llama a la API.

Este método es reproducible y da las cifras de la tabla siguiente. Es necesario decir además lo siguiente, porque las cifras publicadas antes eran otras: **los conteos del análisis preliminar del proyecto eran mayores porque mezclaban generadores con pruebas y contaban por subcadena**. El caso más claro es el de la protección de celdas, que el análisis preliminar reportaba como 75 usos: contando por subcadena sobre generadores y pruebas a la vez salen en efecto 75, pero sobre los generadores solos son 4 — la protección está factorizada en `comun/proteccion.py` y las 71 restantes eran aserciones de la batería. Lo mismo ocurre con los hipervínculos, que pasan de 13 a 3, y con el formato condicional, que pasa de 42 a 17. Ninguna conclusión cualitativa cambia, pero las cifras que se citen en una defensa tienen que ser las reproducibles.

4.4.2. Funciones de Excel presentes en el corpus

Función	Usos	Concepto de dominio
IF	45	condicional; y sobre todo la guarda “todavía no hay nada aquí”
AND / OR	14 / 2	conjunción y disyunción de condiciones
COUNTIF	13	cuántas filas cumplen una condición
VLOOKUP	11	búsqueda por clave : traer un atributo de otra tabla
IFERROR	9	la clave no existe todavía: no es un error, es un estado del dominio
SUMIF	5	total de un atributo sobre las filas que cumplen una condición
SUMIFS	2	lo mismo, con varios criterios a la vez
COUNTIFS	1	cuántas filas cumplen varias condiciones a la vez
COUNTA	2	cuántas filas hay escritas (tamaño efectivo de un listado)

Función	Usos	Concepto de dominio
OFFSET	2	ventana que se redimensiona con lo escrito

COUNTA y OFFSET aparecen concentradas en un solo módulo, `comun/rangos.py`, y siempre juntas: son las dos mitades de un único mecanismo, el del listado que crece. Esa concentración es informativa, y se trata en su propia sección más abajo.

4.4.3. Características de openpyxl presentes en el corpus

Característica	Usos	Concepto de dominio
DefinedName	24	una tabla o una columna tiene nombre y se la referencia por él
conditional_formatting	17	marcado con significado
DataValidation	9	entrada validada
freeze_panes	8	<i>ninguno</i> : es opción de arquitectura
hyperlink	3	navegación entre secciones
auto_filter	3	frontera: filtrar es del dominio, la barra de filtro no
protection	4	rol de campo : qué se escribe y qué se calcula
tabColor (vía colorear_pestana)	21 llamadas	navegación: a qué sección pertenece cada parte

La columna derecha es donde está el trabajo de análisis. `freeze_panes` aparece 8 veces y no aporta ningún concepto: congelar una fila de encabezado es una comodidad de rejilla que una página web resuelve con CSS y que un buffer de Emacs resuelve de otra manera, o de ninguna. `protection`, en cambio, aparece solo 4 veces y es el mejor candidato a concepto nuevo de todo el corpus. La frecuencia no mide la importancia: mide la repetición.

4.5. Construcciones que la taxonomía de 2026 no cubre

Aquí está el hallazgo empírico central del capítulo. La taxonomía de (Morales Lazo 2026) — tabla, plantilla, fila, columna, celda, columna calculada, referencia cruzada, parámetro, regla de marcado, cuantificador existencial, hoja, libro— cubre una parte sustancial del corpus: las columnas calculadas por aritmética y comparación, el recuento condicional cruzando hojas,

el marcado condicional, la plantilla instanciada con datos distintos y los parámetros. Decirlo es obligatorio, porque exagerar el hueco sería deshonesto y además contraproducente.

El tercio que no cubre es, en su totalidad, el que tiene que ver con que el documento sea **editable y vivo**, no solo legible. Son ocho construcciones.

4.5.1. Búsqueda por clave: el *join* que falta

Once usos de VLOOKUP, repartidos por los tres generadores:

```
tribunales/hoja_dia.py:42
    IFERROR(VLOOKUP($<estudiante>, TesisTribunal, <idx>, FALSE), "")

departamento/hoja_asignacion.py:88
    IF($A4="", "", IFERROR(VLOOKUP($A4, AsignaturasTabla, 2, 0), ""))

departamento/hoja_carga.py:102
    IFERROR(VLOOKUP(<id_bloque>, ProfesoresTabla, <columna>, 0), "")

horarios/hoja_grupo.py:291
    IFERROR(VLOOKUP("<grupo>#" & <celda_asig>, RangoDocencia, 2, 0), "")
```

En términos de dominio: *dada una clave en la fila actual, traer un atributo de la fila correspondiente de otra tabla*. Es lo que hace que el libro de tribunales cumpla lo que promete: se elige el estudiante y el resto del tribunal se autocompleta.

El corpus además distingue la copia de la búsqueda, y lo documenta. En `departamento/hoja_asignacion.py:104`: *“Es BUSCARV y no una copia: si fuera copia, cambiar el id dejaría en pantalla el nombre del anterior.”* Esa frase es exactamente la diferencia entre una derivación viva y un valor congelado, dicha por el usuario del artefacto y no por el diseñador del lenguaje.

No es expresable en el lenguaje de 2026. Sus nodos de expresión permiten contar cuántas veces aparece un valor, pero no recuperar el valor asociado. Y aquí está la observación que conviene retener: **este concepto es exactamente lo que (Cápiro Colomar 2025) llamó “consulta” y lo que (Morales Lazo 2026) dejó fuera al concentrarse en lo visual**. Las dos tesis anteriores tienen cada una la mitad de lo que hace falta: 2025 tiene consultas sin tablas ni agregación; 2026 tiene tablas y agregación sin consultas. Nadie las juntó.

4.5.2. Clave compuesta

La clave de la búsqueda no siempre es un campo. En `horarios/hoja_grupo.py:289–292` se construye concatenando dos:

```
ws[L.celda_profesor(dia_idx, turno)] = (
    f'=IF({celda_asig}="", "", '
    f'IFERROR(VLOOKUP("{grupo.id}#"&{celda_asig}, '
    f'{HOJA_DATOS.RANGO_DOCENCIA},2,0), ""))'
```

El docstring de esa función lo explica en términos de dominio (`horarios/hoja_grupo.py:272-275`): *“Se busca en la tabla derivada de la hoja Docencia con la clave <grupo>#<asignatura>: el grupo es fijo en esta hoja, así que va escrito en la fórmula, y la asignatura sale de la celda de arriba.”*

Es decir, la relación entre profesor y turno no está dada por un campo: está dada por el par (grupo, asignatura). El lenguaje tiene que poder decir *“la clave de esta relación son estos dos campos”* sin que quien escribe la descripción tenga que inventarse la concatenación ni elegir el separador. El # es mecanismo de Excel; el par es dominio.

4.5.3. Entrada validada

Nueve usos de `DataValidation`, siempre de tipo lista:

```
# tribunales/hoja_dia.py:114-115
dv = DataValidation(type="list", formula1="EstudiantesValidos",
                    allow_blank=True, showErrorMessage=True,
                    errorStyle="information")
```

En términos de dominio: *esta columna se escribe a mano y sus valores admisibles son los de aquel conjunto.*

Lo decisivo es `errorStyle="information"`. **Se avisa, no se bloquea.** Y eso es una decisión de dominio, no de Excel. El comentario de `horarios/hoja_grupo.py:175-177` lo razona: *“el aviso es no bloqueante, de modo que en Calc/Excel se pueden escribir aulas fuera del listado (el dropdown queda solo como ayuda). Sin una acción de error explícita, Calc asume ‘Stop’ y rechaza los valores nuevos.”* Y `departamento/hoja_asignacion.py:131-134` da la razón de negocio: *“hay que poder escribir un id que todavía no está en la lista (una asignatura recién añadida, un profesor que aún no se ha dado de alta) sin que Calc o Excel lo rechacen. Quien se equivoque lo verá en ámbar, que es un aviso y no un veto.”*

Eso es el par **restricción dura / restricción blanda**. Hay restricciones que invalidan y restricciones que solo advierten, y la diferencia entre las dos es del dominio: depende de si el conjunto de valores admisibles está cerrado o está creciendo. El concepto además es fuertemente traducible entre arquitecturas —desplegable en Excel, `<select>` en web, `completing-read` en Emacs, clave foránea o `CHECK` en SQL—, lo que lo convierte en un buen candidato al núcleo. Ninguna tesis anterior lo tiene.

4.5.4. Rol de campo: qué se escribe y qué se calcula

Cuatro usos de protección en los generadores, porque está factorizada en un solo módulo, `comun/proteccion.py`. El docstring de ese módulo dice qué es y qué no es (`comun/proteccion.py:8-10`): *“Sin contraseña a propósito: el objetivo es evitar que se borre una fórmula sin querer, no proteger secretos.”*

Eso confirma que **el concepto no es “proteger”**. Proteger es el mecanismo que Excel ofrece. Lo que el corpus está declarando es el **rol** de cada campo: de entrada o derivado. Y la prueba de que es un rol y no una medida de seguridad está en que de esa única

declaración salen, en el corpus, tres consecuencias distintas: el bloqueo de la celda, el color de la pestaña que lo anuncia (`comun/vista.py:11-14`) y la etiqueta del índice de la portada (`comun/portada.py:29-31`), donde las etiquetas son literalmente **se escribe**, **se calcula** y **datos del problema**.

De esa misma declaración salen consecuencias en las cuatro arquitecturas del trabajo:

Arquitectura	Campo de entrada	Campo derivado
Excel	celda desbloqueada, fondo blanco, desplegable	celda bloqueada, fórmula
Web	<code><input></code> o <code><select></code>	texto calculado, sin foco
SQL	columna de la tabla	vista o columna generada
Emacs	el cursor entra y edita	propiedad de solo lectura

Es un concepto simple de enunciar, con consecuencias en todas las arquitecturas, y que explica por qué el documento generado sigue siendo útil después de generarse. Por eso es, en la valoración de este trabajo, **el mejor candidato a concepto nuevo del dominio**.

4.5.5. Capacidad y reserva: el documento que crece

`comun/rangos.py` resuelve, en 54 líneas, un problema que ninguna de las tres tesis anteriores se planteó: **el documento generado sigue vivo después de generarse**, y el usuario añade filas.

```
def rango_dinamico(hoja: str, columna: str, fila_inicial: int,
                   capacidad: int, columnas: int = 1) -> str:
    h = quote_sheetname(hoja)
    fila_final = fila_inicial + capacidad - 1
    return (f"OFFSET({h}!${columna}${fila_inicial},0,0,"
            f"COUNTA({h}!${columna}${fila_inicial}:${columna}${fila_final}),"
            f"${columnas})")
```

El docstring del módulo plantea el dilema y documenta dos trampas descubiertas en la práctica. El dilema (`comun/rangos.py:3-6`):

Un rango fijo obliga a elegir entre dos males: si abarca solo lo escrito, añadir una fila no la mete en el desplegable; si abarca la reserva, el desplegable muestra las filas vacías.

La primera trampa (`comun/rangos.py:12-15`):

Ojo con las columnas de fórmulas: COUNTA cuenta celdas no vacías, y una fórmula que devuelve "" ocupa celda. Un rango anclado en una columna calculada abarca siempre toda su capacidad; da igual para BUSCARV y SUMAR.SI, pero no sirve para dimensionar nada.

La segunda (`comun/rangos.py:17-20`):

La trampa, también comprobada: COUNTA cuenta celdas no vacías, no filas ocupadas. Una fila en blanco en medio del listado no reduce la cuenta, pero desplaza el final del rango: con un hueco, la última entrada escrita queda fuera y en su lugar entra una celda vacía. Por eso las guías piden no dejar huecos.

Esa segunda trampa tiene consecuencia directa en el documento: la portada avisa al usuario en prosa (`comun/portada.py:42-48`), porque *“el efecto de dejar un hueco es silencioso: la lista se corta ahí y lo que queda debajo desaparece de los desplegables sin ningún mensaje”*. Es un error de hoja de cálculo del tipo que la literatura describe como difícil de detectar precisamente por no producir ningún aviso (Panko 1998; Hermans, Pinzger, y Deursen 2012).

La política de reserva también está en el dominio y no en la mecánica (`comun/rangos.py:29-35`): la capacidad es *“la mitad de lo declarado, con un mínimo”*, porque *“crecer un 50 % sin regenerar cubre el curso siguiente en la práctica”*.

En términos de dominio: *este listado tiene una capacidad reservada y su tamaño efectivo es el de lo escrito*. Y aquí está lo interesante para la tesis: este concepto **solo tiene sentido en arquitecturas de rejilla preasignada**. Una web con un botón “añadir fila” no tiene este problema en absoluto. Por eso es el caso de estudio ideal del mecanismo de capacidades y de degradación: el lenguaje declara la intención (“este listado crece”), y cada arquitectura la cumple como puede, o la ignora justificadamente porque en su modelo no hace falta.

4.5.6. Relación uno a muchos

Un profesor imparte varias cosas. El corpus necesita mostrar, en el bloque de cada profesor, la lista de lo que imparte; y no la conoce al generar, porque depende de lo que se elija después en la hoja de asignación.

La solución tiene tres piezas. La primera es una **clave numerada** construida en la hoja auxiliar (`departamento/hoja_datos.py:65-73`): la clave de la fila es '`<prof>#<n>`', donde `n` es el número de aparición de ese profesor hasta esa fila, calculado con un COUNTIF de rango creciente. La segunda son **líneas reservadas**: cada bloque tiene `filas_por_profesor` líneas, rellenas con búsquedas planas contra esa clave (`departamento/hoja_carga.py:107-112`). La tercera es el **aviso de desbordamiento** (`departamento/hoja_carga.py:113-117`):

```
if col == "A" and k == fpp - 1:
    # Ultima linea reservada: si hay desborde, avisa cuantas faltan
    # (la propia linea deja de mostrarse, por eso se suma 1).
    buscar = (f'IF({conteo}>{fpp},'
              f'("{+}&({conteo}-{fpp}+1)&" más)",{buscar})')
```

Es decir: si el profesor imparte más cosas de las que caben, la última línea deja de mostrar un detalle y muestra (+3 más). El docstring del módulo explica por qué todo esto está hecho con fórmulas planas (`departamento/hoja_carga.py:9-14`): *“Son fórmulas planas: se comportan igual en Excel y en Calc.”*

Nada de esto pertenece al lenguaje. Lo que pertenece al lenguaje es la frase *“cada profesor tiene un detalle de lo que imparte, y ese detalle no se conoce al generar”*. La clave numerada,

las líneas reservadas y el aviso de desbordamiento son tres invenciones de la arquitectura Excel para cumplir esa frase con un motor que no sabe generar filas. Una web itera sobre la colección y no inventa nada.

4.5.7. Conteo de distintos

El panel de la hoja de asignación muestra cuántas **asignaturas distintas** imparte cada profesor. Es una agregación que COUNTIF no sabe hacer, y el corpus lo dice expresamente (departamento/panel_profesores.py:19-23):

Por qué la cuenta de asignaturas necesita la hoja Auxiliar. CONTAR.SI no sabe contar valores distintos, y las fórmulas matriciales no se comportan igual en Excel y en Calc.

El rodeo son dos columnas en la hoja auxiliar (departamento/hoja_datos.py:90-115). La columna L construye la clave del par '<profesor>|<asignatura>', con una guarda que merece atención: se deja vacía si falta cualquiera de las dos mitades, porque *“se elige antes a quién se le da que qué se le da, y entre un paso y otro la clave sería '<profesor>|', que es una asignatura distinta de su propio derecho y le sumaría una de más en el panel”*. La columna M marca con un 1 la **primera aparición** de cada par, con otro COUNTIF de rango creciente. Sumar M filtrando por profesor da la cuenta de distintos con una fórmula plana.

El detalle final es revelador del nivel de refinamiento del corpus (departamento/hoja_datos.py:109-111): la columna M devuelve 0 y no "" a propósito, *“es un sumando, y una cadena vacía en medio de un rango de suma es un valor que algunas versiones de Calc se niegan a ignorar”*.

En el dominio hay un solo concepto: *cuántos valores distintos de este atributo hay entre las filas que cumplen esta condición*. La columna de primera aparición es una invención de la arquitectura. Que el corpus haya tenido que inventarla, y documentar por qué, es la mejor prueba de que el concepto existe y de que Excel no lo tiene.

4.5.8. Navegación

Tres hipervínculos, 21 llamadas a colorear pestaña y dos hojas dedicadas.

La **portada** (comun/portada.py) es la primera hoja de los tres libros. Dice de dónde salió el libro, cuándo, qué hay en cada hoja y qué hacer para cambiar los datos. Su índice es una lista de nombres de hoja enlazados, cada uno con su descripción y su etiqueta de rol. El detalle técnico de cómo se construye el enlace (comun/portada.py:51-60) es puro mecanismo de OOXML (*ECMA-376: Office Open XML File Formats 2016*) y no tiene nada que ver con el dominio.

La hoja **Localizar** del libro de tribunales (tribunales/hoja_localizar.py) es una celda de entrada global y una tabla por cada par (día, local): se escribe un nombre en B1 y la hoja dice en qué momentos participa esa persona y en calidad de qué. La fórmula del rol son IF anidados, *“no IFS, por compatibilidad con Calc”* (tribunales/hoja_localizar.py:44-47).

Los **colores de pestaña** clasifican las hojas en tres grupos, con el mismo vocabulario que usa el índice: azul, *aquí se escribe*; gris azulado, *esto se calcula solo*; morado, *los datos del*

problema (tribunales/estilos.py:17-19, departamento/estilos.py:25-27).

En términos de dominio: *el documento tiene secciones, un punto de entrada y un modo de moverse entre ellas*. Es lo más parecido en el corpus al concepto de *pagina* de (Cápiro Colomar 2025), y es de las cosas que más cambian de forma entre arquitecturas —pestañas, rutas, *buffers*, pantallas—, lo que lo convierte en un buen banco de pruebas de la representación intermedia.

4.6. Lo que ya nombra bien el corpus

Hay un lugar del corpus donde el diseño correcto ya está hecho, y conviene señalarlo porque es la mejor evidencia disponible de una tesis de este trabajo.

La hoja de cobertura del departamento tiene una columna **Estado** que **no pinta el estado: lo escribe**. Y lo escribe en texto, en orden de gravedad, con concordancia de plural. El docstring lo justifica (departamento/hoja_cobertura.py:112-121):

Primero si no tiene ninguna fila de carga (nadie ha empezado), luego si las que tiene no cubren las horas declaradas (falta crear alguna) y por último si alguna se quedó sin profesor. **Un color solo no distingue estos tres casos, y son tres trabajos distintos.**

El singular se distingue del plural con un IF. Es una línea más de fórmula a cambio de que la hoja no diga “Faltan 1 profesores” a quien la lea.

Y la fórmula (departamento/hoja_cobertura.py:123-133):

```
faltan = f"${COL_SIN_PROFESOR}{f}"
profesores = (f'IF({faltan}=1,"Falta 1 profesor",'
              f'"Faltan "&{faltan}&" profesores"')
```

Esto es importante por dos razones. La primera: **ahí el color no alcanzaba, y el corpus degradó a texto por su cuenta**. Nadie lo diseñó como mecanismo de degradación; ocurrió porque el usuario necesitaba distinguir tres situaciones que un relleno de celda no distingue. La segunda: la hoja mantiene las dos cosas a la vez, el texto y el color, y el color se deriva del texto (departamento/hoja_cobertura.py:190-196): la regla verde dispara cuando la columna Estado dice "Completa".

De ahí sale la forma que debe tener una marca en el lenguaje: **la regla no declara un color, declara un significado**.

```
(marca :cuando (< (col asignadas) (col frecuencia))
           :estado insuficiente)
```

Y cada arquitectura decide cómo se ve: en Excel un relleno, en una web un color más un icono, en Emacs una *face*, en una impresión en blanco y negro necesariamente texto. El color no sobrevive al cambio de arquitectura; el significado sí. Esta es la corrección directa del punto débil de (Morales Lazo 2026), donde la regla de marcado se define con un color de fondo pero el nodo del árbol no lo lleva, y el color termina apareciendo dentro del JSON que

produce el generador, sin que el documento explique de dónde sale ni cómo se asocia a cada regla.

4.7. La frontera: qué entra en el lenguaje y qué no

El riesgo más común de un trabajo así es que el lenguaje crezca sin control: cada concepto nuevo multiplica el trabajo por el número de arquitecturas. Por eso la frontera se declara por escrito y de antemano, que es exactamente lo que (Morales Lazo 2026) no hizo con sus nodos de colocación en rejilla y tuvo que reconocer después, en una nota, que dependían del *backend*.

El criterio es una sola distinción: **intención frente a mecanismo**. La intención es un hecho de la situación y es lo mismo en todas las arquitecturas. El mecanismo es lo que una arquitectura concreta se inventa para cumplir esa intención con las herramientas que tiene.

Intención (entra)	Mecanismo (no entra)
“Esta lista crece después de generar”	“Reserva veinte filas y define un rango con OFFSET y COUNTA”
“Cada profesor tiene un detalle de lo que imparte”	“Reserva ocho líneas y constrúyeles la clave <prof>#<n>”
“Cuántas asignaturas distintas imparte”	“Una columna que marca con 1 la primera aparición del par”
“La clave de esta relación son grupo y asignatura”	“Concaténalas con #”
“Este campo se escribe a mano”	“Desbloquea la celda antes de proteger la hoja”
“El documento tiene secciones y un punto de entrada”	“Coloca un elemento contando filas desde el final de una columna”
“Esta marca significa <i>insuficiente</i> ”	“Píntalo de #EF9A9A”

De la columna derecha se sigue una regla operativa: **las hojas auxiliares y las columnas de apoyo se las inventa la arquitectura; el lenguaje no las nombra jamás**. La hoja *Auxiliar* del libro del departamento no existe en el dominio: existe porque Excel no sabe contar distintos. Su propio docstring lo reconoce al justificar el nombre (`departamento/hoja_datos.py:32-34`): se llama *Auxiliar* y no *Datos* porque “*solo guarda cálculos intermedios de las fórmulas*”.

Queda fuera del lenguaje, como **opción de arquitectura** declarada aparte:

- Anchos de columna, altos de fila, congelado de paneles (`freeze_panes`, 8 usos), colores de pestaña, configuración de impresión, ocultar la cuadrícula.
- La disposición absoluta de bloques dentro de una hoja.
- Los colores concretos. El lenguaje dice *insuficiente*; una **paleta**, que es opción de arquitectura, dice de qué color se pinta eso en Excel.

- El autofiltro y el orden como controles de interfaz. Aquí hay un matiz que el corpus obliga a hacer: *poder filtrar por un id* es del dominio, y de hecho la hoja de cobertura lo usa como parte de su respuesta (`departamento/hoja_cobertura.py:17-19`). Pero la decisión de conceder el filtro y negar el orden porque “*en una hoja cuyas filas tienen una gemela por posición en otra hoja, ordenar descuadraría la gemela sin avisar*” (`comun/proteccion.py:60-63`) es consecuencia de un mecanismo de rejilla, no de la situación.

4.8. Método de verificación disponible

El capítulo termina con el instrumento que hace que todo lo anterior sea comprobable y no opinable.

El problema es concreto: **pytest no evalúa fórmulas**. La batería de 5 782 líneas verifica que la celda contiene la cadena esperada, no que esa cadena produzca el valor correcto al abrir el libro. Una batería verde no dice nada sobre si el documento funciona. Es la versión automatizada del problema que la literatura de errores en hojas de cálculo lleva describiendo décadas: la fórmula está escrita, nadie la ejecuta, y el error solo aparece cuando alguien mira el número (Panko 1998; Ko et al. 2011).

El procedimiento que sí verifica es un *round-trip* por LibreOffice:

1. Generar el libro con el generador.
2. Rellenarlo con `openpyxl` como lo haría un usuario.
3. Pasarlo por `soffice --headless --convert-to xlsx`, que recalcula todas las fórmulas y guarda los valores.
4. Releer con `data_only=True` y comprobar los valores calculados.

Con ese procedimiento **se cazaron tres fallos que la batería daba por buenos**. Los guiones que lo implementan están en el proyecto (`documentacion/2026-08-26-verificar-roundtrip.py` y `documentacion/2026-08-27-verificar-guias.py`).

Para la tesis esto es más que una comodidad. Es lo que permite afirmar que el documento generado **funciona**, y no solo que se produjo. (Morales Lazo 2026) afirma en sus conclusiones que los `.xlsx` generados “*se evalúan correctamente en Excel y LibreOffice Calc*” y lo respalda con capturas de pantalla que muestran valores ya calculados; lo que no hay en ninguna de las tres tesis anteriores es un **procedimiento reproducible que cualquiera pueda volver a ejecutar**. Esa es una fuente de rigor barata y diferenciadora, y es la que se usa en el capítulo de validación para sostener la equivalencia entre el libro generado desde la descripción declarativa y el que hoy produce el código Python.

Capítulo 5

El lenguaje

Este capítulo describe el lenguaje **tal como está implementado**, no como convendría que estuviera. Cada forma, cada opción y cada regla que aparece aquí se puede localizar en un archivo del sistema, y se cita el archivo y, cuando existe, la función. Donde el documento de diseño del proyecto dice una cosa y el código dice otra, manda el código, y se advierte en el sitio.

El lenguaje es un DSL interno de Common Lisp. Esa decisión, heredada de la línea de investigación de la facultad ([Arcia Corcho 2017](#)) y estándar en la literatura ([Fowler 2010](#); [Hudak 1996](#)), tiene una consecuencia operativa que conviene fijar antes de seguir: **no hay analizador léxico ni sintáctico**. El lector de Lisp convierte el texto en listas y símbolos ([Steele 1990](#); *American National Standard for Information Systems. Programming Language Common Lisp* 1994), y las macros de `lenguaje/macros.lisp` convierten esas listas en nodos del árbol durante la expansión. El compilador del lenguaje es literalmente ese archivo más `lenguaje/expresiones.lisp`: unas cuatrocientas setenta líneas. En la metodología de seis pasos de ([Arcia Corcho 2017](#)), el paso de construcción del analizador desaparece por completo.

5.1. Visión de conjunto

Una situación, en este lenguaje, son cuatro cosas: **datos**, **derivaciones vivas**, **marcado con significado** y **entrada validada**. Son las cuatro patas del capítulo anterior, y el lenguaje tiene una forma de declaración para cada una. Antes de la letra pequeña conviene ver una descripción completa y corta. La siguiente es `corpus/plan-del-grupo.lisp`, entera, sin recortes:

```
(defsituacion plan-del-grupo (:etiqueta "Plan de asignaturas del grupo D111")

  (coleccion asignaturas
    (:etiqueta "Asignaturas")
    (:clave abbrev)
    (:crece)
    (campo abbrev      :rol fijo      :etiqueta "Abrev")
    (campo nombre      :rol fijo      :etiqueta "Asignatura")
```

```

(campo frecuencia :rol fijo      :tipo entero :etiqueta "Frec")
(campo asignadas  :rol entrada  :tipo entero :etiqueta "Asignadas")
(campo faltan     :rol derivado  :tipo entero :etiqueta "Faltan"
  (- (de fila frecuencia) (de fila asignadas)))
(:datos (("AL" "Algebra Lineal"      3)
  ("L" "Logica"                      2)
  ("IP" "Introduccion a la Programacion" 2)
  ("AM" "Analisis Matematico"         2)
  ("ICD" "Introduccion a Ciencia de Datos" 2)
  ("F" "Fisica"                      2))))

(marca insuficiente
:cuando (< (de fila asignadas) (de fila frecuencia))
:sobre  (nombre asignadas faltan)
:severidad advertencia
:explica (texto (plural (de fila faltan) "Falta" "Faltan") " "
  (de fila faltan) " "
  (plural (de fila faltan) "turno" "turnos"))))

(marca excedida
:cuando (> (de fila asignadas) (de fila frecuencia))
:sobre  (nombre asignadas faltan)
:severidad problema
:explica (texto (plural (- (de fila asignadas) (de fila frecuencia))
  "Sobra" "Sobran")
  " " (- (de fila asignadas) (de fila frecuencia)) " "
  (plural (- (de fila asignadas) (de fila frecuencia))
  "turno" "turnos"))))

(marca completa
:cuando (y (= (de fila asignadas) (de fila frecuencia))
  (no (vacio? (de fila asignadas))))
:sobre  (nombre)
:severidad informativa)

(vista plan :de asignaturas :entrada t :etiqueta "Plan del grupo"))

```

Son cuarenta y siete líneas. Contienen las cuatro patas: los datos están en `:datos`; la derivación viva está en el campo `faltan`; el marcado con significado está en las tres marcas, cada una con un nombre y una severidad; y la entrada está en el campo `asignadas`, el único que el usuario escribe.

Lo que no contienen es igual de importante. No hay una sola coordenada, ni un color, ni una fila de reserva, ni un rango que crezca, ni un `OFFSET`; tampoco se dice qué pasa cuando la hoja se protege. De esta misma descripción salen, sin tocarla, un libro de cálculo con fórmulas vivas, una página web que recalcula al teclear y una tabla de texto plano con una columna de estado en palabras.

5.2. Las formas de declaración

Son seis: `defsituacion`, `coleccion`, `campo`, `marca`, `vista` y `parametro`. Que sean estas seis y no otras es resultado del análisis del capítulo anterior, que es la fase que decide qué conceptos existen en un DSL (Mernik, Heering, y Sloane 2005; Kelly y Tolvanen 2008). Todas se reconocen **por el nombre del símbolo y no por su identidad**, mediante la función `es de lenguaje/expresiones.lisp`, que compara `symbol-name`. La consecuencia práctica es que una descripción se puede escribir desde cualquier paquete sin importar nada, que es como tiene que comportarse un lenguaje interno. La misma función `normalizar` interna todo identificador que entra en el árbol en el paquete `situacion.nucleo`, de modo que el `fila` que escribe el autor y el `FILA` que el análisis espera son el mismo objeto.

5.2.1. `defsituacion`

Definición 4.1 (descripción de situación). Una *descripción de situación* es una forma (`defsituacion NOMBRE OPCIONES . CUERPO`) donde `NOMBRE` es un símbolo, `OPCIONES` es una lista de pares clave-valor y `CUERPO` es una secuencia, en cualquier orden, de declaraciones de parámetro, colección, marca y vista.

La macro está en `lenguaje/macros.lisp`, función `defsituacion`. De las opciones lee **una sola**: `:etiqueta`, que es el título del documento; si no está, usa el nombre capitalizado. El cuerpo se clasifica por el nombre de la cabeza de cada forma —`PARAMETRO`, `COLECCION`, `MARCA`, `VISTA`— y cualquier otra cosa provoca un error de expansión con el nombre de la forma que no supo leer.

La expansión produce tres cosas: un `defparameter` con el nombre dado, ligado al nodo `situacion` construido; una llamada a `registrar-situacion`, que lo guarda en la tabla `*situaciones*` indexado por `symbol-name`; y el nombre como valor. El registro por nombre existe para que las pruebas y las arquitecturas puedan encontrar una descripción sin que su autor exporte nada.

Un punto que conviene no pasar por alto: **`defsituacion` no comprueba nada más allá de la sintaxis**. La comprobación estática —que las variables estén ligadas, que los campos existan, que la colección declare orden si se habla de la fila anterior— es una pasada aparte, `analisis:analizar` en `analisis/comprobacion.lisp`, que se invoca explícitamente antes de materializar (`demonstracion/demonstracion.lisp`, función `generar`). Declarar y validar están separados a propósito: un árbol mal formado se puede construir e inspeccionar, que es lo que necesita una prueba del análisis.

5.2.2. `coleccion`

Definición 4.2 (colección). Una *colección* es un conjunto de filas homogéneas: todas tienen los mismos campos. Se declara con (`coleccion NOMBRE . CUERPO`), donde el cuerpo admite las formas (`:etiqueta T`), (`:clave C ...`), (`:orden C`), (`:crece`), (`:datos ((...) ...)`) y cualquier número de declaraciones (`campo ...`).

La compila `compilar-coleccion`. Cualquier otra forma dentro del cuerpo es un error, con

el mensaje «No se como leer ~s dentro de la coleccion ~a». Las opciones, una por una, tal como las lee el código:

- **:etiqueta** — cómo se muestra. Por defecto, el nombre capitalizado.
- **:clave** — la lista de campos que identifican una fila. Es una lista, no un campo: la clave compuesta no necesita sintaxis adicional. Por defecto, vacía.
- **:orden** — un solo campo. Es lo que habilita hablar de la fila anterior y de la siguiente; sin esta opción, esa forma no compila. Por defecto, **nil**.
- **:crece** — sin argumento. Declara que la lista no está cerrada.
- **:datos** — las filas conocidas al generar, como lista de listas de valores en el orden de los campos de rol **:fijo**.

Definición 4.3 (clave). La *clave* de una colección es la lista de campos cuyo valor identifica una fila dentro de ella. Puede tener más de un elemento.

Definición 4.4 (orden declarado). Una colección tiene *orden declarado* si nombra un campo por cuyo valor está ordenada. Solo sobre una colección con orden declarado son legales las formas de vecindad.

Definición 4.5 (crecimiento). Una colección *crece* si el usuario añadirá filas después de generar el documento. Es un hecho del dominio, no una instrucción sobre filas en blanco.

El campo **origen** del nodo **coleccion** (**nucleo/modelo.lisp**) documenta un valor (**:una-fila-por campo :de coleccion**) para colecciones proyectadas de otras. **compile-coleccion lo fija siempre a :declarada: no hay sintaxis para declarar el otro valor**. El concepto está en el modelo y no está en el lenguaje. Se dice aquí y no se vuelve a mencionar como si existiera.

5.2.3. campo

Definición 4.6 (campo). Un *campo* es un atributo compartido por todas las filas de una colección. Se declara con (**campo NOMBRE :rol R [:tipo T] [:etiqueta E] [:dominio D] [:al-violar A] [:opcional O] [EXPRESION]**).

La compila **compile-campo**. La separación entre las opciones con palabra clave y la expresión suelta del final la hace **partir-opciones**: los pares clave-valor se recogen como opciones y todo lo demás, en orden, como resto. La expresión de un campo derivado va suelta al final, sin palabra clave, porque (**campo faltan :rol derivado (- ...)**) se lee mejor que la alternativa con **:formula**. Esa es la única forma posicional de todo el lenguaje.

Valores por defecto, tal como están en el código: **:rol** es **fijo**, **:tipo** es **texto**, **:al-violar** es **advertir**, **:opcional** es **nil** y **:etiqueta** es el nombre capitalizado.

compile-campo impone tres reglas en tiempo de expansión, y las tres dan error con un mensaje que explica el porqué:

1. Un campo **:rol derivado** sin expresión es un error: «es derivado y no dice cómo se calcula».

2. Un campo que no es derivado y trae una expresión es un error: «Si querías que se calculara, ponle `:ROL DERIVADO`».
3. Un campo con `:dominio` cuyo rol no es `:entrada` es un error, con la razón dicha: «Un dominio dice qué puede escribir el usuario; si nadie escribe ahí, no significa nada».

Definición 4.7 (dominio de entrada). El *dominio* de un campo de entrada es una expresión de conjunto que denota los valores admisibles. `:al-violar` dice qué ocurre cuando el usuario escribe uno que no está: `advertir` avisa sin bloquear, `impedir` bloquea.

La distinción entre restricción dura y restricción blanda es del dominio, no de la hoja de cálculo: los nueve desplegables del corpus se generan con aviso no bloqueante, y el corpus documenta la razón de negocio (capítulo 3). El lenguaje la declara una vez y cada arquitectura la realiza a su manera: en Excel, `validaciones-de` (`excel/emision.lisp`) emite una validación de lista; en la web, `escribir-dominios` (`web/emision.lisp`) emite un par `{bloquea, valores}`, y el render de `web/recursos.lisp` elige un `<select>` cuando bloquea y un `<input>` con `<datalist>` más un aviso «no está en la lista» cuando solo advierte.

Dos observaciones medidas, no interpretadas. `:opcional` se guarda en el nodo y **ninguna arquitectura ni el análisis lo consultan**: hoy es documentación. Y el `:tipo` se normaliza a palabra clave pero **no se valida**: el análisis no comprueba que sea uno de los seis tipos previstos. De los seis, solo `entero` y `numero` tienen efecto observable, en la coerción numérica de los controles de la página (`web/recursos.lisp`).

5.2.4. marca

Definición 4.8 (marca). Una *marca* es una condición booleana, evaluada fila a fila, que asocia a las filas que la cumplen un **estado con nombre** y una **severidad**. Se declara con `(marca NOMBRE :cuando COND :sobre (CAMPO ...)) [:en COLECCION] [:severidad S] [:explica EXPR])`.

La compila `compilar-marca`. `:cuando` y `:sobre` son obligatorias y su ausencia da un error nombrando la marca. La severidad por defecto es `advertencia`, y el análisis (`comprobar-marca`) rechaza cualquiera que no sea `:informativa`, `:advertencia` o `:problema`.

`:en` no aparece en el modelo de diseño y sí en el código, y su razón está en `coleccion-de-la-marca` (`analisis/comprobacion.lisp`): si la marca no dice sobre qué colección se evalúa, se deduce del alcance, y **solo si la deducción es inequívoca**. Si los campos del alcance existen en más de una colección, el análisis da error y pide `:EN`. El comentario del código es explícito sobre el motivo: adivinar en ese caso sería un acierto que depende de cómo estén escritas las cosas, que es exactamente el defecto del color por orden de aparición de (Morales Lazo 2026).

`:explica` acepta cualquier expresión de texto, incluida una cadena literal. En `corpus/defensas-de-tesis.lisp` las dos marcas la usan como cadena; en `corpus/plan-del-grupo.lisp` dos de las tres la construyen con `texto` y `plural`.

5.2.5. vista

Definición 4.9 (vista). Una *vista* es una forma de organizar y recorrer lo que hay. Se declara con (vista NOMBRE :de COLECCION [:etiqueta E] [:entrada T] [:secciones C] [:agrupada-por C]).

La compila `compilar-vista`; `:de` es obligatoria. Una vista no tiene ni una coordenada: las secciones se declaran por el campo que las distingue, no por dónde caen. `:entrada` marca el punto de entrada del documento, que es el concepto de dominio que el corpus descubrió por las malas —el tutor tuvo que preguntar dónde se entraban los datos (capítulo 3).

Estado real, medido: el análisis comprueba que la colección existe y que los campos de `:secciones` y `:agrupada-por` le pertenezcan (`comprobar-vista`). Del lado de las arquitecturas, **solo la de texto plano lee las vistas**, y solo para escribir un índice y señalar cuál es la de entrada (`texto/emision.lisp`). Las de Excel y web emiten una tabla por colección y no consultan ni `:secciones` ni `:agrupada-por`. La navegación como capacidad se pide contando vistas: si hay más de una, `requerimientos` (`protocolo/requerimientos.lisp`) pide `con-navegacion`. Es la parte menos desarrollada del lenguaje y se declara así.

5.2.6. parametro

Definición 4.10 (parámetro). Un *parámetro* es una constante con nombre, propia de la situación. Se declara con (parametro NOMBRE :tipo T :valor V [:rol R] [:etiqueta E]).

La compila `compilar-parametro`. Por defecto `:tipo` es `numero` y `:rol` es `fijo`. Lleva `rol` por la misma razón que un campo: un tope de carga docente se edita dentro del documento; una constante del dominio, no.

Las dos arquitecturas que emiten expresiones lo resuelven **en tiempo de emisión**: `emitir-expresion` sobre un `ref-parametro` busca el parámetro y escribe su valor como literal (`excel/formula.lisp` y `web/expresiones.lisp`). Es decir, hoy `:rol entrada` en un parámetro no produce ningún control. Y hay que decirlo entero: **ninguna de las dos descripciones del corpus declara un parámetro**. La forma está implementada y probada por el lado del núcleo, pero no está ejercitada por el oráculo.

5.3. El rol de campo

Esta es la sección importante del capítulo, porque el rol de campo es el concepto nuevo de este trabajo. No aparece en ninguna de las tres tesis de la línea ([Arcia Corcho 2017](#); [Cápiro Colomar 2025](#); [Morales Lazo 2026](#)).

Definición 4.11 (rol de campo). El *rol* de un campo dice quién pone su valor. Tiene tres valores:

- `:fijo` — el valor se escribe al generar y no cambia.
- `:entrada` — el valor lo escribe una persona, después de generar.
- `:derivado` — el valor se calcula a partir de otros valores de la situación.

Dicho así parece una anotación menor. Lo que lo convierte en un concepto es lo que se sigue de él. De **una sola declaración** salen consecuencias distintas en cada arquitectura, y ninguna de esas consecuencias está escrita en la descripción. La tabla siguiente se lee de las funciones que citan `nucleo:rol` o `nucleo:campos-de-rol` en los tres emisores:

Rol	Hoja de cálculo	Página web	Texto plano
<code>:fijo</code>	celda con valor; bloqueada al proteger la hoja	celda de texto	columna con el valor
<code>:entrada</code>	celda dentro de un rango editable; con dominio, validación de lista	<code><input></code> , o <code><select></code> / <code><datalist></code> según <code>:al-violar</code>	columna con el valor; los admisibles, al pie de la tabla
<code>:derivado</code>	fórmula, recalculada por el motor de la hoja	función en <code>DERIV</code> , reevaluada en cada <code>render()</code>	valor ya calculado por el evaluador de referencia

Las funciones exactas: en Excel, `rangos-editables`, `formulas-de`, `celdas-de` y `validaciones-de`, todas en `excel/emision.lisp`; en la web, `escribir-render`, `escribir-derivaciones` y `escribir-dominios` (`web/emision.lisp`) más el bucle de dibujo de `web/recursos.lisp`; en texto, `escribir-coleccion` (`texto/emision.lisp`).

Hay una cuarta consecuencia que es la que mejor ilustra el punto. `color-de-pestana` (`excel/emision.lisp`) pinta la pestaña de azul si la hoja tiene algún campo de entrada y de gris si todo se calcula. Nadie declaró un color de pestaña. Salió del rol, igual que en los libros del corpus.

De dónde sale el concepto. Sale de la protección de celdas del corpus. El docstring de `comun/proteccion.py` dice qué es y qué no es: «*Sin contraseña a propósito: el objetivo es evitar que se borre una fórmula sin querer, no proteger secretos.*» Proteger es el mecanismo que Excel ofrece; lo que el corpus está declarando es el rol. Que ese sea el riesgo que se quiere evitar no es una manía local: la literatura sobre errores en hojas de cálculo documenta que la fórmula sobrescrita a mano es uno de los defectos más frecuentes y más difíciles de ver (Panko 1998; Hermans, Pinzger, y Deursen 2012).

Sobre la magnitud hay que ser preciso, porque la cifra circula en dos versiones. El comentario de cabecera de `nucleo/modelo.lisp` afirma que «donde hoy hay setenta y cinco llamadas de protección repartidas, hay un solo hecho del dominio». Esa cifra procede del análisis preliminar y **está superada por el conteo reproducible del capítulo 3**: contando por subcadena sobre generadores y pruebas a la vez salen 75; contando con límite de palabra y solo sobre los cuatro paquetes de generadores son 4, porque la protección está factorizada en `comun/proteccion.py`. El comentario del código conserva la cifra vieja y debe corregirse. El argumento no depende de la cifra: sea 4 o 75, hoy la decisión de qué se escribe y qué se calcula vive repartida entre llamadas de protección, listas de celdas editables y fórmulas, en

tres paquetes que no se conocen; aquí es un hecho declarado una vez por campo, del que cada arquitectura deriva lo suyo.

El rol es además la puerta del mecanismo de conformidad. `requerimientos` (`protocolo/requerimientos.lisp`) recorre los campos y, según el rol, pide `con-entrada`, `con-dominio-de-entrada` o `con-derivacion-viva`. Una arquitectura que no pueda cumplir alguna lo dice, y queda en el informe. En el informe de «Plan de asignaturas del grupo D111», texto plano declara una capacidad nativa y degrada cuatro; Excel cumple cuatro de forma nativa y emula una; la web cumple las cinco. Tres informes distintos para una sola descripción, sin tocarla.

5.4. El álgebra de expresión

Las expresiones se compilan en `lenguaje/expresiones.lisp`, funciones `compile-expression` y `compile-forma-compuesta`, y producen los nodos de `nucleo/expresion.lisp`. La lista es cerrada y completa. Son dieciséis formas escritas y quince clases de nodo.

Forma que se escribe	Nodo que construye	Dónde se compila
42, "texto", t, nil	literal	compile-expression
(parametro N)	ref-parametro	compile-forma-compuesta
(de V CAMPO)	ref-campo	compile-acceso
(de (anterior V) CAMPO), (de (siguiente V) CAMPO)	proyeccion sobre vecino	compile-acceso
(+ - * / = /= < <= > >=	aplicacion	forma-de-operador-p
y o no ...)		
(si P E S)	condicional	compile-forma-compuesta
(vacio? X), (sin-escribir X)	sin-escribir	compile-forma-compuesta
(texto ...)	concatenacion	compile-forma-compuesta
(plural N "sing" "plur")	concordancia	compile-forma-compuesta
(los CAMPO de COL [:donde C])	conjunto	compile-conjunto
(cuantas COL [:donde C])	agregado :cuantas	compile-agregado
(suma CAMPO de COL [:donde C])	agregado :suma	compile-agregado-de-campo
(minimo ...), (maximo ...)	agregado :minimo, :maximo	compile-agregado-de-campo
(cuantas-distintas CAMPO de COL [:donde C])	agregado :distintas	compile-agregado-de-campo
(la-fila-de COL :donde C)	busqueda	compile-busqueda
(el CAMPO de FILA [:si-no V])	proyeccion	compile-proyeccion
(existe V :en COL :donde C [:distinta-de W])	existe	compile-existe

Forma que se escribe	Nodo que construye	Dónde se compila
(comparten A B (CAMPO ...))	comparten	compilar-forma-compuesta

Ninguna de estas construcciones está por si acaso. La justificación de cada una contra el corpus —archivo y línea— está en el capítulo 3 y no se repite aquí; la cabecera de `nucleo/expresion.lisp` la reproduce en forma de tabla para que el lector del código la tenga a mano.

Tres detalles del compilador que no se ven en la tabla. El valor por defecto de `:si-no` en una proyección es la cadena vacía, no `nil` (`compilar-proyeccion`); eso hace que la guarda de ausencia sea la conducta por omisión y no algo que haya que recordar escribir. Las formas (`anterior V`) y (`siguiente V`) sueltas **no compilan**: dan un error que dice cómo escribirlas, «(DE (`anterior V`) CAMPO)», porque el vecino solo tiene sentido si se dice qué campo suyo se quiere. Y (`de (anterior V) CAMPO`) no produce un `vecino` desnudo sino una `proyeccion` sobre él con valor por defecto `nil`: la primera fila no tiene anterior, y eso es un caso normal, no un error. El documento de diseño lista (`anterior v`) como forma del álgebra; el código no la admite suelta, y aquí manda el código.

También hay que decir lo que el diseño anuncia y el código no tiene: la forma (`las-filas-de COL :donde C`), la relación uno a muchos, **no está implementada**. No aparece en `lenguaje/expresiones.lisp` ni hay nodo para ella en `nucleo/expresion.lisp`. La capacidad `con-relacion-uno-a-muchos` sí está declarada en `protocolo/capacidades.lisp`, y por tanto ninguna descripción puede pedirla hoy.

5.4.1. Por qué un solo nodo para los operadores

Aritmética, comparación y lógica van en **un solo nodo**, `aplicacion`, con el operador guardado como símbolo. La lista de operadores es el parámetro `+operadores+` de `lenguaje/expresiones.lisp`: `+ - * / = /= < <= > >= Y O NO`.

(Morales Lazo 2026) resuelve esto al revés: tiene `expr-gt`, `expr-gte`, `expr-lt`, `expr-equals`, `expr-add`, `expr-subtract` y una clase más por cada operación. Es una decisión razonable si solo va a existir un destino. Deja de serlo en cuanto hay dos, y por una razón estructural: en un sistema de funciones genéricas el despacho es sobre la clase del nodo (Keene 1989), así que **una clase por operador significa un método por operador y por arquitectura**.

Con un solo nodo, una arquitectura escribe **un método y una tabla**. Se puede comprobar en el código, y los dos casos son instructivos porque las dos tablas no se parecen:

```
;; excel/formula.lisp
(defparameter +infijos+
  '((+ . "+") (- . "-") (* . "*") (/ . "/")
    (= . "=") (/= . "<>") (< . "<") (<= . "<=") (> . ">") (>= . ">=")))
```

El método de Excel usa la tabla para los diez infijos y trata los tres lógicos aparte, porque

en una hoja de cálculo se escriben como llamadas: `AND(...)`, `OR(...)`, `NOT(...)`. El de la web usa otra tabla y tiene que hacer algo más:

```
;; web/expresiones.lisp
(defparameter +operadores-js+
  '(("+" . "+") ("-" . "-") ("*" . "*") ("/" . "/")
    ("=" . "===") ("/=" . "!==" )("<" . "<") ("<=" . "<=")
   (">" . ">") (">=" . ">=")))
```

Las comparaciones numéricas se envuelven en una coerción, `N(...)`, porque en JavaScript `"10" < "9"` es cierto; y la igualdad se delega en una función `IG` en vez de en `===`, por la misma razón. El comentario del código lo dice: es la misma coerción que hace el evaluador de referencia, y por eso los dos coinciden. **Ese es el argumento del diseño en una frase:** dos arquitecturas con convenciones de comparación distintas cupieron cada una en un método, y cuando llegue la tercera será un método más, no trece.

5.5. Ámbitos y la semántica de fila

Esta sección es la crítica del capítulo, porque contiene la regla que impide que el modelo de la hoja de cálculo se cuele dentro del lenguaje.

La regla es esta: la iteración es siempre explícita. No existe «la fila actual» implícita ni «la fila de arriba». No hay manera de escribir una referencia a un campo sin nombrar la variable de fila: el nodo `ref-campo` (`nucleo/expresion.lisp`) tiene dos ranuras, `variable` y `campo`, y las dos son obligatorias. `compilar-expresion` rechaza un símbolo suelto fuera de un `:donde` con un mensaje que dice qué escribir en su lugar: «Fuera de un `:DONDE` hay que decir de qué fila se habla, con `(DE FILA ...)` o `(DE OTRA ...)`».

Definición 4.12 (ámbito). Un *ámbito* es el par formado por (a) un conjunto de ligaduras, cada una de la forma *símbolo* $\rightarrow$ *colección*, y (b) cuál de esas variables es la fila que se está evaluando.

La clase está en `nucleo/ambito.lisp` y tiene exactamente dos ranuras, `ligaduras` y `actual`. **No contiene ninguna dirección.** Es la pieza que sustituye a los parámetros `fila`, `primera-fila` y `ultima-fila` de `compile-excel-formula` de (Morales Lazo 2026): aquellos tres decían *dónde* estaba la fila; este dice *de qué colección* es. La diferencia es que lo segundo significa algo en cualquier destino.

`ambito-extendido` devuelve un ámbito nuevo en vez de modificar el que recibe, porque un ámbito se comparte entre las ramas de una expresión y no debe cambiar bajo los pies de nadie. Es una decisión de una línea con consecuencias: el recorrido del análisis puede bajar por varias ramas sin copiar ni deshacer nada.

Las cuatro reglas que impone el ámbito, declaradas en la cabecera de `nucleo/ambito.lisp` y comprobadas en `analisis/comprobacion.lisp`, función `comprobar-expresion`:

1. **Toda derivación se evalúa en un ámbito.** El de la expresión de un campo derivado liga `FILA` a la colección de ese campo (`ambito-de-coleccion`).

2. (de `V CAMPO`) exige que `V` esté ligada y que `CAMPO` pertenezca a su colección. El mensaje de error enumera las variables ligadas en ese punto y, en el otro caso, los campos que la colección sí tiene.
3. existe, la-fila-de, los agregados y los ligan una variable nueva y extienden el ámbito para su condición. En el código es una sola cláusula `typecase` que cubre los cuatro tipos de nodo, construye el ámbito interno y baja las hijas por él.
4. La vecindad exige orden declarado.

La cuarta regla merece su propio párrafo. `comprobar-expresion`, ante un nodo `vecino`, busca la colección de la variable y comprueba que declare orden. Si no lo declara, el error dice por qué:

```
se pide la fila anterior de citaciones, pero esa coleccion no declara orden.
Sin orden declarado, "anterior" solo puede significar "la de arriba", que es
una propiedad de como se dibuja y no del problema. Anade (:orden <campo>) a
la coleccion.
```

Lo decisivo es **dónde** se comprueba eso: en `analisis/comprobacion.lisp`, que no depende de ningún paquete de arquitectura. El error sale antes de que exista un destino. Esa es la diferencia con el `previous-of` de (Morales Lazo 2026), que de hecho significaba «la fila de arriba en la rejilla»: allí la corrección de la expresión dependía de que el destino tuviera rejilla, y por tanto no era comprobable sin él. Aquí «anterior» significa «la anterior según el orden declarado», que se traduce sin ambigüedad a un índice de arreglo, a una función de ventana sobre ese mismo `ORDER BY`, o a una referencia relativa en una hoja —esto último porque la arquitectura de Excel se compromete a escribir las filas en ese orden, y queda registrado en su plan.

Hay un tercer efecto, y es el que lo vuelve operativo: la vecindad **pide una capacidad**. `pedir-por-expresion` (`protocolo/requerimientos.lisp`) traduce cualquier nodo `vecino` en una petición de `con-orden-declarado`. Una arquitectura que no pueda garantizar el orden lo declarará y quedará en el informe, en vez de producir en silencio un documento donde «anterior» significa otra cosa.

5.6. La decisión de lectura del `:DONDE`

Es la decisión de lectura más importante del lenguaje, y está documentada como tal en la cabecera de `lenguaje/expresiones.lisp`.

La regla: dentro de un `:donde`, un símbolo suelto es un campo de la fila candidata. Para hablar de la fila de fuera hay que decirlo con `(de fila ...)`.

La implementa el parámetro `en-donde` de `compilar-expresion`: cuando vale cierto y la forma es un símbolo, construye un `ref-campo` sobre la variable `+variable-candidata+`, que es el símbolo `CANDIDATA` del paquete del núcleo. Si hay búsquedas anidadas, la interior la sombrea, que es la regla de alcance habitual.

El efecto es que la búsqueda por clave se escribe así:

```
(la-fila-de tesis :donde (= estudiante (de fila estudiante)))
```

Léase en voz alta: «*la fila de tesis cuyo estudiante es el estudiante de esta fila*». Es exactamente la frase del dominio. Compárese con la alternativa, que sería (= (de candidata estudiante) (de fila estudiante)): dice lo mismo y no se lee igual.

Este es el criterio de calidad heredado de (Cápiro Colomar 2025), y conviene decir de dónde viene. LDMAg genera con **def-entidad** dos accesorios por atributo, con las dos variantes gramaticales del español, y pluraliza el nombre de la entidad por su cuenta. El propósito declarado es que la descripción se lea como español, y ese propósito es el que se hereda: **el tutor tiene que poder leer una descripción y decir si es correcta sin que se la expliquen**. Ninguna prueba automática sustituye ese criterio. Lo que no se hereda es el mecanismo: aquí no se generan accesorios por atributo, se elige una regla de lectura que hace que la frase salga bien sin generar nada.

Hay una excepción medida, y es real. **existe no aplica esta regla**. **compile-existe** compila su **:donde** con **en-donde** en falso, de modo que dentro de un **existe** un símbolo suelto da error y hay que escribir (de otra dia), (de fila dia). La razón está en el diseño de la forma: **existe liga una variable con nombre propio** —(existe otra :en citasiones ...)— y esa variable convive con la de fuera, así que un símbolo suelto sería ambiguo entre las dos. En **la-fila-de**, **cuantas**, **suma**, **minimo**, **maximo**, **cuantas-distintas** y **los**, en cambio, la variable candidata no tiene nombre visible y el símbolo suelto no puede referirse a otra cosa. La regla, dicha con precisión: **el símbolo suelto se admite exactamente donde la variable de fila es anónima**. La cabecera del archivo enuncia la regla sin esa salvedad; el código la tiene, y es el código el que manda.

5.7. La marca declara significado

Definición 4.13 (severidad). La *severidad* de una marca es su peso dentro del documento, y es una de tres: **:informativa**, **:advertencia**, **:problema**. Una marca no declara un color: declara un nombre —el significado— y una severidad, y cómo se ve lo decide cada arquitectura.

El contraste con (Morales Lazo 2026) está en el capítulo 2 y aquí solo se usa su conclusión: allí la definición 3.8 dice que una regla de marcado tiene una condición y un color de fondo, el nodo del árbol no tiene color, y las tres reglas del caso real quedan distinguidas por comentarios y por el orden en que están escritas. El color aparece por primera vez aguas abajo, en el JSON. La figura 6.2 de ese trabajo muestra los tres colores corridos.

Aquí las tres marcas de **corpus/plan-del-grupo.lisp** se llaman **insuficiente**, **excedida** y **completa**, y cada una declara su severidad. La misma declaración se realiza de tres maneras distintas:

Arquitectura	Realización	Dónde
Hoja de cálculo	relleno de celda por severidad: FFC7CE, FFEB9C, DDEBF7	+paleta+ y relleno-de, excel/emision.lisp
Página web	clase de CSS por severidad: problema, advertencia, informativa	+paleta-web+, web/emision.lisp; los colores, en web/recursos.lisp
Texto plano	una palabra en la columna Estado	texto-del-estado, texto/emision.lisp

Las tres tablas son de tres líneas, ninguna está en el lenguaje y ninguna sabe de las otras. El comentario de `+paleta+` lo dice: la marca declara un significado y una severidad; que el problema se vea rojo es decisión de esta arquitectura.

La tercera realización es la que prueba que la decisión era necesaria. `texto-del-estado` prefiere la explicación de la marca si la tiene, porque está escrita para leerse, y si no la tiene escribe el nombre del estado. Ese nombre es el significado que declara la marca, no un color, y por eso se puede escribir tal cual. El artefacto de texto plano de «Plan de asignaturas del grupo D111» produce en esa columna, medido: `completa`, `Sobra 1 turno`, `Falta 1 turno`. Y la fila `AM` —frecuencia 2, asignadas 3— sale como `excedida`, que es justo la fila que la figura 6.2 de (Morales Lazo 2026) pinta mal. En una impresión en blanco y negro, una regla que solo lleva color no se ve; una que lleva significado, sí. El color no sobrevive al cambio de arquitectura; el significado sí.

5.8. Lo que el lenguaje NO dice

El criterio de admisión de una construcción es la distinción entre **intención** y **mecanismo**. Una intención es un hecho del problema que se describe; un mecanismo es lo que un destino concreto se inventa para cumplirla. El lenguaje dice intenciones. Nunca nombra mecanismos.

El ejemplo canónico es el crecimiento:

- **:crece sí.** Dice que la lista no está cerrada y que el usuario añadirá filas después de generar. Es un hecho del dominio, comprobable preguntando a quien usa el documento.
- **«reserva veinte filas» no.** Es lo que una arquitectura de rejilla se inventa para cumplirlo. Y de hecho, la fórmula de reserva la decide `capacidad` (`excel/plan.lisp`): la mitad de los datos, con un mínimo. La página web no reserva nada: pone un botón, y por eso declara `con-crecimiento` como cumplida de forma nativa mientras Excel la declara emulada.

El mismo criterio aplicado a las otras decisiones que el lenguaje deja fuera, y dónde acaban viviendo:

Lo que el lenguaje no dice	Quién lo decide
en qué columna cae un campo	<code>excel/plan.lisp</code>
cuántas filas se reservan	<code>capacidad</code> , <code>excel/plan.lisp</code>
el color de cada severidad	<code>+paleta+</code> / <code>+paleta-web+</code>
el color de la pestaña	<code>color-de-pestana</code> , derivado del rol
si la hoja se protege, y qué rangos quedan abiertos	<code>rangos-editables</code> , derivado del rol
la guarda <code>IF(...="", "", ...)</code> de las filas vacías	<code>con-guarda-de-fila</code> , <code>excel/emision.lisp</code>
si la comparación numérica necesita coerción	<code>web/expresiones.lisp</code>
si las derivaciones se recalculan o se congelan	la capacidad <code>con-derivacion-viva</code> y, si falta, <code>resolver-carencia</code>

La guarda merece una nota, porque es donde la frontera se ve mejor. El corpus abre casi toda fórmula con `IF(...="", "", ...)`, y la razón es que en Excel y en Calc una referencia a una celda vacía se muestra como 0. Eso es un problema de las filas de reserva, que son un mecanismo de rejilla. Por tanto la guarda **no está en el lenguaje**: la pone `con-guarda-de-fila` al emitir, y solo cuando la colección declara `:crece`; una página web no tiene filas vacías. Lo que sí está en el lenguaje es `vacio?`, que significa otra cosa: *este campo de entrada todavía no se ha rellenado*, que es un hecho del dominio. La marca `completa` del plan la usa para no declararse completa sobre una fila en la que nadie ha escrito.

Este es también el filtro que se aplicó al heredar de (Morales Lazo 2026). `expr-counta` — contar celdas no vacías — no entra: en el dominio la pregunta es cuántas filas hay, y eso es (`cuantas coleccion`). La prueba de fuego es corta: **si al nombrar un concepto hay que mencionar una celda, el concepto pertenece al destino**.

5.9. Una descripción completa, comentada

`corpus/defensas-de-tesis.lisp` es la descripción que tiene a la vez las tres cosas difíciles: búsqueda por clave, entrada validada y cuantificación existencial. Ninguna tesis anterior de la línea puede describir las tres.

Empieza por lo que ya se sabe y no se toca:

```
(coleccion tesis
  (:etiqueta "Tesis")
  (:clave estudiante)
  (campo estudiante :rol fijo :etiqueta "Estudiante")
  (campo tutor      :rol fijo :etiqueta "Tutor")
  (campo oponente   :rol fijo :etiqueta "Oponente")
  (campo presidente :rol fijo :etiqueta "Presidente")
  (campo secretario :rol fijo :etiqueta "Secretario")
  (:datos (("Juan Perez" "P. Alonso" "M. Ramirez" "L. Gomez" "A. Suarez")
    ...)))
```

Cinco campos fijos y una clave de un solo campo. Nadie escribe aquí: es el catálogo que la búsqueda va a consultar.

Después viene lo que se planifica a mano:

```
(coleccion citaciones
  (:etiqueta "Citaciones")
  (:clave dia momento local)
  (:orden momento)
  (:crece)
  (campo dia      :rol fijo :etiqueta "Dia")
  (campo momento :rol fijo :etiqueta "Momento")
  (campo local    :rol fijo :etiqueta "Local"))
```

Tres cosas declaradas de una vez. La clave es **compuesta**: una citación se identifica por el día, el momento y el local, y eso se dice con una lista, sin inventar un separador ni concatenar nada —el # de horarios/hoja_grupo.py:291 es mecanismo de Excel; el par es dominio. Hay **orden declarado** por momento, que es lo que permitiría hablar de la citación anterior. Y **crece**, porque siempre aparece un caso más.

El único campo que se escribe a mano:

```
(campo estudiante :rol entrada
  :etiqueta "Estudiante"
  :dominio (los estudiante de tesis)
  :al-violar advertir)
```

Rol de entrada, con dominio y con la severidad de la violación declarada. El dominio es una expresión de conjunto, no una lista fija: si alguien da de alta una tesis, el desplegable se entera. `escribir-dominios` (`web/emision.lisp`) lo emite como función precisamente por eso. Y `advertir`, no `impedir`, porque hay que poder anotar un caso antes de darlo de alta; el comentario de la descripción lo dice con esas palabras.

El tribunal no se escribe: se trae solo.

```
(campo tutor :rol derivado :etiqueta "Tutor"
  (el tutor de (la-fila-de tesis
    :donde (= estudiante (de fila estudiante)))
    :si-no ""))
```

Los cuatro campos del tribunal siguen ese molde. Léase: *«el tutor de la fila de tesis cuyo estudiante es el estudiante de esta fila, y si no hay ninguna, nada.»* Dentro del `:donde`, `estudiante` suelto es el campo de la fila candidata; `(de fila estudiante)` es el de la citación. El `:si-no ""` no es un adorno: es la mitad del concepto, porque el usuario escribe antes de que exista la fila relacionada, y por eso las once búsquedas del corpus van envueltas en una guarda de ausencia. Este bloque es, en una línea de descripción, lo que en `tribunales/hoja_dia.py:40-43` son tres funciones de Excel anidadas.

La razón de ser del documento es la marca:

```
(marca profesor-citado-dos-veces
  :en citasiones
  :cuando (existe otra :en citasiones
    :distinta-de fila
    :donde (y (= (de otra dia) (de fila dia))
      (= (de otra momento) (de fila momento))
      (comparten otra fila
        (tutor oponente presidente secretario))))
  :sobre      (tutor oponente presidente secretario)
  :severidad problema
  :explica    "Este profesor esta citado en dos locales a la vez")
```

Aquí está todo lo del capítulo junto. `:en citasiones` desambigua, porque `tutor`, `opponente`, `presidente` y `secretario` existen en las dos colecciones y la deducción por alcance sería un acierto por azar. `existe otra` liga una variable **con nombre propio**, y por eso dentro de su `:donde` hay que escribir `(de otra dia)` y `(de fila dia)`: las dos filas conviven y ninguna es la candidata anónima. `:distinta-de fila` dice explícitamente que una citación no colisiona consigo misma; en (Morales Lazo 2026) esa exclusión se escondía en la fórmula generada. Y `comparten` evita desplegar dieciséis comparaciones —cuatro roles contra cuatro— en una frase del dominio: *estas dos citasiones comparten algún miembro de tribunal*.

La segunda marca es de una línea y sirve de contraste:

```
(marca sin-asignar
  :en citasiones
  :cuando      (vacio? (de fila estudiante))
  :sobre       (estudiante)
  :severidad informativa
  :explica     "Este hueco todavia no tiene defensa")
```

Severidad informativa, no problema: un hueco sin asignar no es un error, es un estado. Que el lenguaje distinga las tres severidades es lo que permite que la hoja lo pinte de azul claro y no de rojo, sin que la descripción mencione ningún color.

Cierra con dos vistas:

```
(vista dia :de citasiones :entrada t :etiqueta "Citasiones del dia"
  :agrupada-por local)
(vista tribunal :de tesis :etiqueta "Tesis y sus tribunales")
```

Dos vistas significan navegación, y `requerimientos` lo traduce en una petición de `con-navegacion`. Texto plano no la tiene y la **emula** apilando las vistas con un índice al principio; Excel y la web la cumplen. Esa diferencia queda en el informe de conformidad: para esta descripción, texto plano declara 2 capacidades nativas, 1 emulada y 5 degradadas; Excel, 7 nativas y 1 emulada; la web, 8 nativas. Una sola descripción, tres informes distintos, ninguno silencioso.

Y el resultado medido, que es lo que cierra el capítulo: con los datos de `datos-de-las-`

defensas —dos citaciones el lunes a las 09:00 en locales distintos, que comparten a A. Suárez como secretario— las dos filas salen marcadas, en las tres arquitecturas. La colisión que el documento existe para detectar, la detecta.

Capítulo 6

La arquitectura del sistema

Este capítulo describe el sistema construido: cómo está dividido, por qué está dividido así, y qué parte de esa división es una afirmación y qué parte es algo que la herramienta impide violar. El código al que se refiere está en el directorio `situacion/` del repositorio, se carga con `ASDF` y se ejecuta con un `SBCL` recién instalado y nada más.

La descripción que sigue no es una propuesta. Todo lo que se afirma aquí se leyó del código que compila y produce artefactos, y cuando el documento de diseño previo y el código no coinciden, se describe el código y se dice dónde está la diferencia. El diagnóstico central de esta tesis sobre (Morales Lazo 2026) surgió precisamente de contrastar lo que un documento afirmaba con lo que una firma de función permitía; sería incoherente reproducir aquí el mismo error a favor propio.

La experimentación que sostiene las afirmaciones de este capítulo está en el capítulo 7. Aquí se dice qué se construyó y por qué; allí, cómo se comprobó.

6.1. Criterios de diseño

El sistema se construyó contra seis criterios fijados de antemano. La diferencia con los «principios de diseño» que enuncian los tres trabajos previos de la línea no está en el contenido sino en el estatuto: **cinco de los seis se comprueban con una prueba automática**, y una afirmación que se comprueba deja de ser una afirmación.

C1 — Ninguna palabra de direccionamiento cruza hacia arriba. Ni en el núcleo ni en el protocolo puede aparecer vocabulario de posición: letra de columna, coordenada, número de fila, rango, celda, hoja, libro, el nombre de ninguna función de hoja de cálculo, `openpyxl` ni `xlsx`.

El criterio lleva un matiz sin el cual la prueba sería teatro: **fila sí es palabra del dominio**. Una colección tiene filas y (`de fila campo`) es vocabulario legítimo. Lo prohibido es la *dirección* de una fila, no su existencia. La lista es de direccionamiento, no de estructura. Está escrita en `pruebas/invariantes.lisp`, en `*vocabulario-de-direccionamiento*`, con esa justificación en el propio archivo.

C2 — Toda operación del protocolo recibe la arquitectura y despacha sobre ella. Es el criterio que (Morales Lazo 2026) incumple en el punto exacto donde más importa, y el que aquí no se vigila sino que se hace cumplir: la macro `definir-operacion`, en `protocolo/operacion.lisp`, rechaza en tiempo de expansión cualquier operación cuya lista de parámetros no empiece por `arquitectura`. El sistema no compila si se intenta.

C3 — Las arquitecturas no ven el lenguaje. Un backend depende del protocolo y de nada más. No puede usar las macros de declaración ni por descuido, porque el sistema en el que vive no las tiene cargadas. La independencia entre la sintaxis y el contrato de extensión deja de ser una promesa y pasa a ser una propiedad del grafo de dependencias de `situacion.asd`.

C4 — Ninguna arquitectura ve a otra. No hay dependencias entre backends. El de la página web no puede reutilizar nada del de la hoja de cálculo, ni al revés. Lo que dos compartan sube a una clase de capacidad, que es el único sitio donde se comparte.

C5 — Lo que una arquitectura se inventa es suyo y no sale de ahí. Las hojas auxiliares, las columnas de apoyo, las filas de reserva y el plano de libro pertenecen a la arquitectura que los necesita. Ningún otro sistema los referencia, y hoy no puede: ninguno depende de `situacion/excel`.

C6 — Un archivo, una responsabilidad. Es regla del proyecto. No se comprueba sola, pero sí se vigila: el archivo más grande del sistema tiene 397 líneas y es el evaluador de referencia.

Criterio	¿Prueba automática?	Cómo
C1	sí	I1, recorrido de símbolos de los dos paquetes
C2	sí	I2, más el rechazo en tiempo de expansión de <code>definir-operacion</code>
C3	sí	I3, sobre las dependencias declaradas en <code>situacion.asd</code>
C4	sí	I3, regla específica para los sistemas de arquitectura
C5	sí, indirectamente	I3: nadie depende de <code>situacion/excel</code>
C6	no	revisión, con el tamaño de archivo como señal

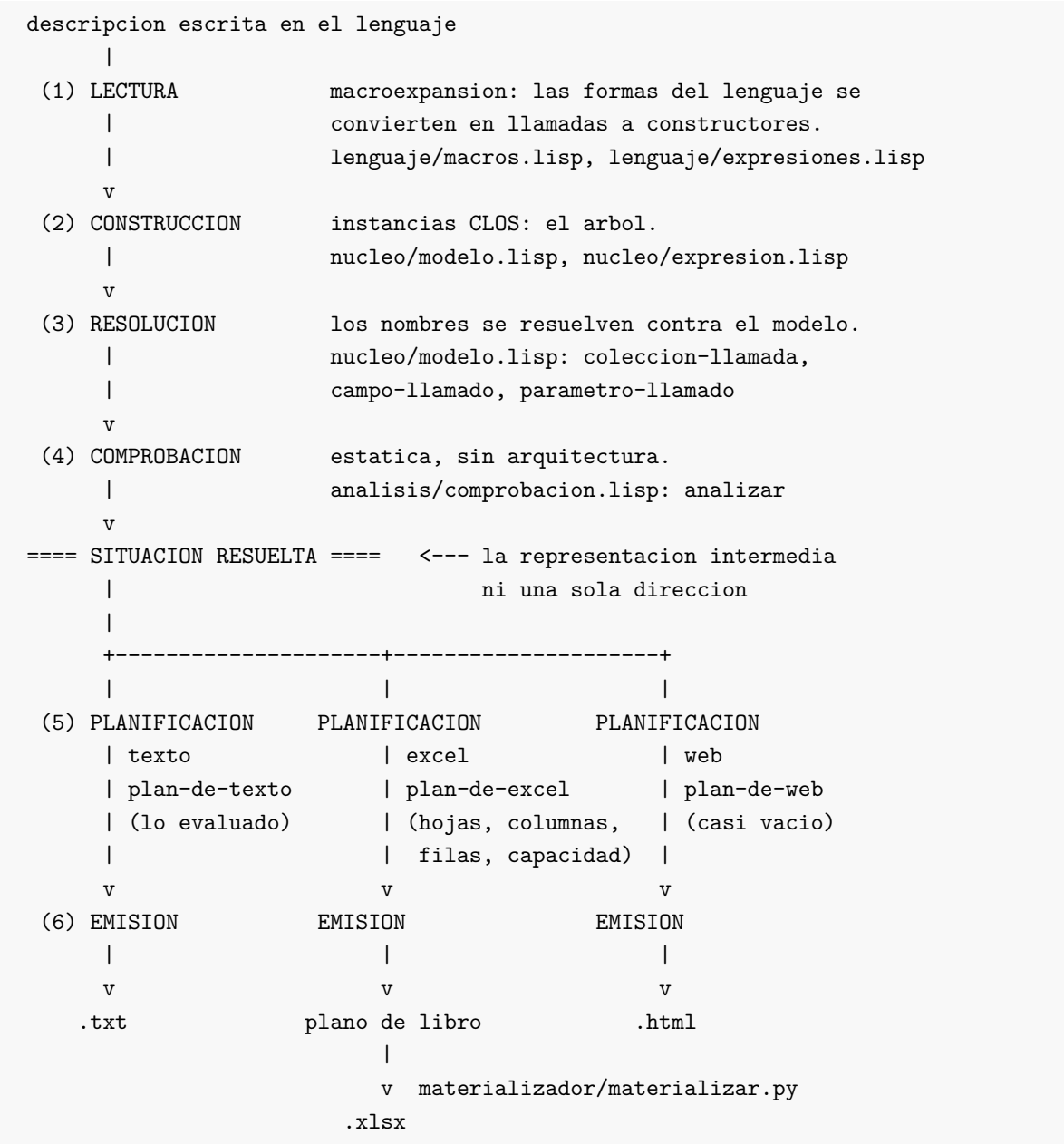
Los tres invariantes están implementados y su ejecución se reporta en el capítulo 7, junto con la prueba —imprescindible— de que cada uno *muerde*: que detecta el caso que dice detectar y no pasa en verde por una lista vacía o un error de lógica.

Hay un séptimo criterio que **no** es automático y conviene declararlo como tal, porque es el

que más peso tiene con el cliente y ninguna prueba lo alcanza: el usuario tiene que poder leer una descripción y decir si es correcta sin que se la expliquen. Si hace falta explicarla, el vocabulario está mal elegido. Es la mejor idea heredada de (Cápiro Colomar 2025), y se comprueba enseñándosela a una persona.

6.2. Las seis fases de la compilación

El sistema compila en seis fases. La frontera que importa cae entre la cuarta y la quinta: **las cuatro primeras no conocen ninguna arquitectura**, y su salida —la situación resuelta— es la representación intermedia cuya independencia afirma esta tesis.



Fase 1, lectura. Las macros del sistema `situacion/lenguaje` expanden las formas del lenguaje a llamadas a los constructores del núcleo. Aquí se cobra la ventaja que identifica

([Arcia Corcho 2017](#)) al construir un lenguaje embebido: no hay analizador léxico ni sintáctico porque los hace el intérprete del anfitrión ([Graham 1993](#); [Seibel 2005](#)). Es el paso 5 de los seis de la metodología de 2017 desapareciendo por completo.

Fase 2, construcción. Instancias CLOS. Nada está resuelto todavía: los nombres siguen siendo símbolos y un campo derivado guarda su expresión como árbol, nunca como texto.

Fase 3, resolución. Aquí hay que ser preciso, porque el documento de diseño previo describe una pasada de resolución que convierte símbolos en referencias a objetos y construye un grafo de dependencias global. **Eso no existe en el código, y manda el código.** Los nombres siguen siendo símbolos durante toda la vida del árbol, y se resuelven contra el modelo cada vez que hacen falta, con tres funciones de búsqueda de `nucleo/modelo.lisp`: `coleccion-llamada`, `campo-llamado` y `parametro-llamado`. El grafo de dependencias tampoco es global: la detección de ciclos la hace `comprobar-ciclos`, en `analisis/comprobacion.lisp`, construyendo una tabla de dependencias local a cada colección, y solo con las referencias que se quedan dentro de la misma fila.

La diferencia hay que declararla. A esta escala —colecciones de decenas de campos— la búsqueda lineal es irrelevante, y no materializar la resolución en una estructura aparte evita que haya dos representaciones del mismo árbol que puedan desincronizarse. Pero significa que la fase 3 no es hoy una pasada con salida propia, sino una operación que las fases 4, 5 y 6 invocan cuando la necesitan.

Fase 4, comprobación. `analisis/comprobacion.lisp` contesta todo lo que se puede contestar mirando solo la descripción: referencias a campos que no existen, variables de fila sin ligar, ciclos entre derivados, marcas cuyo alcance no cae en ninguna colección o cae en dos, vistas que muestran una colección inexistente, severidades que no son una de las tres. La comprobación central —y la que impide que se cuele el modelo de la rejilla— es la del orden: (`anterior` fila) solo es legal sobre una colección que declare por qué campo está ordenada. El mensaje de error lo explica al usuario en esos términos:

```
Sin orden declarado, "anterior" solo puede significar "la de arriba",  
que es una propiedad de como se dibuja y no del problema.  
Anade (:orden <campo>) a la coleccion.
```

Esta fase atiende, además, una recomendación explícita de ([Morales Lazo 2026](#)), que pedía una herramienta de validación estática del árbol que detectase referencias a columnas inexistentes antes de la fase de generación.

Fase 5, planificación. Esta fase es nueva. Ni ([Cápiro Colomar 2025](#)) ni ([Morales Lazo 2026](#)) la tienen: en los dos, decidir y escribir ocurren en la misma pasada. Separarlas resuelve dos problemas distintos, y conviene enunciarlos por separado porque se suelen confundir.

Primero: le da a la degradación un sitio donde ocurrir. Si decidir y escribir son lo mismo, el momento en que un backend descubre que no puede hacer algo lo sorprende a mitad de un `format`, y lo único que puede hacer es improvisar —callar, omitir o escribir algo aproximado— sin dejar rastro. Con una fase de planificación existe un punto del proceso, anterior a toda escritura, en el que la arquitectura declara qué le están pidiendo y qué va a hacer con cada

cosa. El informe de conformidad es el resultado de ese punto. Sin fase 5 no hay informe, y sin informe la extensibilidad vuelve a ser una afirmación.

Segundo: hace que la emisión no necesite recibir direcciones. Éste es el mecanismo técnico que desarma el problema de 2026. Si la escritura de una expresión ocurre antes de que se sepa dónde va todo, la función que escribe tiene que recibir por parámetro dónde está la fila, dónde empieza la tabla y dónde acaba —que es exactamente la firma de `compile-excel-formula`—. Con la planificación separada, cuando llega la emisión el plan **ya** sabe dónde va cada cosa, y el backend se lo pregunta a su propio plan en vez de recibirlo de fuera. La consecuencia se ve en el código: en `excel/formula.lisp` la fila que se está generando viaja en la variable especial `*fila*`, declarada en ese archivo y propia de ese paquete. No cruza el protocolo.

En la implementación, la fase 5 la abre `materializar`, que barre `protocolo:requerimientos` declarando capacidad por capacidad lo que la descripción pide, y después llama a `planificar`. Conviene anotar otra diferencia con el documento de diseño: la firma documentada de `planificar` dice devolver `(values plan informe)`, pero el informe lo construye la macro `con-informe` alrededor del cuerpo entero de `materializar`, y `planificar` devuelve solo el plan. El efecto observable es el mismo, pero la documentación del protocolo está por detrás del código en ese punto.

Fase 6, emisión. El backend escribe su artefacto consultando su plan. Para texto y para web eso es escritura directa a un flujo. Para la hoja de cálculo es un plano de libro en JSON que un materializador en Python convierte en `.xlsx` con `openpyxl`, porque no existe biblioteca de Common Lisp capaz de escribir OOXML (*ECMA-376: Office Open XML File Formats 2016*) con fórmulas, formato condicional, validación de datos, rangos nombrados y protección de celdas.

6.3. El evaluador de referencia

`nucleo/evaluador.lisp` es el archivo más grande del sistema: 397 líneas. Dada una situación y unos datos, calcula todos los valores derivados y dice qué marcas disparan en cada fila. Su punto de entrada es `evaluar-situacion`.

Su justificación inmediata es mecánica. Una arquitectura con derivaciones congeladas —un documento impreso, una página estática, y en este sistema la de texto plano— necesita que alguien evalúe las expresiones antes de escribirlas. Ese alguien no puede ser el backend: si cada uno evaluara por su cuenta, cada uno tendría su propia semántica y la afirmación de que la descripción es independiente del destino no significaría nada. La prueba de que esto no es hipotético está en `texto/emision.lisp`: la arquitectura de texto no tiene **ningún** método de `emitir-expression`, y no lo necesita, porque su plan consiste en el resultado del evaluador. Sin evaluador de referencia, esa arquitectura no podría existir sin reimplementar el lenguaje por su cuenta.

Pero la pieza trae tres cosas más, y son las que la hacen central.

Primera: es la definición de qué significa el lenguaje. La semántica deja de ser «lo

que haga la hoja de cálculo» y pasa a estar escrita en un sitio localizable. Ése es exactamente el punto ciego que comparten los tres trabajos previos de la línea: ninguno declara su modelo de evaluación, y por eso en los tres la semántica termina siendo la del destino. Con evaluador, toda arquitectura tiene que coincidir con él, y coincidir o no es una pregunta con respuesta.

Segunda: es un oráculo que no depende de ningún destino. Se puede probar una búsqueda por clave, un cuantificador existencial o un conteo de distintos sin generar ningún archivo y sin tener escrito ningún backend. Eso permitió desarrollar y probar el álgebra de expresión completa antes de que existiera la primera arquitectura, que es el orden inverso al de los tres trabajos anteriores.

Tercera: cierra el lazo de verificación. El evaluador da el valor esperado y la suite de conformidad comprueba que cada arquitectura da el mismo. En la demostración, ese oráculo se escribe a disco como `salida/<situacion>-esperado.json` y es contra él que se comparan los valores que LibreOffice calcula al abrir el libro y los que la página calcula al cargarse. Los resultados están en el capítulo 7.

6.3.1. La coerción, y por qué está en un solo sitio

Un campo sin rellenar vale NIL o cadena vacía. Un número escrito a mano puede llegar como texto. Comparar dos celdas vacías tiene que dar cierto. Esas decisiones son semántica del lenguaje, no detalles de implementación: cambiarlas cambia el resultado de una descripción. Por eso están normalizadas en un solo lugar del evaluador, en cuatro funciones cortas:

```
(defun vacio-p (valor)
  (or (null valor) (and (stringp valor) (string= valor ""))))

(defun como-numero (valor)
  (cond ((realp valor) valor)
        ((vacio-p valor) 0)
        ((stringp valor) (or (ignore-errors (read-from-string valor)) 0))
        (t 0)))

(defun como-texto (valor)
  (cond ((null valor) "")
        ((stringp valor) valor)
        ((integerp valor) (format nil "~d" valor))
        ((realp valor) (format nil "~a" valor))
        ((eq valor t) "si")
        (t (princ-to-string valor))))

(defun iguales-p (a b)
  (cond ((and (realp a) (realp b)) (= a b))
        ((and (vacio-p a) (vacio-p b)) t)
        ((or (vacio-p a) (vacio-p b)) nil)
        ((and (stringp a) (stringp b)) (string= a b))
        (t (equal a b))))
```

Las cuatro reglas son: el vacío vale cero en aritmética; el texto que parece número se lee como

número; el vacío se escribe como cadena vacía; dos vacíos son iguales entre sí y un vacío no es igual a nada más.

Que estén en un solo sitio tiene una consecuencia directa sobre la extensión. Una arquitectura que emite código ejecutable tiene que reproducir esas cuatro reglas en su destino, y sabe exactamente cuáles son porque están juntas y documentadas. La arquitectura web lo hace con cuatro funciones de apoyo en la biblioteca que inyecta en la página —VACIO, N, T e IG, en `web/recursos.lisp`— que son la contraparte una a una de las cuatro de arriba. Por eso `emitir-expresion` para la web no traduce `< a <` sino a `(N(a) < N(b))`: sin esa coerción explícita, "10" sería menor que "9" en JavaScript y la página discreparía del evaluador. Si la coerción estuviera esparcida, cada regla sería un descubrimiento arqueológico para quien añada un destino, y la afirmación «toda arquitectura coincide con el evaluador» sería improbable en la práctica aunque fuese cierta en el papel.

6.4. El protocolo

El sistema `situacion/protocolo` es el contrato de extensión: lo que recibe quien quiera incorporar una arquitectura de salida nueva. Depende del núcleo y de nada más. En particular **no** depende del lenguaje, porque el contrato se define contra el dominio y no contra la sintaxis —ni, sobre todo, contra su primer implementador.

6.4.1. Las operaciones

`protocolo/genericas.lisp` declara las operaciones del protocolo. Todas con `definir-operacion`, ninguna con `defgeneric` a secas. Se copian con sus cadenas de documentación, que son el contrato propiamente dicho:

```
(definir-operacion capacidades (arquitectura)
  (:documentation
    "Las capacidades que ARQUITECTURA cumple de forma nativa.

    Normalmente no hace falta especializarla: el metodo por defecto las
    deduce de las clases de las que hereda."))

(definir-operacion nivel-de-recalculo (arquitectura)
  (:documentation
    "Cuando recalcula ARQUITECTURA los valores derivados.

    Uno de :VIVA, :BAJO-DEMANDA o :CONGELADA.

    El lenguaje describe siempre derivaciones vivas; el nivel que una
    arquitectura puede ofrecer es cosa suya."))

(definir-operacion planificar (arquitectura situacion)
  (:documentation
    "Construye el plan de ARQUITECTURA para SITUACION."
```

Devuelve (VALUES PLAN INFORME).

El plan es un objeto cuya forma pertenece por completo a la arquitectura: el de Excel decide que hoja, que columna y que fila ocupa cada cosa, cuantas filas de reserva hacen falta y que hojas auxiliares hay que inventarse; el de la web decide que componentes hay y como se llaman sus variables. El nucleo no mira dentro."))

```
(definir-operacion planificar-coleccion (arquitectura coleccion plan)
  (:documentation
    "Planifica una coleccion dentro de PLAN."))
```

```
(definir-operacion planificar-campo (arquitectura campo coleccion plan)
  (:documentation
    "Planifica un campo dentro de PLAN.
```

Es el sitio natural de las reglas transversales: que en cualquier arquitectura con entrada lo derivado quede de solo lectura se escribe una vez, como metodo auxiliar sobre CON-ENTRADA."))

```
(definir-operacion planificar-vista (arquitectura vista plan)
  (:documentation
    "Planifica una vista dentro de PLAN: secciones, agrupacion y navegacion."))
```

```
(definir-operacion materializar (arquitectura situacion datos destino)
  (:documentation
    "Planifica y emite de una vez. Es el punto de entrada de alto nivel.

    Devuelve (VALUES DESTINO INFORME)."))
```

```
(definir-operacion emitir (arquitectura plan destino)
  (:documentation
    "Escribe el artefacto. DESTINO es un flujo o una ruta."))
```

```
(definir-operacion emitir-expresion (arquitectura expresion ambito plan)
  (:documentation
    "Traduce un nodo de expresion a lo que ARQUITECTURA use para expresar calculo.

    EXPRESION  el nodo a traducir.
    AMBITO     que variables de fila hay ligadas y sobre que coleccion
               recorre cada una. No contiene ninguna direccion.
    PLAN       el plan de esta arquitectura, donde vive el
               direccionamiento que ella misma decidio en la fase 5."))
```

```
(definir-operacion emitir-marca (arquitectura marca ambito plan)
  (:documentation
    "Materializa una marca.
```

```

    La marca declara un significado y una severidad, nunca un color. Como se
    ve es cosa de la arquitectura."))

(definir-operacion emitir-entrada (arquitectura campo ambito plan)
  (:documentation
    "Materializa un campo de entrada: donde se escribe, con que dominio de
    valores, y si violarlo avisa o impide."))

(definir-operacion emitir-vista (arquitectura vista plan)
  (:documentation
    "Materializa una vista: sus secciones y su navegacion."))

(definir-operacion resolver-carencia (arquitectura capacidad nodo)
  (:documentation
    "Que hace ARQUITECTURA cuando le piden CAPACIDAD para NODO y no la
    cumple. Devuelve (VALUES RESPUESTA NOTA)."))

```

Son **trece** operaciones registradas, no doce ni once. El documento de diseño previo hablaba de once funciones genéricas; el código declara trece con `definir-operacion`, y el registro que lleva la propia macro es lo que la prueba del invariante I2 recorre.

Hay que decir además, porque el capítulo no puede callarlo: **cuatro de las trece no están especializadas por ninguna arquitectura**. `planificar-vista`, `emitir-marca`, `emitir-entrada` y `emitir-vista` están declaradas, documentadas y registradas, y ninguna de las tres arquitecturas define un método para ellas. Lo que esas cuatro nombran sí se materializa —las marcas se convierten en formato condicional en la hoja de cálculo y en clases de CSS en la página, los campos de entrada en validación de datos y en controles `<input>` y `<select>`, y las vistas en pestañas y en secciones—, pero lo hace código interno de cada backend, no un método colgado de la operación del protocolo. Son, hoy, contrato declarado sin uso. Se mantienen porque fijan la forma que tendrá ese contrato y porque su sola declaración ya está sujeta a C2, pero presentarlas como puntos de extensión ejercitados sería falso.

6.4.2. La macro que hace cumplir el criterio

`protocolo/operacion.lisp` tiene una sola responsabilidad: que no se pueda declarar una operación del protocolo que no reciba la arquitectura. Se copia literal:

```

(defun parametro-de-arquitectura-p (lambda-lista)
  "Cierta si LAMBDA-LISTA empieza por un parametro requerido llamado
  ARQUITECTURA.

  Se compara por nombre y no por identidad de simbolo a proposito: asi la
  comprobacion sigue valiendo si algun dia el protocolo se declara desde
  otro paquete."
  (let ((primero (first lambda-lista)))
    (and (symbolp primero)
         (not (null primero))
         (string= (symbol-name primero) (symbol-name '#:arquitectura)))))

```

```

(defmacro definir-operacion (nombre lambda-lista &body opciones)
  "Declara una operacion del protocolo.

  Identica a DEFGENERIC salvo por dos cosas:

  1. Exige que el primer parametro sea ARQUITECTURA. Si no lo es, senala
     un error en tiempo de expansion y el sistema no compila. Es el
     invariante I2, hecho cumplir por construccion en vez de vigilado.

  2. Registra la operacion, para que las pruebas puedan recorrer el
     protocolo entero y comprobar que no se ha colado ninguna por otra
     via."
  (unless (parametro-de-arquitectura-p lambda-lista)
    (error "La operacion ~s del protocolo no recibe la arquitectura.~@
           Su lambda-lista es ~s y tiene que empezar por ARQUITECTURA.~@
           ~@
           Una operacion que no despacha sobre la arquitectura no tiene~@
           donde enganchar una segunda: es el fallo que dejo sin cerrar la~@
           tesis de 2026 y que este trabajo se propone corregir."
           nombre lambda-lista))
    `(progn
      (eval-when (:compile-toplevel :load-toplevel :execute)
        (registrar-operacion ',nombre ',lambda-lista))
      (defgeneric ,nombre ,lambda-lista ,@opciones)))

```

El rechazo ocurre **en tiempo de expansión**, no en tiempo de ejecución. Es la diferencia entre una comprobación y una regla. Una comprobación en tiempo de ejecución solo protege a quien ejecute el caso; un **error** dentro del cuerpo de la macro impide que el archivo se compile, y por tanto que el sistema exista. Una operación mal formada no es un defecto que se detecte más tarde: es un programa que no se puede construir. Ésa es la forma más fuerte de garantía que ofrece un DSL interno en Common Lisp (Graham 1993; *American National Standard for Information Systems. Programming Language Common Lisp* 1994).

La macro registra además cada operación con su lista de parámetros, y ese registro es lo que permite a la prueba I2 recorrer el protocolo entero y comprobar que ninguna genérica exportada se coló sin pasar por aquí. Se guarda la lista de parámetros y no solo el nombre para no depender del protocolo de metaobjetos (Kiczales, Rivières, y Bobrow 1991), que no forma parte del estándar (*American National Standard for Information Systems. Programming Language Common Lisp* 1994).

6.4.3. La tabla de contraste con 2026

El punto de extensión de (Morales Lazo 2026) para el cálculo es:

```

(defgeneric compile-excel-formula
  (nodo col-map data-names fila primera-fila ultima-fila))

```

El de este trabajo es:

```
(definir-operacion emitir-expresion (arquitectura expresion ambito plan))
```

Parámetro a parámetro:

compile-excel-formula (Morales Lazo 2026)	emitir-expresion	Qué cambia
<i>(no recibe el destino)</i>	arquitectura, primer parámetro	Hay dónde enganchar una segunda arquitectura
nodo	expresion	Lo mismo, sin mencionar destino en el nombre
col-map: símbolo → letra de columna	<i>(no está en la firma)</i>	Vive dentro del plan de la hoja de cálculo
fila: número de fila	<i>(no está en la firma)</i>	Variable especial *fila* , interna de situacion.excel
primera-fila	<i>(no está en la firma)</i>	fila-primera de la hoja, dentro del plan
ultima-fila	<i>(no está en la firma)</i>	fila-ultima , calculada del plan
data-names, sin documentar	—	—
—	ambito	Qué variables de fila hay ligadas y sobre qué colección
—	plan	Opaco: lo fabrica, llena y lee la arquitectura
El nombre menciona el destino	El nombre no menciona ningún destino	—
Dos genéricas porque en Excel una produce texto y otra JSON	Las operaciones se separan por fase, no por destino	La división vale para cualquier destino

El último renglón merece detenerse. En 2026, que hubiera dos funciones genéricas distintas se justificaba porque *en Excel* una expresión produce texto que se incrusta y una estructura produce un objeto JSON. Eso es una propiedad del backend colándose en el diseño del contrato. Aquí la división es temporal —primero se planifica, después se emite— y no presupone nada del destino.

6.4.4. Ámbito frente a plan

Donde 2026 pasaba cuatro parámetros de rejilla, aquí van dos objetos con papeles distintos, y la distinción es el eje del diseño.

ambito es del núcleo y es del dominio. Vive en `nucleo/ambito.lisp`. Dice qué variables de fila hay ligadas y sobre qué colección recorre cada una. Nada más. Es la misma estructura para todas las arquitecturas y no contiene ninguna dirección. Sustituye a **fila**, **primera-**

`fila` y `ultima-fila`: aquellos tres decían *dónde* estaba la fila; éste dice *de qué colección* es. La diferencia es que lo segundo significa algo en cualquier destino.

plan es de la arquitectura y es opaco. Lo fabrica, lo llena y lo lee ella; el núcleo se limita a transportarlo de una operación a otra sin abrirlo nunca. El plan de la hoja de cálculo —`plan-de-excel`, en `excel/plan.lisp`— tiene hojas, columnas, fila de encabezado, fila primera, capacidad reservada y rangos nombrados. El de la web —`plan-de-web`, en `web/emision.lisp`— tiene la situación y el entorno, y su propia documentación dice que está «casi vacío: una página no tiene direcciones que repartir, que es justo por lo que esta arquitectura sirve de control».

De ahí sale la frase que resume el trabajo, escrita en la cabecera de `excel/plan.lisp`: **el col-map no desapareció, bajó de sitio.** Sigue existiendo —es el *slot* `columns` de la clase `hoja`, una lista asociativa de nombre de campo a letra— pero dentro del plan de una arquitectura concreta, que es donde tiene sentido. El núcleo no lo ve, el protocolo no lo nombra y ninguna otra arquitectura sabe que existe. Lo que desapareció no es el direccionamiento, imprescindible para escribir una hoja de cálculo: lo que desapareció es que estuviera en la firma del punto de extensión.

6.5. La jerarquía de capacidades

`protocolo/capacidades.lisp` define una raíz, `arquitectura`, y **doce** capacidades componibles. Una arquitectura concreta no hereda de `arquitectura` directamente: hereda de las capacidades que cumple, y cada capacidad ya hereda de la raíz.

Las doce son: `con-rejilla`, `con-derivacion-viva`, `con-derivacion-por-consulta`, `con-entrada`, `con-dominio-de-entrada`, `con-orden-declarado`, `con-relacion-uno-a-muchos`, `con-conteo-de-distintos`, `con-crecimiento`, `con-marcado-visual`, `con-marcado-textual` y `con-navegacion`. Cada una se declara con la macro `definir-capacidad`, que define la clase y la registra, y cada una lleva su documentación diciendo qué arquitecturas suelen tenerla y de qué construcción del corpus sale. El documento de diseño previo listaba once; el código tiene doce, porque `con-crecimiento` —que es el mejor caso de estudio del mecanismo entero— entró después.

La idea de componer destinos por herencia múltiple de propiedades, en lugar de enumerar destinos, no es original de este trabajo. (Arcia Corcho 2017) la atribuye a ADOL* y a GEMURAL, y la explica con el ejemplo clásico de los lenguajes de salida. Hay que ser exacto con la atribución, porque de la exactitud depende que la contribución propia se vea: **allí la idea se describe, no se implementa.** Ninguno de los tres trabajos previos de la línea llegó a construirla. En (Morales Lazo 2026), todas las clases del árbol declaran superclases vacías y la clase que identificaría el destino nunca se define. Lo que este trabajo aporta no es la idea sino su realización, y la evidencia de que sirve para algo.

Sirve para tres cosas concretas, y ninguna es taxonomía decorativa.

Primera: compartir métodos sin que dos arquitecturas se vean. Un método puede especializarse en la capacidad, no en la arquitectura. Es lo que permite que la hoja de cálculo

y la página compartan comportamiento de marcado visual sin conocerse entre sí, que es lo que C4 exige y lo que una tabla de banderas no daría.

Segunda: reglas transversales con métodos auxiliares. En `excel/plan.lisp` hay un método `:after` sobre `planificar-campo` especializado no en `excel` sino en `protocolo:con-entrada`:

```
;;; Regla transversal, escrita una vez para toda arquitectura con entrada:
;;; lo derivado queda de solo lectura.
(defmethod protocolo:planificar-campo :after ((a protocolo:con-entrada)
                                             campo coleccion plan)
  ...)
```

Una regla del dominio —en cualquier arquitectura donde el usuario pueda escribir, lo derivado es de solo lectura— vive en un sitio y vale para todas las arquitecturas presentes y futuras. Es la idea de los métodos auxiliares y de `recognize-pattern` de (Arcia Corcho 2017) aplicada aquí, y es un uso del despacho de CLOS (Keene 1989) que (Morales Lazo 2026) no llega a hacer: en aquel trabajo hay un solo `defmethod` concreto en todo el sistema.

Tercera: decidir qué hay que degradar. Comparar lo que la situación pide con lo que la arquitectura declara es una operación sobre clases. `cumple-p` es literalmente (`typep arquitectura (find-class capacidad)`). La operación `capacidades` no hace falta especializarla: el método por defecto deduce las capacidades recorriendo el registro y preguntando `typep`.

6.5.1. Qué cumple cada arquitectura

Capacidad	texto	excel	web
con-rejilla	no	sí	no
con-derivacion-viva	no	sí	sí
con-derivacion-por-consulta	no	no	no
con-entrada	no	sí	sí
con-dominio-de-entrada	no	sí	sí
con-orden-declarado	no	sí	sí
con-relacion-uno-a-muchos	no	no	sí
con-conteo-de-distintos	no	no	sí
con-crecimiento	no	no	sí
con-marcado-visual	no	sí	sí
con-marcado-textual	sí	sí	sí
con-navegacion	no	sí	sí
Total	1	8	10

La tabla se lee de las tres declaraciones de clase, en `texto/arquitectura.lisp`, `excel/arquitectura.lisp` y `web/arquitectura.lisp`, y es lo que `capacidades` devuelve en tiempo de ejecución.

Dos lecturas de esta tabla justifican que la jerarquía exista.

La primera: **no hay una arquitectura que pueda más y otras que puedan menos**. La hoja de cálculo tiene rejilla y la página no; la página tiene crecimiento, relación uno a muchos y conteo de distintos, y la hoja de cálculo no. La que a primera vista parece la más capaz tiene que emular tres cosas que en una página son gratis. Si las arquitecturas se ordenaran por potencia, la jerarquía sobraría y bastaría un nivel de degradación. No se ordenan.

La segunda: **texto plano cumple una sola capacidad, y eso es deliberado**. Se eligió precisamente porque no puede casi nada. Obliga a que la maquinaria de degradación exista desde el primer día en vez de ser un capítulo que se escribe al final.

6.6. El mecanismo de degradación

Ninguno de los tres trabajos previos se plantea qué pasa cuando una arquitectura no puede expresar algo. (Morales Lazo 2026) lo reconoce en una nota —un backend cuyo formato de salida no admita cierto posicionamiento «podría simplemente prescindir de esta funcionalidad»— y ahí queda. Aquí es un paso de primera clase, con cuatro respuestas posibles y un informe que las registra todas.

6.6.1. Las cuatro respuestas

Respuesta	Qué significa	Ejemplo real, copiado del sistema
cumple	Lo hace de forma nativa	web ante con-crecimiento : «asignaturas sigue creciendo despues de generar»
emula	No lo tiene, pero puede fabricarlo. Declara a qué coste	excel ante con-crecimiento : «20 filas de reserva y un rango que se dimensiona con lo escrito»
degrada	Hay una versión más pobre que sigue sirviendo para lo mismo	texto ante con-marcado-visual : «el estado se escribe en una columna, sin color»
rechaza	No hay versión honesta y fingirla sería mentir	texto ante cualquier capacidad no prevista: «texto plano no tiene nada parecido a ~a»

Las emulaciones declaran su límite, y ése es el renglón que separa un prototipo de algo usable. El de la hoja de cálculo ante el crecimiento dice, literal:

Limite comprobado en el corpus: no admite huecos en medio del listado, porque el conteo de celdas no vacias no distingue una fila en blanco de una fila ausente y la ultima entrada se queda fuera del rango.

Esa clase de detalle vive hoy, en el repositorio real de la facultad, enterrada en el comentario de un módulo de Python. Aquí sale impresa cada vez que se compila.

6.6.2. La frontera entre degradar y rechazar

Es la decisión que hay que poder defender, y está escrita en el propio protocolo:

Se degrada cuando el resultado sigue cumpliendo el propósito por otra vía; se rechaza cuando el resultado *parecería* correcto y no lo sería. Un documento que aparenta admitir entrada y no la admite es peor que un error.

El criterio se ve funcionando en la arquitectura de texto. Ante **con-entrada** degrada: escribe el valor actual y el documento no es editable. Nadie va a creer que un archivo `.txt` es un formulario, así que el resultado sigue sirviendo para lo que servía. Ante **con-dominio-de-entrada** degrada también, pero hace algo más que renunciar: escribe los valores admisibles al pie de la tabla, y en el artefacto de defensas aparece como **Estudiante admite: Juan Perez, Laura Fernandez, Mario Gomez, Ana Valdes**. La información del dominio no se pierde; cambia de forma. Eso es degradar bien.

Hay una regla de cierre que importa: **el método por defecto de resolver-carencia, especializado en la raíz arquitectura, rechaza**. Si una arquitectura no declara qué hacer con una carencia, no se asume que algo saldrá. Callar no puede significar «ya se verá».

6.6.3. El sistema de condiciones, y dónde vive la política

Aquí hay que declarar otra diferencia con el documento de diseño previo, y es de fondo. Aquel proponía implementar la degradación con **restart-case** y tres reinicios —**emular**, **degradar**, **omitir**— que un manejador exterior elegiría. **El código no hace eso**. Lo que hace **requerir**, en `protocolo/carencia.lisp`, es:

1. Si la arquitectura cumple la capacidad, anota **:cumple** y termina.
2. Si no, **señala carencia-de-capacidad**, que es una condición sin herencia de **error**: es informativa, y si nadie la maneja el programa sigue. Quien quiera observar la compilación instala un manejador y se entera de todo lo que se emula o se degrada sin cambiar nada.
3. Llama a **resolver-carencia**, que despacha sobre la arquitectura, y obtiene (**values respuesta nota**).
4. Consulta la política. Si la respuesta es **:rechaza**, o la política es **:estricta**, o la política es **:solo-nativo** y la respuesta fue **:emula** o **:degrada**, señala el error **capacidad-no-disponible**. Si no, anota la respuesta con su nota en el informe.

El resultado observable es el que el diseño buscaba —la política vive fuera del backend— pero el mecanismo es una función genérica, no un conjunto de reinicios. Tiene una ventaja que el diseño no había previsto: qué hace cada arquitectura ante cada carencia queda escrito de forma declarativa y localizada, en un solo método por arquitectura. Y un coste: el manejador exterior no puede decidir *cómo* se resuelve una carencia concreta, solo si acepta la resolución que el backend propuso.

La política es una variable especial:

```
(defvar *politica* :permisiva
  "Que se acepta cuando una arquitectura no cumple una capacidad.

  :PERMISIVA   se aceptan emulaciones y degradaciones. Es la de por
                defecto y la que se usa para producir documentos.
  :ESTRICTA    cualquier carencia es un error. Sirve para saber
                exactamente que descripciones caen enteras dentro de una
                arquitectura.
  :SOLO-NATIVO ni siquiera se permite emular. Sirve para medir cuanto de
                una descripcion es nativo y cuanto es andamiaje.")
```

Que la política esté fuera del backend convierte el mecanismo en algo medible. Compilar la misma descripción en los tres modos contra las tres arquitecturas da tres respuestas distintas sobre el mismo código, y eso es una tabla de resultados experimentales que sale del diseño sin escribir nada más.

6.6.4. El informe de conformidad

Cada llamada a `requerir` deja una línea. Al final, `escribir-informe` las imprime con su resumen. Esto es la salida literal del sistema para la descripción «Plan de asignaturas del grupo D111», compilada contra la hoja de cálculo:

```
Informe de conformidad      Arquitectura: excel
-----
emula    con-crecimiento    20 filas de reserva y un rango que
se dimensiona con lo escrito. Limite comprobado en el corpus: no admite
huecos en medio del listado, porque el conteo de celdas no vacias no
distingue una fila en blanco de una fila ausente y la ultima entrada se
queda fuera del rango.
cumple   con-entrada        asignadas lo escribe el usuario
cumple   con-derivacion-viva faltan se recalcula al editar
cumple   con-marcado-visual  la marca insuficiente senala un estado
cumple   con-marcado-textual la marca insuficiente explica por que
                               en palabras
-----
4 nativas, 1 emuladas, 0 degradadas, 0 rechazadas
```

Y la misma descripción, sin tocar una línea, compilada contra texto plano:

```
Informe de conformidad      Arquitectura: texto
-----
degrada  con-crecimiento    se escribe lo que hay; para anadir
                               filas hay que regenerar
degrada  con-entrada        se escribe el valor actual; el
                               documento no es editable
degrada  con-derivacion-viva los valores se calculan al generar y
                               no se recalculan
degrada  con-marcado-visual  el estado se escribe en una columna,
```

cumple	con-marcado-textual	sin color la marca insuficiente explica por que en palabras
--------	---------------------	---

1 nativas, 0 emuladas, 4 degradadas, 0 rechazadas

Lo que hay que leer en el par no es la cuenta. Es que la lista de la izquierda es la misma en los dos —son las capacidades que **la descripción** pide, no las que el backend sabe hacer— y que la de la derecha cambia por completo. Ese barrido lo produce `protocolo:requerimientos`, que vive en el protocolo y no en cada backend, por una razón precisa: si cada arquitectura decidiera por su cuenta qué le están pidiendo, dos darían informes distintos para la misma descripción y la comparación no significaría nada.

El informe es la evidencia que ninguno de los tres trabajos previos tiene. Convierte «el mecanismo de extensión funciona» en algo que se enseña. Los seis informes del sistema se analizan en el capítulo 7.

6.7. Estructura de sistemas

`situacion.asd` declara once sistemas. El grafo de dependencias no es administrativo: es la prueba de dos criterios, y por eso el archivo lleva su justificación en la cabecera.

```
situacion/nucleo      depende de: (nada)
    paquete, nodo, modelo, expresion, ambito, evaluador

situacion/lenguaje    depende de: nucleo
    paquete, expresiones, macros

situacion/protocolo   depende de: nucleo
    paquete, operacion, capacidades, genericas, informe,
    carencia, requerimientos

situacion/analisis    depende de: nucleo
    paquete, comprobacion

situacion/texto       depende de: protocolo
    arquitectura, emision

situacion/excel       depende de: protocolo
    arquitectura, plan, formula, plano, emision

situacion/web         depende de: protocolo
    arquitectura, recursos, expresiones, emision

situacion/corpus      depende de: lenguaje
    paquete, plan-del-grupo, defensas-de-tesis

situacion/demostracion depende de: corpus, analisis, texto, excel, web
```

```

situacion/pruebas      depende de: todo lo anterior
    paquete, marco, invariantes, nucleo, conformidad, ejecutar

materializador/        Python. Fuera del sistema Lisp.
    materializar.py, verificar.py, verificar-web.js

```

Las siete reglas que el grafo hace cumplir, con el criterio que cada una garantiza:

Regla	Criterio que garantiza
nucleo no depende de nada	C1: la representación intermedia no puede contaminarse, porque no hay de dónde
protocolo depende solo de nucleo	C2: el contrato se define contra el dominio
protocolo no depende de lenguaje	C3: un backend no puede ver las macros
protocolo no depende de ninguna arquitectura	El contrato no se define contra su primer implementador, que es el error de 2026
Toda arquitectura depende solo de protocolo	C3
Ninguna arquitectura depende de otra	C4
Nadie fuera de situacion/excel puede nombrar su plan	C5

La cuarta merece subrayarse, porque es la que ordena el trabajo entero. En 2026 el punto de extensión se escribió mirando Excel, que era lo único que había, y salió con la forma de Excel. Aquí el protocolo se escribió antes que el primer backend y se falsó de inmediato con dos destinos que no comparten nada: uno evalúa fórmulas A1 dentro de una rejilla, el otro ejecuta funciones sobre arreglos en un navegador.

Tres decisiones del grafo conviene justificarlas aparte. **analisis** no depende del protocolo: para cuando llega un backend, la descripción ya está comprobada. **corpus** depende del lenguaje y de nada más: las descripciones de los libros de la facultad no saben que existe ningún destino. Y **pruebas** depende de todo y nada depende de ella: es el único sistema autorizado a mirar dentro de los demás.

La estructura real difiere del documento de diseño previo en la distribución de archivos —aquel preveía diez archivos para el núcleo y diez para la arquitectura de la hoja de cálculo, además de un sistema separado de conformidad y una arquitectura SQL—. La de arriba es la que existe: la conformidad vive dentro de **situacion/pruebas**, la arquitectura SQL no se implementó, y la de texto plano, que el documento previo no listaba, sí existe y es la que obligó a que la degradación funcionara desde el primer día.

6.8. Registro de decisiones

Nueve decisiones de diseño, cada una con la alternativa que se descartó y con su coste. Se declaran los costes porque una decisión sin coste declarado no es una decisión: es una

preferencia.

D1 — El protocolo transporta un plan opaco, no parámetros de direccionamiento.

La alternativa era pasar explícitamente lo que necesita cada backend, como hacía ([Morales Lazo 2026](#)). Se descartó porque cualquier lista explícita de parámetros se fija en la forma del primer backend que se implemente, y ése es literalmente el defecto que este trabajo corrige. *Coste*: el núcleo no puede razonar sobre el direccionamiento, y por tanto no puede optimizarlo ni diagnosticarlo. Se asume: no es su trabajo.

D2 — Planificar y emitir son dos fases separadas. La alternativa era una sola pasada que decidiera y escribiera a la vez. Se descartó por dos motivos independientes: es lo que obliga a que la función de emisión reciba números de fila, porque en el momento de escribir todavía no se sabe dónde va todo; y es lo que deja la degradación sin un sitio donde ocurrir. *Coste*: dos recorridos del árbol en lugar de uno. A la escala del dominio —colecciones de decenas de filas— es irrelevante.

D3 — Las capacidades son clases y se componen por herencia múltiple, no banderas en una tabla. La alternativa era que cada arquitectura llevara una lista de símbolos con lo que sabe hacer. Se descartó porque con clases un método puede especializarse en la capacidad, y dos arquitecturas comparten implementación sin conocerse, que es lo que C4 exige; con banderas habría que consultarlas a mano en cada punto del código. *Coste*: la jerarquía hay que acertarla razonablemente pronto. Añadir una capacidad transversal más tarde obliga a revisar las composiciones de todas las arquitecturas existentes, y eso ya ocurrió una vez con `con-crecimiento`.

D4 — La degradación se resuelve con una operación del protocolo y una política externa. La alternativa inicial, escrita en el documento de diseño, era usar `restart-case` con reinicios que un manejador exterior eligiera. La alternativa descartada de raíz era que `planificar` devolviera un valor especial, porque eso cablea la política dentro de cada backend. La implementación se quedó a medio camino de la propuesta inicial y conviene decirlo: qué hace una arquitectura ante una carencia lo decide ella, en un método de `resolver-carencia`; si esa resolución se acepta lo decide `*politica*`, que vive fuera. *Coste*: un manejador exterior no puede imponer una resolución distinta de la que el backend propuso, solo aceptarla o convertirla en error.

D5 — El .xlsx lo escribe un materializador en Python. La alternativa era un emisor OOXML propio en Lisp ([ECMA-376: Office Open XML File Formats 2016](#)). Se descartó tras comprobar que no existe biblioteca de Common Lisp capaz de escribir fórmulas, formato condicional, validación de datos, rangos nombrados y protección de celdas; escribir una son semanas de trabajo sin aporte científico. *Coste*: el sistema tiene dos lenguajes y hay que poder defender por qué esta frontera sí es legítima. La respuesta corta es que el materializador no recibe la situación, solo el plano de libro: no tiene acceso a nada del dominio ni aunque quisiera.

D6 — El lenguaje describe siempre derivaciones vivas; el nivel de recálculo es una capacidad de la arquitectura. La alternativa era decidir globalmente entre derivaciones vivas y congeladas. Se descartó porque cualquiera de las dos deja fuera destinos legítimos.

Como capacidad, un documento impreso deja de ser un caso que el diseño no contempla y pasa a ser una degradación declarada con su línea en el informe. *Coste*: quien escribe una descripción tiene que pensar siempre en términos de recálculo, aunque el destino que le interese no recalculé.

D7 — La vecindad exige orden declarado. La alternativa era permitir *anterior* sin más, como hace el *previous-of* de (Morales Lazo 2026). Se descartó porque sin orden declarado «anterior» solo puede significar «la fila de arriba en la rejilla», que es el modelo de la hoja de cálculo colándose en el lenguaje y no tiene traducción a ningún otro destino. *Coste*: hay descripciones que hoy son implícitas y hay que hacer explícitas. Es un coste bien pagado: el error se detecta en la fase 4, con un mensaje que explica por qué.

D8 — La marca declara significado y severidad; el color es opción de la arquitectura. La alternativa era que la regla de marcado llevara color, como propone la definición correspondiente de (Morales Lazo 2026). Se descartó porque el color no sobrevive al cambio de destino y el significado sí, y porque en una impresión en blanco y negro la marca tiene que ser texto o no se ve. El corpus de la facultad ya lo hacía a mano antes de que existiera este lenguaje. *Coste*: hace falta una paleta por arquitectura: `+paleta+` en `excel/emision.lisp` y `+paleta-web+` en `web/emision.lisp`, cada una un artefacto más que mantener.

D9 — El protocolo se escribe antes que el primer backend, y se falsa con dos. La alternativa era extraer el protocolo del backend de la hoja de cálculo una vez escrito. Se descartó porque extraer una abstracción de un solo caso produce la abstracción de ese caso, que es de nuevo el error de 2026. *Coste*: el primer protocolo iba a estar mal y había que rehacerlo cuando el segundo destino lo falseara. Se asumió y se planificó: por eso la arquitectura de texto plano existe desde el primer día y se eligió precisamente por no poder casi nada.

Capítulo 7

Implementación

Este capítulo describe cómo está construido el sistema. No qué debería hacer: qué hace. Todo lo que aparece aquí se leyó del código fuente y, donde hay números, se midieron ejecutando el sistema el 21 de septiembre de 2026. El código vive en `situacion/`, dentro del repositorio `Horario`, y se ejecuta entero con un solo comando: `./demostrar.sh`.

El capítulo anterior explicó el diseño. Este muestra la realización, con el código en la mano. Cuando el documento de diseño y el código no coinciden, manda el código y se dice explícitamente.

7.1. Panorama

El sistema está partido en diez sistemas ASDF de Common Lisp, más un materializador escrito en Python. Los tamaños, contados con `wc -l` sobre los archivos fuente:

Sistema	Archivos	Líneas	Qué hace
<code>situacion/nucleo</code>	6	918	El modelo semántico: situaciones, colecciones, campos, expresiones, ámbitos, marcas, vistas, y el evaluador de referencia. No sabe que existe ninguna arquitectura.
<code>situacion/lenguaje</code>	3	488	Las macros que construyen el árbol en tiempo de expansión.

Sistema	Archivos	Líneas	Qué hace
situacion/analisis	2	322	Comprobaciones estáticas, independientes del destino.
situacion/protocolo	7	741	El contrato de extensión: capacidades, operaciones, carencias, informe.
situacion/texto	2	209	Arquitectura de salida a texto plano.
situacion/excel	5	834	Arquitectura de salida a hoja de cálculo.
situacion/web	4	672	Arquitectura de salida a página con recálculo vivo.
situacion/corpus	3	241	Los dos libros de la facultad, descritos en el lenguaje.
situacion/pruebas	6	862	Invariantes, pruebas del núcleo y juego de conformidad.
situacion/demostracion		105	Genera los artefactos de las tres arquitecturas.
Total Lisp	39	5392	
materializador/ (Python)	2	387	Convierte el plano de libro en .xlsx y verifica el resultado.
materializador/ (JavaScript)	1	109	Verifica la página generada contra el evaluador de referencia.

Hay un hecho de esta tabla que importa más que los tamaños: **el núcleo, el protocolo, las tres arquitecturas y las pruebas no tienen ninguna dependencia**. El sistema `situacion/nucleo` se declara con `:depends-on ()` literalmente, y los scripts `ejecutar-pruebas.sh` y `generar-demostracion.sh` arrancan SBCL con `--no-userinit`, sin Quicklisp y sin cargar nada del entorno del usuario. Eso no es purismo. Quien reciba el protocolo para escribir su propia arquitectura tiene que poder cargarlo con un SBCL recién instalado; si para leer el contrato de extensión hiciera falta media docena de bibliotecas, la afirmación de

que el mecanismo es utilizable por un tercero se volvería difícil de sostener. El propio marco de pruebas (`pruebas/marco.lisp`, 100 líneas) se escribió a mano por esa razón, en vez de usar una biblioteca de pruebas existente.

Esa restricción tiene consecuencias visibles dentro del código. El protocolo deduce las capacidades de una arquitectura con `typep` y no recorriendo la lista de precedencia de clases, porque eso exigiría el MOP, que no es Common Lisp estándar (*American National Standard for Information Systems. Programming Language Common Lisp* 1994); está comentado en `protocolo/genericas.lisp`, en el método por defecto de `capacidades`. Y la arquitectura de hoja de cálculo trae su propio serializador de JSON de unas cuarenta líneas (`excel/plano.lisp`, `escribir-json`) por el mismo motivo.

La única excepción es el materializador de Excel, que usa `openpyxl`, una biblioteca de Python. La sección correspondiente explica por qué esa excepción existe y por qué está donde está.

7.2. La arquitectura de texto plano

7.2.1. Por qué existe

No existe para producir documentos. Existe para que haya **dos arquitecturas desde el primer commit**, que es la única salvaguarda real contra el error de (Morales Lazo 2026). Un protocolo diseñado mirando un solo destino sale con la forma de ese destino, se escriba cuando se escriba. La firma de `compile-excel-formula` lleva `col-map`, `fila`, `primera-fila` y `ultima-fila` porque, mientras solo hay un destino, esos parámetros parecen razonables. Solo un segundo destino los delata.

Y se elige texto plano **precisamente porque no puede casi nada**. No tiene rejilla, ni fórmulas, ni color, ni entrada. Esa elección es deliberada y se justifica en el encabezado de `texto/arquitectura.lisp`: obliga a que la maquinaria de degradación exista desde el principio, en vez de ser un capítulo que se escribe al final. Su primer informe de conformidad ya trae degradaciones de verdad, y por tanto el mecanismo de carencias tuvo que funcionar antes de que existiera nada más.

La clase declara una sola capacidad:

```
(defclass texto (protocolo:con-marcado-textual)
  ()
  (:documentation
   "Salida a texto plano: una tabla alineada por coleccion, con los valores
    ya calculados y el estado escrito en palabras."))
```

7.2.2. Su resolución de carencias, entera

Son veinticuatro líneas. Es todo lo que esta arquitectura necesita decir sobre las nueve capacidades que no cumple:

```
(defmethod protocolo:resolver-carencia ((a texto) capacidad nodo)
  (declare (ignore nodo))
  (case capacidad
    (protocolo:con-derivacion-viva
     (values :degrada "los valores se calculan al generar y no se recalculan"))
    (protocolo:con-entrada
     (values :degrada "se escribe el valor actual; el documento no es editable"))
    (protocolo:con-dominio-de-entrada
     (values :degrada "los valores admisibles se listan al pie de la tabla"))
    (protocolo:con-marcado-visual
     (values :degrada "el estado se escribe en una columna, sin color"))
    (protocolo:con-navegacion
     (values :emula "las vistas se apilan con un indice al principio"))
    (protocolo:con-crecimiento
     (values :degrada "se escribe lo que hay; para anadir filas hay que regenerar"))
    (protocolo:con-orden-declarado
     (values :cumple "las filas se escriben en el orden declarado"))
    (t (values :rechaza
              (format nil "texto plano no tiene nada parecido a ~(~a~)"
                       capacidad))))))
```

Conviene mirar tres cosas. La primera: casi todo se degrada y casi nada se rechaza. La frontera es la que declara el protocolo en `protocolo/carencia.lisp`: se degrada cuando el resultado sigue cumpliendo el propósito por otra vía, y se rechaza cuando el resultado *parecería* correcto y no lo sería.

La segunda: la última cláusula. Si llega una capacidad que esta arquitectura no contempla, se rechaza. El método por defecto del protocolo hace lo mismo para cualquier arquitectura que calle. **Callar no puede significar “algo saldrá”**, y eso está escrito como comportamiento, no como recomendación.

La tercera: `con-orden-declarado` devuelve `:cumple` desde `resolver-carencia`, aunque la clase no herede de esa capacidad. La respuesta es honesta —el emisor escribe las filas en el orden en que vienen— y el informe la registra como nativa, pero la clase y el método no dicen lo mismo. Es un detalle menor y real del código.

7.2.3. Cómo emite

Esta arquitectura no tiene derivación viva, así que su plan consiste en evaluar la situación entera con el evaluador de referencia del núcleo y quedarse con los valores ya calculados:

```
(defmethod protocolo:materializar ((a texto) situacion datos destino)
  (protocolo:con-informe (a)
    (dolist (r (protocolo:requerimientos situacion))
      (destructuring-bind (capacidad nodo detalle) r
        (protocolo:requerir a capacidad nodo :detalle detalle)))
    (let* ((entorno (nucleo:hacer-entorno situacion datos))
           (plan (make-instance 'plan-de-texto
```

```

:situacion situacion
:entorno entorno
:evaluado (nucleo:evaluar-situacion situacion datos))))
(protocolo:emitir a plan destino))))

```

Ese cuerpo es la demostración de que el evaluador de referencia hace falta. Sin él, esta arquitectura no podría existir sin reimplementar por su cuenta la semántica del lenguaje, que es exactamente lo que el diseño quiere impedir.

La degradación del marcado visual se ve en `escribir-coleccion`: donde una hoja de cálculo pondría un relleno, aquí va una palabra en una columna llamada `Estado`. La del dominio de entrada se ve en el mismo sitio: los valores admisibles se listan al pie de la tabla. El artefacto medido de `plan-del-grupo` escribe en esa columna `completa`, `Sobra 1 turno` y `Falta 1 turno`, y la fila AM —frecuencia 2, asignadas 3— sale marcada como excedida, que es justo el caso que la figura 6.2 de (Morales Lazo 2026) pinta mal.

Doscientas nueve líneas. Ese es el costo real de una arquitectura de salida completa contra este protocolo, y es el número que conviene tener a mano cuando se discute si el mecanismo de extensión sirve.

7.3. La arquitectura de hoja de cálculo

7.3.1. El plan y el direccionamiento

`excel/plan.lisp` es donde vive el direccionamiento, y el único sitio donde vive. El `col-map` de (Morales Lazo 2026) —el diccionario de símbolo de columna a letra de Excel— no ha desaparecido: está en este archivo. Lo que cambió es que ya no está en la firma del punto de extensión, sino dentro del plan de una arquitectura concreta. El núcleo no lo ve, el protocolo no lo nombra, y ninguna otra arquitectura sabe que existe.

```

(defclass hoja ()
  ((nombre :accessor nombre :initarg :nombre)
   (coleccion :accessor coleccion :initarg :coleccion)
   (columnas :accessor columnas :initarg :columnas
              :documentation "Lista asociativa de nombre de campo a letra.")
   (fila-encabezado :accessor fila-encabezado :initarg :fila-encabezado)
   (fila-primera :accessor fila-primera :initarg :fila-primera)
   (capacidad :accessor capacidad :initarg :capacidad)
   (rango-nombrado :accessor rango-nombrado :initarg :rango-nombrado)))

```

Las funciones de direccionamiento son cuatro y caben en veinte líneas: `letra-de-columna` convierte un índice en A, B, ..., Z, AA; `columna-de` busca la letra de un campo; `celda` compone una referencia con sus signos de dólar; `rango-de-columna` compone el rango absoluto de una columna completa. Ninguna sale del paquete `situacion.excel`. El invariante I4 comprueba que ningún otro paquete referencia un símbolo interno de esta arquitectura.

La planificación de una colección decide la geometría:

```
(defmethod protocolo:planificar-coleccion ((a excel) coleccion plan)
  (let* ((campos (nucleo:campos coleccion))
        (columnas (loop for campo in campos
                        for i from 0
                        collect (cons (nucleo:nombre campo) (letra-de-columna i))))
        (n-datos (max 1 (length (nucleo:datos coleccion))))
        (capacidad (if (eq (nucleo:crecimiento coleccion) :crece)
                        (+ n-datos (max (reserva-minima a) (floor n-datos 2)))
                        n-datos)))
    ...)))
```

El encabezado va en la fila 3 y los datos empiezan en la 4. La reserva mínima es 20 filas y sale del corpus: con pocos datos, reservar un porcentaje se queda en nada. Con las seis asignaturas del caso del plan del grupo, la capacidad medida es 26 y el rango de datos es A4:A29.

Aquí aparece también la primera regla transversal escrita contra una capacidad y no contra una arquitectura:

```
(defmethod protocolo:planificar-campo :after ((a protocolo:con-entrada)
                                             campo coleccion plan)

  (declare (ignore coleccion plan))
  (when (eq (nucleo:rol campo) :derivado)
    t))
```

Es un método auxiliar sobre `con-entrada`, no sobre `excel`. La idea —una regla, un sitio, todas las arquitecturas presentes y futuras— es la que ([Arcia Corcho 2017](#)) atribuye a ADOL*, y es lo que el sistema de objetos de Common Lisp permite expresar sin andamiaje ([Keene 1989](#); [Kiczales, Rivières, y Bobrow 1991](#)). El bloqueo real lo aplica la emisión, que es quien conoce las celdas; lo que importa es que la regla esté declarada en un solo lugar.

7.3.2. La emisión de expresiones

`excel/formula.lisp` es el equivalente de `compile-excel-formula`, con dos diferencias que son el argumento entero de la tesis. La primera: el método se especializa en `excel`, de modo que otra arquitectura escribe los suyos y este archivo no se toca. La segunda: las direcciones no llegan por parámetro, se piden al plan.

La fila que se está generando es estado interno del backend y viaja en una variable especial del paquete:

```
(defvar *fila* nil
  "Numero de fila que se esta generando. Interno de esta arquitectura.")
(defvar *hoja* nil
  "Hoja cuya fila se esta generando.")
```

Que no cruce el protocolo es justamente lo que se buscaba.

El archivo especializa once métodos de `emitir-expresion`. Los cuatro sencillos:

```

(defmethod protocolo:emitir-expresion ((a excel) (e nucleo:literal) ambito plan)
  (texto-de-literal (nucleo:valor e)))

(defmethod protocolo:emitir-expresion ((a excel) (e nucleo:ref-campo) ambito plan)
  ;; Columna fijada y fila relativa: así la misma expresion vale para todas
  ;; las filas del rango.
  (celda *hoja* (nucleo:campo e) *fila*))

(defmethod protocolo:emitir-expresion ((a excel) (e nucleo:condicional) ambito plan)
  (format nil "IF(~a,~a,~a)"
    (protocolo:emitir-expresion a (nucleo:prueba e) ambito plan)
    (protocolo:emitir-expresion a (nucleo:entonces e) ambito plan)
    (protocolo:emitir-expresion a (nucleo:si-no e) ambito plan)))

(defmethod protocolo:emitir-expresion ((a excel) (e nucleo:sin-escribir) ambito
↪ plan)
  (format nil "(~a=\"\"")
    (protocolo:emitir-expresion a (nucleo:argumento e) ambito plan)))

```

La aplicación resuelve aritmética, comparación y lógica con una tabla de infijos y tres casos nombrados. Que el núcleo unifique todos los operadores en un solo nodo `aplicacion` —en lugar de una clase por operador, como hace (Morales Lazo 2026)— tiene aquí su consecuencia práctica: un backend escribe un método y una tabla, no doce métodos.

```

(defparameter +infijos+
  '((+ . "+") (- . "-") (* . "*") (/ . "/")
    (= . "=") (/= . "<>") (< . "<") (<= . "<=") (> . ">") (>= . ">=")))

(defmethod protocolo:emitir-expresion ((a excel) (e nucleo:aplicacion) ambito plan)
  (let ((partes (mapcar (lambda (x) (protocolo:emitir-expresion a x ambito plan))
    (nucleo:argumentos e)))
    (op (nucleo:operador e)))
    (cond
      ((assoc op +infijos+) ...)
      ((string= (symbol-name op) "Y") (format nil "AND(~{~a~^,~})" partes))
      ((string= (symbol-name op) "O") (format nil "OR(~{~a~^,~})" partes))
      ((string= (symbol-name op) "NO") (format nil "NOT(~a)" (first partes)))
      (t (error "Operador sin traduccion a hoja de calculo: ~a" op)))))

```

Los agregados. Una hoja de cálculo no sabe recorrer con una condición arbitraria: sabe contar y sumar sobre un rango con un criterio (Sestoft 2014). El emisor reconoce las formas que el corpus usa de verdad. Una función auxiliar, `igualdad-simple`, detecta si la condición es una igualdad entre un campo de la candidata y una expresión de fuera; si lo es, la traducción es `COUNTIF` o `SUMIF`. Si no hay condición, es `COUNTA` sobre la primera columna. Y si la condición es más complicada, **falla con un mensaje claro en vez de emitir una fórmula plausible**:

```
(error "Esta hoja de calculo solo sabe agregar con un criterio de~@
igualdad sobre un campo. La condicion de ~(~a~) es mas~@
complicada." (nucleo:operacion e))
```

Esa decisión es deliberada y aparece comentada en el archivo. Una fórmula que parece correcta y no lo es, en un documento que el usuario va a crear, es peor que un error: es el modo de fallo que la literatura sobre errores en hojas de cálculo describe como el más caro de detectar (Panko 1998; Hermans, Pinzger, y Deursen 2012; Abraham y Erwig 2006).

El caso `:distintas` es distinto, porque la capacidad no existe en el destino. Se pide explícitamente para que el informe la registre, y después se emula:

```
(protocolo:requerir a 'protocolo:con-conteo-de-distintos e
:detalle "cuantos valores distintos toma ...")
(format nil "SUMPRODUCT((~a<>\"\\\")/COUNTIF(~a,~a&\"\\\"))" ...)
```

La proyección sobre una búsqueda. Aquí aparece una asimetría real entre las dos arquitecturas, y es una de las cosas más informativas del código. La hoja de cálculo **no especializa** `emitir-expresion` sobre el nodo `busqueda` ni sobre `vecino`: los atiende únicamente a través de `proyeccion`. La razón es que en una hoja de cálculo no hay forma de representar una expresión cuyo valor sea una fila; `VLOOKUP` devuelve directamente un valor de celda. En la página, en cambio, una fila es un objeto, y `busqueda` sí es una expresión de primera clase. El mismo álgebra de expresión, dos granularidades de traducción.

```
(defun emitir-busqueda (a proyeccion busqueda ambito plan)
...
(when (< pos-valor pos-clave)
  (error "BUSCARV solo mira a la derecha de la clave, y ~(~a~) esta a~@
la izquierda de ~(~a~) en ~(~a~)." ...))
(format nil "IFERROR(VLOOKUP(~a,~a,~d,FALSE),~a)" ...))
```

La guarda `IFERROR` no es un adorno. Las once búsquedas del corpus van envueltas en una, porque el usuario escribe la clave antes de que exista la fila relacionada; el valor por defecto es la mitad del concepto, y por eso el lenguaje lo declara con `:si-no`.

El cuantificador existencial. Una hoja de cálculo no tiene cuantificadores. Se emula con la suma de productos de vectores booleanos, que es el idioma establecido y el que usa el corpus. `factores-de-existencia` descompone la condición: una conjunción se convierte en varios factores, una igualdad entre un campo de la candidata y una expresión de fuera se convierte en un factor de comparación contra el rango completo, y `comparten` se despliega en una suma de todas las combinaciones de columnas. La exclusión de la propia fila se traduce comparando `ROW`.

7.3.3. Fórmulas realmente emitidas

Estas son literales, tomadas de `salida/plan-del-grupo-plano.json` y `salida/defensas-de-tesis-plano.json` el 21 de septiembre de 2026:

```
E4 =IF($A4="", "", ($C4-$D4))
E4 =IF($A4="", "", IFERROR(VLOOKUP($D4,Tesis!$A$4:$E$7,2,FALSE), ""))
```

Y las condiciones de formato condicional:

```
excedida      ($D4>$C4)
completa      AND(($D4=$C4),NOT(($D4="")))
```

La fórmula de detección de colisión, que es el caso más grande, aparece en la comparación lado a lado más adelante.

7.3.4. Cómo se emula el crecimiento

El crecimiento es el mejor caso de estudio del mecanismo entero, y merece la pena seguirlo completo porque toca cuatro sitios del código.

Primero, la declaración de la carencia, en `excel/arquitectura.lisp`:

```
(protocolo:con-crecimiento
  (values :emula
    (format nil
      "~d filas de reserva y un rango que se dimensiona con lo~@
      escrito. Limite comprobado en el corpus: no admite huecos~@
      en medio del listado, porque el conteo de celdas no vacias~@
      no distingue una fila en blanco de una fila ausente y la~@
      ultima entrada se queda fuera del rango."
      (reserva-minima a))))
```

La nota no es decorativa: el juego de conformidad exige que toda respuesta `:emula` o `:degrada` traiga una nota de más de veinte caracteres (`pruebas/conformidad.lisp`, prueba `conformidad-cada-respuesta-lleva-nota`). Una emulación sin su límite escrito es una promesa.

Segundo, la reserva de filas en `planificar-coleccion`, ya mostrada.

Tercero, el rango nombrado dinámico, en `rangos-nombrados-de`:

```
OFFSET(Asignaturas!$A$4,0,0,COUNTA(Asignaturas!$A$4:$A$29),5)
```

El rango se dimensiona con lo escrito, no con la reserva completa. Ese es el mecanismo, y ese es también el origen exacto del límite documentado: `COUNTA` cuenta celdas no vacías, no filas ocupadas. Si el usuario deja un hueco en medio del listado, `COUNTA` devuelve un número menor que la posición de la última entrada y esa entrada se queda fuera del rango. La emulación funciona mientras el listado se llene sin huecos.

Cuarto, la guarda de fila, en `excel/emision.lisp`:

```
(defun con-guarda-de-fila (a coleccion hoja ambito plan formula)
  (let ((clave (or (first (nucleo:clave coleccion))
    (nucleo:nombre (first (nucleo:campos coleccion))))))
```

```
(if (eq (nucleo:crecimiento coleccion) :crece)
  (format nil "IF(~a=\"\", \"\", ~a)" (celda hoja clave *fila*) formula)
  formula)))
```

Sin ella, las filas reservadas se llenan de ceros. Y las fórmulas se escriben **también en las filas de reserva**, porque si no, la primera fila que escriba el usuario no calcularía nada. Es, dice el comentario del código, una de las cosas que el corpus aprendió por las malas.

Cuatro lugares del código, una capacidad. En la arquitectura web, la misma capacidad se cumple con un botón.

7.3.5. El plano de libro y las tres reglas

La emisión no escribe el `.xlsx`: produce un **plano de libro**, una descripción puramente mecánica en JSON. Su vocabulario es cien por ciento de hoja de cálculo —celdas, fórmulas ya compiladas, estilos, rangos nombrados, validaciones, protecciones— y eso está bien, porque a esa altura ya estamos dentro de la arquitectura de Excel. Lo que importa son las tres reglas que lo gobiernan, escritas en el encabezado de `excel/plano.lisp`:

1. **El plano lo posee esta arquitectura.** Ningún otro sistema lo conoce.
2. **El núcleo no lo ve nunca.**
3. **Web y texto no tienen nada parecido: emiten directamente.** La asimetría la causa que un `.xlsx` sea un contenedor binario comprimido (*ECMA-376: Office Open XML File Formats 2016*), que es un hecho mecánico y no semántico.

El plano de una hoja lleva trece claves: nombre, fila de encabezado, fila primera, capacidad, color de pestaña, encabezados, celdas, fórmulas, editables, validaciones, formatos condicionales, rangos nombrados y leyenda. Tres salen del rol de campo y no de ninguna decisión aparte: el color de pestaña —azul si en la hoja se escribe a mano, gris si todo se calcula—, los rangos editables y las validaciones. La paleta de severidades, en cambio, vive en `excel/emision.lisp` y no en el nodo de la marca:

```
(defparameter +paleta+
  '(:problema . "FFC7CE")
    (:advertencia . "FFEB9C")
    (:informativa . "DDEBF7")))
```

Que esté aquí y no en la marca es la corrección del hallazgo sobre (Morales Lazo 2026): allí la regla de marcado declaraba color en la definición, el nodo no lo llevaba, y el backend lo asignaba por orden de aparición. Aquí la marca declara un significado y una severidad, y cada arquitectura decide cómo se ve.

7.4. El materializador en Python

7.4.1. Qué hace

`materializador/materializar.py` son 221 líneas que leen un plano de libro y producen un `.xlsx` con `openpyxl`. Hace ocho cosas por hoja, en este orden: escribir encabezados, escribir celdas, escribir fórmulas, aplicar formatos condicionales, aplicar validaciones, escribir la leyenda, aplicar la presentación (bordes, anchos de columna, inmovilizar el encabezado, color de pestaña) y proteger la hoja. Después registra los rangos nombrados en el libro.

7.4.2. Qué no hace

No conoce el dominio. No sabe qué es un profesor, ni una defensa, ni una asignatura. Conoce celdas, fórmulas, estilos y rangos. Todo el trabajo semántico ya lo hizo la arquitectura de Excel, en Lisp. Su función más significativa en este sentido es `_nombre_valido`:

```
def _nombre_valido(nombre):  
    """Los nombres de hoja tienen tope de 31 caracteres y prohíben algunos  
    signos. Es una restricción del formato, no del dominio, y por eso se  
    resuelve aquí."""  
    limpio = "".join(c for c in nombre if c not in "[]:*/\\")  
    return limpio[:31] or "Hoja"
```

Ese comentario marca exactamente la frontera. Un tope de 31 caracteres en un nombre de hoja es una restricción del formato OOXML (*ECMA-376: Office Open XML File Formats 2016*). No tiene nada que ver con el dominio y por eso no sube.

7.4.3. Por qué existe

Porque no hay ninguna biblioteca de Common Lisp capaz de **escribir** `xlsx` con fórmulas, formato condicional, validación de datos, rangos nombrados y protección de celdas. Se comprobó: las bibliotecas de Common Lisp para `xlsx` que existen son de lectura. `lisp-xl` es un cargador orientado a archivos grandes, y la biblioteca `xlsx` de Carlos Ungil lee celdas por dirección. Para escribir con esas características, la vía habitual desde Lisp sería enlazar por FFI contra una biblioteca de C como `libxlsxwriter`, lo que introduciría una dependencia binaria mucho más pesada que un script de Python. Escribir un emisor OOXML desde cero son semanas de trabajo que no aportan nada a la pregunta de esta tesis, y reintroducirían errores que el corpus ya resolvió contra el usuario real.

7.4.4. Por qué esto no es el error de 2026

Se parece por fuera —hay un JSON con fórmulas de Excel en el medio—, así que hay que poder explicarlo en dos frases:

En 2026, el JSON con fórmulas **era el punto de extensión**: era el sitio por donde, según aquel trabajo, iba a entrar un backend HTML. Por eso la afirmación era falsa: lo que ese JSON lleva no es una descripción del cálculo, es el cálculo ya compilado a Excel.

Aquí el punto de extensión es el protocolo, que está **cuatro fases más arriba**. El plano vive dentro de una arquitectura concreta, por debajo de ese punto, y ninguna otra arquitectura lo toca: la web y el texto plano emiten directamente, sin intermediario.

El mapa completo, con la frontera marcada:

```
situacion/excel (Lisp)
  fase 5: plan          decide hojas, columnas, filas, reservas
  fase 6: emission      produce el PLANO DE LIBRO (JSON)
  |
  | <--- frontera: por DEBAJO del punto de extension
  v
materializador/materializar.py (Python)
  lee el plano, llama a openpyxl
  |
  v
  .xlsx
```

Hay además una ventaja práctica: el plano es comprobable sin generar archivos. Se puede afirmar sobre él —qué celda lleva qué fórmula, qué rango está protegido— en pruebas rápidas, y dejar el round-trip por LibreOffice para verificar que las fórmulas evalúan.

Discrepancia con el documento de diseño. La sección 9.4 de 02-ARQUITECTURA.md dice que el materializador reutiliza siete módulos del repositorio Horario (`comun/proteccion.py`, `comun/rangos.py`, `comun/formato.py`, `comun/leyenda.py`, `comun/impresion.py`, `comun/portada.py`, `comun/estilos_base.py`), 631 líneas ya refinadas contra el usuario real. **No es así en el código.** `materializar.py` es autónomo: importa `json`, `sys`, `pathlib` y `openpyxl`, y nada más. Las ideas de esos módulos —la protección sin contraseña, la leyenda escrita, el ancho de columna calculado— sí están reimplementadas en él, pero el código no se reutiliza. Manda el código.

7.5. La arquitectura web

7.5.1. Por qué se elige

Se elige por ser **lo más distinto posible de una hoja de cálculo manteniendo el recálculo vivo**. No tiene rejilla, no tiene direcciones, no tiene capacidad reservada. Si el protocolo estuviera formulado en términos de hoja de cálculo, la web lo reventaría de inmediato. Es el backend que obliga a que la representación intermedia sea honesta.

Su plan lo demuestra por omisión:

```
(defclass plan-de-web ()
  ((situacion :accessor situacion :initarg :situacion)
   (entorno :accessor entorno :initarg :entorno :initform nil))
  (:documentation "Plan de la web. Opaco para el nucleo, y casi vacio: una
  pagina no tiene direcciones que repartir, que es justo por lo que esta
  arquitectura sirve de control."))
```

Dos ranuras. El plan de Excel tiene cuatro, y su clase `hoja` tiene siete más. Si el protocolo obligara a que el plan tuviera alguna forma concreta, esa diferencia no sería posible.

La clase no hereda de `con-rejilla`, y el comentario del código dice por qué: *“no la tiene, y resulta que no le hace falta: ninguna descripción pide una rejilla, porque la rejilla nunca fue un concepto del dominio”*. Esa frase es comprobable. En los dos casos del corpus, ninguna descripción pide `con-rejilla`, y en los seis informes de conformidad medidos esa capacidad no aparece ni una vez.

7.5.2. La emisión a JavaScript

`web/expresiones.lisp` especializa quince métodos de `emitir-expresion`, cuatro más que la hoja de cálculo. Las funciones de nombres son tres y son triviales, pero conviene verlas porque son la contraparte exacta del direccionamiento de Excel:

```
(defun js-variable (simbolo)
  (string-downcase (subseq (symbol-name simbolo) 0 1)))
(defun js-coleccion (nombre)
  (format nil "D[~s]" (string-downcase (symbol-name nombre))))
(defun js-campo (nombre)
  (format nil "[~s]" (string-downcase (symbol-name nombre))))
```

FILA es `f`. No es direccionamiento: es cómo se llaman las cosas en el código que se emite, y vive enteramente dentro de esta arquitectura, igual que `*fila*` y `*hoja*` viven dentro de la de Excel.

La aplicación de operadores envuelve las comparaciones numéricas en `N`:

```
((member op '("<" "<=" ">" ">=" "+" "-" "*" "/" ) :test #'string=)
  (format nil "(N(~a) ~a N(~a))" (first partes) js (second partes)))
((string= op "=") (format nil "IG(~a,~a)" (first partes) (second partes)))
```

Si no, `"10"` sería menor que `"9"`, porque JavaScript compara cadenas lexicográficamente. Es la misma coerción que hace el evaluador de referencia, y por eso los dos dan el mismo resultado.

7.5.3. La misma expresión en los dos destinos

Esta es la comparación que hay que mirar. La columna del medio y la de la derecha salen de los artefactos generados el 21 de septiembre de 2026, sin retocar.

Nodo del núcleo	Hoja de cálculo	Página
<code>literal</code>	<code>""</code>	<code>""</code>
<code>ref-campo</code>	<code>\$D4</code>	<code>V(f["asignadas"])</code>
<code>aplicacion resta</code>	<code>(\$C4-\$D4)</code>	<code>(N(V(f["frecuencia"]))) - N(V(f["asignadas"])))</code>

Nodo del núcleo	Hoja de cálculo	Página
aplicacion =	(\$D4=\$C4)	IG(V(f["asignadas"]),V(f["frecuencia"])
condicional	IF(a,b,c)	(a ? b : c)
sin-escribir	(\$D4="")	VACIO(V(f["estudiante"]))
agregado :distintas	SUMPRODUCT((r<>"")/COUNTIF(r;r=\$t))	newSet().size

Y las dos construcciones grandes, enfrentadas. La derivación del tutor —el mismo nodo proyeccion sobre busqueda— sale así:

```
=IF($A4="", "", IFERROR(VLOOKUP($D4, Tesis!$A$4:$E$7, 2, FALSE), ""))
```

```
"tutor": (f) => PROY((D["tesis"].find(c => IG(V(c["estudiante"]),
                                     V(f["estudiante"])))) ?? null),
               "tutor", "")
```

Y la marca de colisión —el mismo nodo existe con comparten dentro—:

```
(SUMPRODUCT((((Citaciones!$E$4:$E$27=$E4)+ ... dieciseis terminos ... )>0)
*(Citaciones!$B$4:$B$27=$B4)
*(Citaciones!$A$4:$A$27=$A4)
*(ROW(Citaciones!$A$4:$A$27)<>ROW($A4)))>0)
```

```
cond: (f) => D["citaciones"].some(o => o !== f &&
  (IG(V(o["dia"]),V(f["dia"]))) &&
  IG(V(o["momento"]),V(f["momento"]))) &&
  COMPARTEN(o, f, ["tutor", "oponente", "presidente", "secretario"])))
```

La expresión del lenguaje que produce las dos es una sola, y está en `corpus/defensas-de-tesis.lisp`:

```
(existe otra :en citaciones
:distinta-de fila
:donde (y (= (de otra dia) (de fila dia))
(= (de otra momento) (de fila momento))
(comparten otra fila (tutor oponente presidente secretario))))
```

Ni `excel/formula.lisp` ni `web/expresiones.lisp` ven al otro. Los dos se cuelgan de `emitir-expresion` especializando en su propia arquitectura. Nada de lo que cambia entre las dos columnas subió nunca por encima del protocolo.

7.5.4. Las dos implementaciones de la coerción

Hay un detalle que conviene declarar porque es una deuda real del diseño: la coerción de valores está escrita **dos veces**. Una en Lisp, en `nucleo/evaluador.lisp`:

```
(defun vacio-p (valor)
  (or (null valor) (and (stringp valor) (string= valor ""))))
```

```
(defun iguales-p (a b)
  (cond ((and (realp a) (realp b)) (= a b))
        ((and (vacio-p a) (vacio-p b)) t)
        ((or (vacio-p a) (vacio-p b)) nil)
        ((and (stringp a) (stringp b)) (string= a b))
        (t (equal a b))))
```

Y otra en JavaScript, en `web/recursos.lisp`, dentro de `+ayudas-js+`:

```
function VACIO(x){ return x === null || x === undefined || x === ''; }
function IG(a,b){ if (VACIO(a) && VACIO(b)) return true;
                  if (VACIO(a) || VACIO(b)) return false;
                  if (typeof a === 'number' || typeof b === 'number')
                      return N(a) === N(b);
                  return String(a) === String(b); }
```

Las dos tienen que coincidir. Si no coinciden, la página calcula cosas distintas de las que dice el lenguaje, y el trabajo entero pierde sentido.

Lo importante es que esa coincidencia **es comprobable, y se comprueba**. `materializador/verificar-web.js` extrae el script de la página generada, corta la parte que toca el DOM, lo ejecuta en el módulo `vm` de Node, recalcula todo y compara los valores con el oráculo del evaluador de referencia. La medición del 21 de septiembre de 2026: 48 valores en `plan-del-grupo.html` y 60 en `defensas-de-tesis.html`, cero discrepancias.

La misma lógica vale para la hoja de cálculo, con otro procedimiento: allí quien evalúa las fórmulas es LibreOffice, y `materializador/verificar.py` compara los valores recalculados contra el mismo oráculo. 30 celdas en `plan-del-grupo.xlsx` y 52 en `defensas-de-tesis.xlsx`, cero discrepancias. El capítulo de validación detalla el procedimiento.

Duplicar la coerción no es elegante. La alternativa —que la página llamara a un intérprete— sacrificaría la propiedad que hace útil al artefacto: que sea un archivo HTML autónomo, sin servidor y sin conexión, que se abre con doble clic.

7.5.5. El resto de la página

`web/recursos.lisp` guarda lo que no depende de la descripción: los estilos y la biblioteca de apoyo. `web/emision.lisp` genera lo que sí depende: `D` (los datos), `DERIV` (una función por campo derivado), `MARCAS` (una entrada por marca, con su condición y su explicación), `DOMINIOS` (una función por campo de entrada con dominio) y `ESQUEMA` (la forma de las tablas). Los dominios se emiten como función y no como lista fija porque un dominio puede depender de los datos: si el usuario añade una fila, el desplegable tiene que enterarse. Es lo mismo que hace el rango dinámico de la hoja de cálculo, con mucho menos esfuerzo.

El rol de campo decide aquí exactamente lo mismo que en la hoja de cálculo, con otra realización: de entrada es un control —`select` si el dominio bloquea, `input` con `datalist` si solo advierte—, derivado es texto de solo lectura, fijo es texto. Un solo hecho del dominio,

dos destinos.

7.6. La inversión de capacidades

Esta es la sección que conviene defender con más cuidado, porque es el resultado que demuestra que la jerarquía de capacidades no es taxonomía decorativa.

La intuición dice que una hoja de cálculo puede más que una página web, o al revés, y que las arquitecturas se ordenan de más capaz a menos capaz. El código dice otra cosa.

El crecimiento. En la página es nativo y cuesta diez líneas dentro de `render`:

```
if (tabla.crece) {
  const boton = document.createElement('button');
  boton.textContent = '+ Anadir fila';
  boton.addEventListener('click', () => {
    const nueva = {};
    for (const c of tabla.campos) if (c.rol !== 'derivado') nueva[c.n] = '';
    D[tabla.nombre].push(nueva);
    render();
  });
  seccion.appendChild(boton);
}
```

En la hoja de cálculo hay que reservar filas, escribir las fórmulas también en las reservadas, envolver cada fórmula en una guarda para que no muestre ceros, y definir un rango con `OFFSET` y `COUNTA` que se dimensiona con lo escrito. Cuatro mecanismos, y un límite que hay que documentar: no admite huecos.

El conteo de distintos. En la página es una palabra:

```
new Set(valores.filter(x => !VACIO(x))).size
```

En la hoja de cálculo no existe. `COUNTIF` no sabe contar únicos, y las fórmulas matriciales no se comportan igual en Excel que en LibreOffice Calc, así que hay que emularlo con el rodeo plano `SUMPRODUCT((rango<>"")/COUNTIF(rango,rango&""))`, que es el único portable.

La relación uno a muchos. La página la cumple porque una lista es una lista. La hoja de cálculo la declara emulada con hoja auxiliar de clave numerada y un número fijo de líneas por fila padre, avisando cuántas faltan si se desbordan. Hay que decirlo con honestidad: ninguna de las dos descripciones del corpus ejercita hoy esa capacidad. Está declarada y no está probada.

Y en el otro sentido: la hoja de cálculo hereda de `con-rejilla` y la página no. La página degrada esa capacidad —“*se dibuja una tabla, pero sin direcciones ni coordenadas*”— y eso también aparecería en su informe si alguna descripción la pidiera.

Los conjuntos de capacidades medidos lo dicen numéricamente. De las doce capacidades declaradas en `protocolo/capacidades.lisp`, `excel` hereda de ocho, `web` de diez y `texto` de una. Pero **no están anidados**: `excel` tiene `con-rejilla`, que `web` no tiene; `web` tiene

con-crecimiento, con-conteo-de-distintos y con-relacion-uno-a-muchos, que excel no tiene.

La conclusión es esta, y es la contribución conceptual del capítulo:

**No hay una arquitectura que pueda más y otras que puedan menos.
Cada una puede cosas distintas.**

Si las capacidades se ordenaran en una escala, la jerarquía sobraría: bastaría un número. Como no se ordenan, hace falta nombrar cada propiedad por separado y componer. Esa es la razón de ser del mecanismo, y es exactamente la idea que ([Arcia Corcho 2017](#)) atribuye a ADOL* y GEMURAL, y que ni ese trabajo ni ([Cápiro Colomar 2025](#)) ni ([Morales Lazo 2026](#)) llegaron a implementar.

El resultado está además fijado como prueba. `pruebas/conformidad.lisp` contiene `conformidad-inversion-de-capacidades`, que materializa la misma descripción con las tres arquitecturas y falla si la hoja de cálculo deja de emular el crecimiento o si la página deja de cumplirlo de forma nativa. La inversión no es una observación del autor: es una condición que el sistema se impone a sí mismo.

7.7. Cómo se añade una arquitectura nueva

Esto es lo que recibiría un tercero. Son seis pasos y ninguno toca el núcleo, el protocolo ni otra arquitectura.

1. Definir la clase, heredando de las capacidades que cumple.

```
(defclass mi-arquitectura (protocolo:con-derivacion-viva
                          protocolo:con-marcado-textual)
  ())
```

No hay que declarar las que no se cumplen. El método por defecto de `capacidades` las deduce de las superclases con `typep`.

2. Definir su clase de plan. Puede ser tan rica como la de Excel —que reparte hojas, columnas, filas y reservas— o tan pobre como la de la web, que tiene dos ranuras. El núcleo no mira dentro. Es el objeto donde vive todo el direccionamiento que la arquitectura necesite inventarse.

3. Especializar planificar y emitir. `planificar` construye el plan; `emitir` escribe el artefacto en un flujo o una ruta. En la práctica conviene especializar también `materializar`, que es el punto de entrada de alto nivel y el que envuelve todo en `con-informe`.

4. Especializar emitir-expresion para cada tipo de nodo. Son quince nodos en el álgebra de expresión: `literal`, `ref-parametro`, `ref-campo`, `aplicacion`, `condicional`, `sin-escribir`, `concatenacion`, `concordancia`, `conjunto`, `agregado`, `busqueda`, `proyeccion`, `existe`, `comparten` y `vecino`. No hacen falta los quince: la hoja de cálculo especializa once y atiende `busqueda`, `vecino` y `comparten` desde dentro de otros métodos. Lo que no se especialice y llegue, falla con un error claro, que es el comportamiento correcto.

5. Especializar resolver-carencia para lo que no se cumple. Devolviendo `:emula`, `:degrada` o `:rechaza`, **con su nota**. La nota de una emulación tiene que decir a qué coste y con qué límite; el juego de conformidad lo exige.

6. Pasar el juego de pruebas de conformidad. Una arquitectura es conforme si materializa sin error toda descripción del juego, si cumple en modo estricto las capacidades que declara, y si para cada capacidad que no declara emite una de las cuatro respuestas con su nota, en vez de fallar en silencio o producir algo que parece correcto y no lo es.

Dos garantías acompañan a esos seis pasos, y las dos están implementadas, no prometidas.

La primera: **no se puede declarar una operación del protocolo que no reciba la arquitectura.** `protocolo/operacion.lisp` define la macro `definir-operacion`, que señala un error en tiempo de expansión si el primer parámetro no se llama `arquitectura`. El sistema no compila. No es una convención vigilada por revisión de código: es una imposibilidad. Las trece operaciones del protocolo están declaradas con esa macro.

La segunda: **una arquitectura depende del protocolo y de nada más.** El grafo de dependencias de `situacion.asd` lo declara —`situacion/texto`, `situacion/excel` y `situacion/web` dependen solo de `situacion/protocolo`—, así que una arquitectura no puede ver el lenguaje ni usar sus macros por descuido, y ninguna puede ver a otra. El invariante I3 lee esas declaraciones y comprueba que se cumplen.

El costo real: la arquitectura de texto plano son 209 líneas; la web, 672, de las cuales 246 son estilos y biblioteca de apoyo que no dependen de la descripción. Ese es el orden de magnitud que conviene citar cuando se pregunte cuánto cuesta añadir un destino.

7.8. Dos correcciones al documento de diseño

Se registran aquí porque la regla del trabajo es que manda el código.

Número de operaciones del protocolo. El documento de arquitectura dice doce funciones genéricas y lista doce. El código declara **trece**: las doce más `materializar`, que es el punto de entrada de alto nivel y la que usan la demostración y el juego de conformidad.

Número de capacidades. El documento lista once clases de capacidad, y su tabla de la sección 2.2 no tiene columna de crecimiento. El código declara **doce**: las once más `con-crecimiento`, que es justamente la que sostiene el argumento de la inversión.

Las dos se comprobaron en ejecución, leyendo los registros que mantienen `definir-operacion` y `definir-capacidad`. Ninguna cambia el argumento del trabajo; las dos hay que arreglarlas en el documento de diseño antes de la defensa.

Capítulo 8

Experimentación y validación

Este capítulo presenta la evidencia. Es el capítulo que (Cápiro Colomar 2025) no tiene —su índice pasa de los detalles de implementación directo a las conclusiones— y que (Morales Lazo 2026) tiene a medias: hay capturas de pantalla de libros ya calculados, pero no hay un procedimiento que un tercero pueda volver a ejecutar y que falle solo cuando algo se rompa.

Todo lo que sigue se obtuvo ejecutando el sistema descrito en los capítulos anteriores. Los números salen de dos comandos, ambos en la raíz de `situacion/`: `./ejecutar-pruebas.sh` y `./demostrar.sh`, que va de la descripción al libro calculado en un solo paso. Cuando algo no se pudo medir, se dice. Cuando algo es interpretación y no medición, se dice también.

8.1. Qué hay que demostrar

La pregunta científica de este trabajo tiene dos mitades, y la separación importa para el diseño del experimento.

La **primera mitad** pregunta si la representación intermedia es independiente del sistema que la materializa. Se refuta encontrando una construcción del dominio cuya representación intermedia no pueda materializarse en alguna arquitectura sin meter en ella vocabulario propio de otra. Es una afirmación sobre la *forma* del sistema, y le corresponden dos clases de evidencia: un invariante que recorre el núcleo y el protocolo y falla si encuentra una letra de columna, una coordenada o el nombre de una función de hoja de cálculo; y que la misma descripción, sin tocar una línea, se materialice en tres destinos que no comparten nada.

La **segunda mitad** pregunta si un tercero puede incorporar una arquitectura de salida nueva sin modificar el núcleo. Es una afirmación sobre el *mecanismo de extensión*, y su evidencia también es doble. Que el punto de extensión tenga la forma correcta: que toda operación del protocolo reciba la arquitectura, porque una operación que no despacha sobre ella no tiene dónde enganchar una segunda. Y que tres arquitecturas de naturaleza deliberadamente distinta se hayan escrito contra ese protocolo sin que ninguna conozca a las otras.

La correspondencia entre pregunta y evidencia queda así:

Mitad de la pregunta	Evidencia	Dónde
Independencia de la representación	Invariantes I1 (léxico) e I3 (dependencias)	<code>pruebas/invariantes.lisp</code>
Forma del punto de extensión	Invariante I2 (firma del protocolo)	<code>pruebas/invariantes.lisp</code> , <code>protocolo/operacion.lisp</code>
Que la independencia sirva de algo	La misma descripción en tres arquitecturas	<code>demostrar.sh</code> , <code>salida/</code>
Que lo que una arquitectura inventa no se escape	Invariante I4 (aislamiento del plano)	<code>pruebas/conformidad.lisp</code>
Que la degradación no sea decorativa	Los seis informes de conformidad	<code>protocolo/informe.lisp</code>
Que lo generado además funcione	<i>Round-trip</i> por LibreOffice	<code>materializador/verificar.py</code>

Hay una asimetría que conviene declarar desde el principio. Los invariantes prueban que el sistema *tiene la forma* que se afirma; el *round-trip*, que *produce artefactos correctos*. Ninguna clase sustituye a la otra: un diseño impecable que genere fórmulas rotas no sirve de nada, y un libro que funcione producido por un generador con el destino cableado dentro no responde la pregunta de esta tesis.

8.2. Invariantes arquitectónicos

Las afirmaciones del tipo «el núcleo es independiente de la arquitectura» son las que se caen en las defensas. Aquí dejan de ser afirmaciones: están implementadas como pruebas, sobre un marco propio de cien líneas (`pruebas/marco.lisp`) que deliberadamente no tiene dependencias. Eso último no es purismo. Quien reciba el protocolo para añadir su propia arquitectura tiene que poder cargarlo y comprobarlo con un SBCL recién instalado y nada más, sin Quicklisp y sin descargar bibliotecas.

El resultado de ejecutar `./ejecutar-pruebas.sh` es, literalmente, su última línea:

```
25 pruebas, 453 comprobaciones, 0 fallidas
```

Las veinticinco pruebas se reparten en tres archivos: ocho son los invariantes I1, I2 e I3 (`pruebas/invariantes.lisp`); nueve son del lenguaje, el evaluador de referencia y el análisis estático (`pruebas/nucleo.lisp`); ocho son de conformidad e incluyen los invariantes I4 e I5 (`pruebas/conformidad.lisp`). Interesa más el número de **comprobaciones** que el de pruebas: `i1-lexico` sola hace una por cada símbolo propio de dos paquetes, que es de donde sale la mayoría de las 453.

8.2.1. I1 — Léxico del núcleo y del protocolo

I1 comprueba que ni el núcleo ni el protocolo nombren direcciones. La prueba `i1-lexico` recorre los símbolos de `SITUACION.NUCLEO` y `SITUACION.PROTOCOLO` y falla si el nombre de

alguno contiene vocabulario de direccionamiento: CELDA, COORDENADA, LETRA-DE-COLUMNA, NUMERO-DE-FILA, RANGO, HOJA, LIBRO, los nombres de funciones de hoja de cálculo (COUNTIF, VLOOKUP, SUMPRODUCT, OFFSET, COUNTA, IFERROR, BUSCARV...), más EXCEL, OPENPYXL y XLSX.

Hay tres matices que son los que hacen que la prueba sirva de algo, y conviene declararlos porque sin ellos I1 parecería más fuerte de lo que es.

El primero: **la lista es de direccionamiento, no de estructura**. FILA, CAMPO y COLECCION son vocabulario del dominio y están permitidos. Una colección tiene filas, y (de fila campo) se lee perfectamente. Lo prohibido es la *dirección* de una fila, no su existencia. Prohibir FILA a secas haría la prueba más estricta y bastante más inútil: obligaría a renombrar el concepto central del dominio para contentar a un verificador.

El segundo: **I1 usa dos listas distintas, no una**. La función vocabulario-prohibido-en devuelve, para el núcleo, la lista de direccionamiento más REJILLA, ARQUITECTURA y BACKEND; para el protocolo, solo la de direccionamiento. La distinción tiene razón de ser. El protocolo tiene que poder nombrar la capacidad **con-rejilla**: que una arquitectura declare si organiza sus elementos en filas y columnas es información *sobre la arquitectura*, no sobre la situación, y el mecanismo de degradación no existe sin ella. El núcleo, en cambio, no tiene ningún motivo para saber que existe algo llamado rejilla. Si algún día lo necesitara, no sería que le falta un sinónimo: sería que se metió el destino dentro del modelo, que es exactamente como ([Morales Lazo 2026](#)) terminó con conceptos de rejilla dentro de un árbol supuestamente independiente.

El tercero: **la comprobación es sobre símbolos, no sobre el texto del archivo**. Esto contradice el documento de diseño, que describe I1 como un recorrido del código fuente; manda el código. La diferencia tiene consecuencias en las dos direcciones. A favor: los comentarios pueden hablar de Excel todo lo que haga falta para explicar por qué algo está como está, y de hecho lo hacen. En contra, y hay que decirlo: I1 no vería una cadena literal "COUNTIF" escondida dentro de una función del núcleo. Lo que garantiza es que no existe una función llamada LETRA-DE-COLUMNA, no que el núcleo sea lexicalmente puro en todo sentido concebible.

8.2.2. Cómo se comprobó que I1 muerde

Una prueba de lista negra tiene un modo de fallo silencioso muy fácil: pasa igual de verde con la lista vacía, o con un error de lógica que haga que nunca encuentre nada. Por eso existe **i1-la-prueba-lexica-muerde**, que comprueba la propia comprobación contra casos conocidos, sin tocar los paquetes de verdad. Se le pasan cinco nombres inventados que **deben** detectarse —LETRA-DE-COLUMNA, PRIMERA-CELDA, RANGO-DINAMICO, COMPILAR-FORMULA-EXCEL y CONTAR-CON-COUNTIF— y seis que **no**, porque son vocabulario legítimo del dominio: FILA, CAMPO, COLECCION, DERIVACION, AMBITO y MARCA. Después comprueba la asimetría entre los dos paquetes: que CON-REJILLA sea aceptable en el protocolo y rechazable en el núcleo.

Es una prueba sobre una prueba, y es de las piezas más baratas y más rentables del trabajo. Sin ella, I1 sería una afirmación disfrazada de verificación.

8.2.3. I2 — Firma del protocolo

I2 es el invariante central. Es la detección automática del fallo que dejó sin cerrar ([Morales Lazo 2026](#)): su operación de compilación de expresiones,

```
(defgeneric compile-excel-formula
  (nodo col-map data-names fila primera-fila ultima-fila))
```

no recibe el destino, y sus parámetros son de hoja de cálculo. El análisis está en el capítulo 2; lo que importa aquí es que esa forma se pueda detectar sola.

No se corrige con una convención ni con una revisión de código: se corrige haciendo que el sistema **no admita la forma incorrecta**. El macro `definir-operacion`, en `protocolo/operacion.lisp`, exige que el primer parámetro se llame `arquitectura` y señala un error en tiempo de expansión si no lo es. No compila. Las **trece** operaciones del protocolo están declaradas así en `protocolo/genericas.lisp`, y el macro las registra en una lista que las pruebas recorren después.

Conviene ser preciso sobre lo que I2 comprueba, porque el documento de diseño lo enuncia de otra forma. La función `parametro-de-arquitectura-p` mira el **nombre del primer parámetro de la lambda-lista**, no la clase sobre la que se especializan los métodos, y lo compara por nombre y no por identidad de símbolo para que siga valiendo si el protocolo se declara desde otro paquete. La consecuencia honesta es que I2 garantiza la forma del punto de extensión, no que cada método concreto especialice sobre la clase `arquitectura`.

La prueba `i2-toda-operacion-recibe-la-arquitectura` recorre el registro y comprueba la firma de cada entrada. Por sí sola no demostraría gran cosa: comprobaría lo mismo sobre un registro vacío o sobre un macro roto. Por eso hay otras dos.

8.2.4. Cómo se comprobó que I2 muerde

`i2-se-rechaza-una-operacion-sin-arquitectura` es la prueba negativa. Intenta declarar una operación **con exactamente la forma que tenía la de 2026** y comprueba que el sistema la rechaza:

```
(macroexpand-1
  '(situacion.protocolo::definir-operacion compilar-formula
    (nodo mapa-de-columnas fila primera-fila ultima-fila)))
```

Si esa expansión no señala un error, el invariante no sirve de nada y la prueba falla. Y para que no pase por el motivo equivocado —por ejemplo, porque el macro falle siempre— la misma prueba incluye el control positivo: expandir `(definir-operacion operacion-de-prueba (arquitectura nodo plan))` tiene que funcionar.

La tercera, `i2-ninguna-generica-se-cuela-sin-registrar`, cierra la puerta de atrás: recorre los símbolos exportados por `SITUACION.PROTOCOLO`, se queda con los que son funciones genéricas, y exige que estén en el registro. Tiene una lista explícita de exenciones con cuatro nombres, que son los lectores de las condiciones `carencia-de-capacidad` y `capacidad-no-disponible` declaradas en `protocolo/carencia.lisp`. En Common Lisp (*American Natio-*

nal Standard for Information Systems. Programming Language Common Lisp 1994) un lector declarado con `:reader` es una función genérica, y sin la lista la prueba los reportaría. La lista es explícita y corta a propósito: reconocerlos por una regla automática dejaría la puerta abierta a que una operación de verdad se colara por la misma vía. Añadir un nombre ahí tiene que ser un acto deliberado que se vea en la revisión.

Éste es el punto en el que este trabajo se separa de su antecedente de forma comprobable. No es que se haya *decidido* no repetir el fallo de 2026: es que la herramienta no deja repetirlo.

8.2.5. I3 — Grafo de dependencias

I3 lee las declaraciones de sistema del `.asd` con `asdf:system-depends-on` y comprueba tres cosas en tres pruebas separadas. Que cada sistema dependa solo de lo permitido: el núcleo de nada, y el protocolo, el lenguaje y el análisis solo del núcleo. Que el protocolo no vea el lenguaje, porque el contrato de extensión se define contra el dominio y no contra la sintaxis; si pudiera ver las macros acabaría usándolas, y entonces una arquitectura tendría que conocer el lenguaje para existir. Y que ninguna arquitectura dependa de otra: `situacion/texto`, `situacion/excel` y `situacion/web` dependen del protocolo y de nada más. Lo que dos compartan sube a una clase de capacidad, que es el uso natural del despacho múltiple y la herencia múltiple de CLOS (Keene 1989; Kiczales, Rivières, y Bobrow 1991).

A I1, I2 e I3 se suman dos invariantes que viven con las pruebas de conformidad porque necesitan las arquitecturas cargadas. I4 comprueba que ningún símbolo propio de `SITUACION.EXCEL` se referencie desde los demás paquetes: lo que una arquitectura se inventa es suyo y no sale de ahí. Si otro sistema referenciara el plano de libro, el plano dejaría de ser un detalle interno y pasaría a ser una interfaz, que es lo que le ocurrió al JSON intermedio de 2026. I5 comprueba que no haya capacidades declaradas que ninguna arquitectura ejercite; hoy la única huérfana es `con-derivacion-por-consulta`, que es de SQL, que no está.

8.3. La misma descripción en tres arquitecturas

Éste es el experimento central y el que responde a la primera mitad de la pregunta.

El procedimiento: se toma una descripción del corpus y se compila contra tres arquitecturas de naturaleza deliberadamente distinta —texto plano, libro de hoja de cálculo y página web autónoma— y se comparan los tres artefactos. **La descripción es la misma en los tres casos y no se tocó una línea entre compilación y compilación**; lo único que cambia es la instancia de arquitectura que recibe `materializar`. Todo está automatizado en `demostrar.sh`.

La descripción usada aquí es `corpus/plan-del-grupo.lisp`, que corresponde a la tabla de estadísticas de la hoja de grupo del generador real de horarios (`horarios/hoja_grupo.py`, columnas I a M). Declara una colección con cinco campos —tres fijos, uno de entrada y uno derivado—, tres marcas con significado y severidad, y una vista. El campo derivado es una resta: `(- (de fila frecuencia) (de fila asignadas))`.

8.3.1. El artefacto de texto plano

Plan de asignaturas del grupo D111

=====

Asignaturas

Abrev	Asignatura	Frec	Asignadas	Faltan	Estado
AL	Algebra Lineal	3	3	0	completa
L	Logica	2	2	0	completa
IP	Introduccion a la Programacion	2	2	0	completa
AM	Analisis Matematico	2	3	-1	Sobra 1 turno
ICD	Introduccion a Ciencia de Datos	2	1	1	Falta 1 turno
F	Fisica	2	1	1	Falta 1 turno

Nótese la columna **Estado**. La arquitectura de texto no tiene marcado visual, así que lo degrada a marcado textual y escribe el significado en palabras, con concordancia de plural: «Sobra 1 turno», «Falta 1 turno». La marca **completa** no declara :**explica**, y la arquitectura recurre a su nombre.

8.3.2. El plano de libro

La arquitectura de Excel no emite un **.xlsx**: emite un **plano**, una estructura de datos que describe el libro y que después materializa **materializador/materializar.py** con **openpyxl**. Tres fragmentos literales de **salida/plan-del-grupo-plano.json**:

```
{"celda": "E4", "formula": "=IF($A4=\"\", \"\", ($C4-$D4))"}

{"rango": "B4:B29 D4:D29 E4:E29", "estado": "excedida",
 "severidad": "problema", "formula": "($D4>$C4)", "relleno": "FFC7CE"}

{"nombre": "Asignaturas",
 "referencia": "OFFSET(Asignaturas!$A$4,0,0,COUNTA(Asignaturas!$A$4:$A$29),5)"}

```

Aquí sí hay coordenadas, letras de columna y nombres de función de hoja de cálculo. Es el sitio donde tienen que estar: dentro de la arquitectura de Excel, por debajo del punto de extensión, en una estructura que ningún otro sistema importa —lo comprueba I4—. Es lo contrario de lo que ocurría en 2026, donde esa información subía hasta la representación intermedia.

8.3.3. El JavaScript generado

La arquitectura web emite una página autónoma con recálculo vivo. Fragmento literal de **salida/plan-del-grupo.html**:

```
const DERIV = {};
DERIV["asignaturas"] = {
  "faltan": (f) => (N(V(f["frecuencia"])) - N(V(f["asignadas"])))
}
```

```
};

const MARCAS = [
  {nombre: "excedida", coleccion: "asignaturas", clase: "problema",
   campos: ["nombre", "asignadas", "faltan"],
   cond: (f) => (N(V(f["asignadas"])) > N(V(f["frecuencia"]))),
   explica: (f) => [...].map(T).join("")}
];
```

La misma derivación se convirtió en `=IF($A4="", "", ($C4-$D4))` en un sitio y en una función flecha en el otro. La misma marca **excedida** se convirtió en una regla de formato condicional con relleno `FFC7CE` y en una entrada de un arreglo con clase CSS **problema**. Ninguna de las dos arquitecturas conoce a la otra, y la descripción no menciona ni relleno, ni CSS, ni IF, ni fórmula.

Las funciones N, V, T, VACIO e IG que aparecen ahí no son adorno: son la coerción del núcleo, reimplementada en JavaScript para que un campo sin rellenar valga cero en aritmética y vacío en texto, igual que en el evaluador de referencia. Sin eso, los artefactos coincidirían en las fórmulas y discreparían en los bordes.

8.4. Los informes de conformidad

Cuando una arquitectura no cumple una capacidad de forma nativa, puede **emularla**, **degradarla** o **rechazarla**, y en los tres casos queda anotado. El informe de conformidad es el registro de esas decisiones y se produce solo. Lo que pide cada descripción lo calcula `protocolo/requerimientos.lisp`, que es del protocolo y no de cada backend, para que dos arquitecturas no puedan dar informes distintos sobre lo que se les está pidiendo.

Los seis informes de la corrida completa, resumidos:

Situación	Arquitectura	Nativas	Emuladas	Degradadas	Rechazadas
Plan del grupo	texto	1	0	4	0
Plan del grupo	excel	4	1	0	0
Plan del grupo	web	5	0	0	0
Defensas de tesis	texto	2	1	5	0
Defensas de tesis	excel	7	1	0	0
Defensas de tesis	web	8	0	0	0

Y el detalle de dos de ellos, copiado de la salida real:

Informe de conformidad		Arquitectura: texto
degrada	con-crecimiento	se escribe lo que hay; para anadir filas hay que regenerar
degrada	con-entrada	se escribe el valor actual; el documento no es editable
degrada	con-derivacion-viva	los valores se calculan al generar y no se recalculan
degrada	con-marcado-visual	el estado se escribe en una columna, sin color
cumple	con-marcado-textual	la marca insuficiente explica por que en palabras

1 nativas, 0 emuladas, 4 degradadas, 0 rechazadas		
Informe de conformidad		Arquitectura: excel
emula	con-crecimiento	20 filas de reserva y un rango que se dimensiona con lo escrito. Limite comprobado en el corpus: no admite huecos en medio del listado, porque el conteo de celdas no vacias no distingue una fila en blanco de una fila ausente y la ultima entrada se queda fuera del rango.
cumple	con-entrada	asignadas lo escribe el usuario
cumple	con-derivacion-viva	faltan se recalcula al editar
cumple	con-marcado-visual	la marca insuficiente senala un estado
cumple	con-marcado-textual	la marca insuficiente explica por que en palabras

4 nativas, 1 emuladas, 0 degradadas, 0 rechazadas		

8.4.1. Qué dice esta tabla

Tres lecturas, en orden de importancia.

La primera es la obvia: **texto degrada casi todo, Excel emula una cosa, la web lo cumple todo**. La arquitectura de texto plano se eligió precisamente por pobre: existe para que haya dos arquitecturas desde el primer día, y porque al no poder casi nada obliga a que la maquinaria de degradación exista desde el principio en vez de ser un capítulo que se escribe al final.

La segunda es la interesante, y es un **resultado experimental que invierte la intuición**. La capacidad **con-crecimiento** la cumple de forma nativa la página web y la *emula* la hoja de cálculo, que parece mucho más capaz. Pero crecer en una hoja es veinte filas en blanco reservadas y un rango nombrado que se dimensiona con COUNTA, con las trampas que eso arrastra —el propio informe declara el límite: no admite huecos en medio del listado—. En una página es un botón. Lo mismo ocurre con **con-conteo-de-distintos**, que la web tiene gratis y la hoja de cálculo tendría que emular con una columna auxiliar.

Ésa es la evidencia de que el mecanismo de capacidades **no es taxonomía decorativa**. Si las capacidades se hubieran repartido por prestigio de arquitectura, Excel habría salido por encima de la web en todo. Salen en distinto orden según la capacidad, porque la clasificación responde a qué sabe hacer cada arquitectura y no a cuál parece más potente. El sistema declara **doce** capacidades componibles en `protocolo/capacidades.lisp`; la clase `excel` (`excel/arquitectura.lisp`) hereda de ocho y deliberadamente **no** de `con-crecimiento` ni de `con-conteo-de-distintos`; la clase `web` (`web/arquitectura.lisp`) hereda de diez y deliberadamente **no** de `con-rejilla`, porque no la tiene y resulta que no le hace falta: ninguna descripción pide una rejilla, porque la rejilla nunca fue un concepto del dominio. La clase `texto` hereda de una sola, `con-marcado-textual`.

La tercera lectura es que la columna de rechazos está en cero en los seis casos. No es suerte: la respuesta por defecto de `resolver-carencia`, en `protocolo/carencia.lisp`, es **rechazar**, porque una arquitectura que calle ante una carencia debe producir un error y no un documento a medias. Que la columna esté en cero significa que las tres declararon explícitamente qué hacer con todo lo que se les pidió.

La política de qué se acepta vive fuera de las arquitecturas, en `*politica*`, con tres valores: `:permisiva`, `:estricta` y `:solo-nativo`. La prueba `conformidad-modo-estricto` la ejerce: en modo estricto la web pasa sobre el plan del grupo y texto plano no. Que texto plano no pase es información, no un fallo; si pasara, la política no estaría haciendo nada.

8.5. El *round-trip* por LibreOffice

Éste es el resultado más fuerte del trabajo, y responde a una objeción que toda tesis de generación de hojas de cálculo debería responder y ninguna de esta línea responde.

8.5.1. Por qué una batería de pruebas de Python no basta

Una prueba escrita con `openpyxl` comprueba que en la celda E4 hay la cadena `=IF($A4="", "", ($C4-$D4))`. No comprueba que esa cadena **calcule**. La diferencia no es teórica: `openpyxl` escribe en el archivo lo que se le diga, sin evaluarlo, y el formato (*ECMA-376: Office Open XML File Formats 2016*) admite perfectamente fórmulas sintácticamente válidas que dan `#REF!`, `#VALUE!` o el número equivocado. Un libro puede pasar todas las pruebas de una batería y abrirse lleno de errores.

No es una preocupación exótica: la literatura lleva décadas documentando que las fórmulas mal formadas son frecuentes y difíciles de detectar por inspección ([Panko 1998](#); [Abraham y Erwig 2006](#)), y que los mecanismos de detección que funcionan son los que ejecutan la hoja en vez de leerla ([Hermans, Pinzger, y Deursen 2012](#)). Una hoja de cálculo es un programa funcional, y como tal solo se valida ejecutándolo ([Sestoft 2014](#)).

8.5.2. El procedimiento

`materializador/verificar.py` implementa cuatro pasos:

1. Se genera el libro desde el plano, con `materializar.py`.

2. Se pasa por LibreOffice en modo desatendido (`soffice --headless --convert-to xlsx`), que lo abre, **recalcula** y lo vuelve a escribir.
3. Se relee con `load_workbook(..., data_only=True)`, que devuelve los **valores** y no las fórmulas.
4. Se comparan esos valores con los que produjo el **evaluador de referencia del núcleo**, serializado aparte como oráculo en `salida/<situacion>-esperado.json`.

El punto clave está en el paso 4. El oráculo no se escribió a mano: lo produce `nucleo/evaluador.lisp` a partir de la misma descripción, por un camino que no pasa por Excel en ningún momento. El emparejamiento de columna con campo se hace por nombre interno del campo, que el plano lleva en cada encabezado, y no por etiqueta visible, porque emparejar por etiqueta es adivinar y adivinar da falsos positivos: la etiqueta «Asignatura» casaría con el campo `asignadas`.

8.5.3. El resultado

```
ok    plan-del-grupo.xlsx: 30 celdas evaluadas por LibreOffice coinciden
      con el evaluador de referencia
ok    defensas-de-tesis.xlsx: 52 celdas evaluadas por LibreOffice coinciden
      con el evaluador de referencia
```

Treinta celdas en un libro y cincuenta y dos en el otro, evaluadas por LibreOffice, coinciden exactamente con el evaluador de referencia. Cero discrepancias.

8.5.4. Qué demuestra

Demuestra dos cosas a la vez, y conviene separarlas porque se confunden con facilidad.

Demuestra que **las fórmulas son válidas**: LibreOffice las abrió, las evaluó y produjo valores. No hay un `#REF!` ni un `#VALUE!` escondido en ninguna de las ochenta y dos celdas.

Y demuestra que **la arquitectura de Excel coincide con la semántica del lenguaje**, que es lo que ninguna comprobación de sintaxis puede dar. El evaluador del núcleo calculó **faltan** sin saber que existe Excel; la arquitectura de Excel emitió `=IF($A4="", "", ($C4-$D4))` sin saber que existe un evaluador. Que los dos coincidan en ochenta y dos celdas es evidencia de que la traducción preserva el significado. Si la arquitectura de Excel tuviera invertido el orden de los operandos de la resta, la comparación lo cazaría en la primera fila.

8.5.5. Qué se puede afirmar frente a los antecedentes

Con precisión, porque es fácil pasarse. **Ninguna de las tres tesis anteriores de la línea presenta verificación reproducible y automatizada de que las fórmulas que genera se evalúen.** (Cápiro Colomar 2025) no tiene capítulo de validación. (Arcia Corcho 2017) es una herramienta de construcción de generadores y no llega a este terreno. (Morales Lazo 2026) **sí afirma en sus conclusiones** que los archivos generados se evalúan correctamente en Excel y en LibreOffice Calc, y lo respalda con capturas de pantalla de valores ya calculados.

La diferencia no es que aquello fuera falso. Es que una captura la mira una persona una vez, y un script se vuelve a ejecutar cuando alguien cambia una línea, y falla solo. La afirmación defendible de este trabajo es la modesta: aporta el procedimiento reproducible, no el descubrimiento de que las fórmulas funcionan.

8.6. El caso de la figura 6.2

Éste es el experimento más ilustrativo, porque pone una diferencia de diseño frente a una consecuencia observable.

La descripción `corpus/plan-del-grupo.lisp` usa **a propósito los mismos seis renglones y los mismos valores** de la figura 6.2 de (Morales Lazo 2026), para que la comparación sea directa. En aquella figura, la fila con frecuencia 2 y tres turnos asignados aparece pintada de rojo, y la regla a la que los comentarios asignan el rojo es «asignadas menor que frecuencia», que con 3 y 2 es falsa. Las dos filas que sí cumplen esa regla salen de un azul que aquel trabajo no declara en ninguna parte.

En este lenguaje, la misma fila —AM, Análisis Matemático, frecuencia 2, asignadas 3— sale marcada como **excedida**, con severidad **problema** y el texto «Sobra 1 turno»; y las filas ICD y F salen como **insuficiente**, con «Falta 1 turno». Se ve en el artefacto de texto reproducido más arriba, en el relleno FFC7CE del plano de libro y en la clase CSS **problema** de la página.

Las razones son dos, y las dos son de diseño, no de cuidado al programar.

La primera: **la marca declara significado, no color**. En `corpus/plan-del-grupo.lisp` las tres marcas se llaman **insuficiente**, **excedida** y **completa**, y cada una declara una **:severidad** —advertencia, problema, informativa—. El color no aparece por ningún lado. Lo pone cada arquitectura a partir de la severidad: relleno en la hoja, clase CSS en la página, palabra en el texto.

La segunda: **la asignación no depende del orden en que estén escritas las reglas**. Cada marca lleva su propia condición en **:cuando**, que se evalúa sobre la fila. No hay correspondencia posicional entre el lugar de la regla en el texto y lo que le toca.

Declaración expresa sobre el alcance de esta comparación. La lectura de la figura 6.2 —que los colores se aplican en el orden de las reglas y no según el significado que los comentarios les atribuyen— **es interpretación propia de una imagen de un PDF, no una afirmación del documento de (Morales Lazo 2026), y queda pendiente de confirmar contra el código de aquel trabajo, regenerando el libro y examinando el archivo en vez de la figura**. Si el código lo confirma, el argumento se sostiene solo. Si no lo confirma, se retira, y lo que queda en pie no se resiente: que el color no esté en el nodo del lenguaje es un hecho de aquel documento y no depende de ninguna lectura de imagen.

Conviene no exagerar lo que el caso prueba. No prueba que este sistema sea correcto y el otro incorrecto. Prueba que aquí **ese error concreto no es expresable**, porque no hay ningún sitio donde escribir un color que pueda descolocarse respecto de un significado. La distancia entre lo que el usuario quiere decir y lo que el lenguaje le deja escribir es lo que (Mernik, Heering, y Sloane 2005) y (Voelter 2013) identifican como criterio de calidad de un

lenguaje de dominio específico, y (Ko et al. 2011) como origen de la mayoría de los errores de usuario final.

El segundo caso del corpus, `corpus/defensas-de-tesis.lisp`, da el resultado equivalente sobre una construcción más difícil: la marca `profesor-citado-dos-veces` usa un cuantificador existencial sobre la propia colección, y detecta que las dos citas del lunes a las 09:00 comparten a A. Suárez como secretario. Las dos salen marcadas en las tres arquitecturas.

8.7. Lo que NO está validado

Esta sección da credibilidad a todo lo anterior. Va sin adornos.

El experimento con un tercero no se ha hecho. La segunda mitad de la pregunta científica tiene prevista una prueba empírica concreta: que el tutor implemente la arquitectura de Emacs por su cuenta. No se ha ejecutado. Lo que hay es evidencia estructural —I2, I3, I4 y tres arquitecturas escritas contra el protocolo— que es necesaria pero no suficiente. Que tres arquitecturas escritas por el mismo autor no se vean entre sí no demuestra que el protocolo sea comprensible para alguien que no lo diseñó.

El corpus completo no está descrito. El corpus son tres documentos en uso. Hay dos descripciones, y una de ellas, `plan-del-grupo`, es una tabla de un generador y no el generador entero. El libro de gestión del departamento no está. La afirmación «el lenguaje cubre el corpus» no está probada; lo probado es que cubre dos casos elegidos por difíciles.

La arquitectura SQL no está implementada. El objetivo específico 5 habla de cuatro arquitecturas. Hay tres. `situacion.asd` no declara ningún sistema `situacion/sql`, aunque `i3-ninguna-arquitectura-ve-a-otra` ya lo contempla en su lista y lo comprobaría el día que entre. SQL importaba porque es la única con `con-derivacion-por-consulta` y sin `con-navegacion`.

La equivalencia con el generador de Python no está comprobada. El invariante I7 del documento de diseño —comparar semánticamente el libro generado con el que produce hoy el generador de Python— no está implementado. El *round-trip* comprueba que el libro coincide con el evaluador del propio sistema: eso es coherencia interna, no equivalencia con el artefacto que hoy usa la facultad.

No hay medición de esfuerzo ni comparación cuantitativa con la alternativa imperativa. No se midió cuántas líneas cuesta describir una situación frente a escribir el generador a mano, ni cuánto tarda alguien en añadir una arquitectura. Las dos descripciones son mucho más cortas que los generadores equivalentes, pero cubren menos, y comparar sin normalizar por cobertura sería propaganda. (Peyton Jones, Blackwell, y Burnett 2003) documenta lo fácil que es construir esa comparación de modo que favorezca al sistema propio.

El juego de pruebas de conformidad está diseñado pero no completo. Existe como suite ejecutable —ocho pruebas en `pruebas/conformidad.lisp`— y comprueba lo esencial: que las tres arquitecturas materialicen las dos descripciones, que ninguna capacidad pedida quede sin respuesta, que toda emulación o degradación traiga escrito en qué consiste, y que el modo estricto discrimine. Pero no hay **un caso por capacidad**, que es lo que el diseño pide.

I5, tal como está implementado, comprueba que alguna arquitectura declare cada capacidad, que es más débil. De las doce capacidades declaradas, los dos casos ejercitan ocho.

I6 no está implementado. Es I1 aplicado al dato en vez de al código: serializar las situaciones ya resueltas y buscar allí el vocabulario prohibido. Es justo por ahí por donde se coló el antecedente, cuyo árbol estaba limpio de nombre y cuyo JSON venía lleno de rangos absolutos. Mientras I6 no exista, la pureza está comprobada sobre los nombres del código y no sobre lo que el código produce.

Capítulo 9

Conclusiones

Se recorren los siete objetivos específicos, diciendo en cada caso si se cumplió y con qué evidencia.

1. Caracterizar el dominio. Cumplido parcialmente. El inventario de construcciones con evidencia de archivo y línea está en el capítulo 3 y sale de los tres libros en uso; las construcciones que ninguna taxonomía anterior cubre están identificadas y tienen forma en el lenguaje. Lo que falta es describir el tercer libro.

2. Declarar el modelo de evaluación y la clasificación por capacidades. Cumplido. El modelo está en `nucleo/evaluador.lisp` y las doce capacidades son clases del sistema, declaradas con `definir-capacidad` en `protocolo/capacidades.lisp`. La parte verificable —que cada capacidad tenga al menos un caso de conformidad— está cumplida a medias: hay suite, pero I5 comprueba cobertura por arquitectura y no por prueba.

3. Diseñar el lenguaje y su representación intermedia sin direccionamiento por encima del punto de extensión. Cumplido en la parte comprobada. I1 recorre los símbolos del núcleo y del protocolo con dos listas de prohibidas y pasa, y `i1-la-prueba-lexica-muerde` verifica que esa comprobación detecta lo que debe y acepta lo que debe. Queda sin implementar I6, la misma comprobación aplicada al dato.

4. Especificar el protocolo de arquitectura como contrato de extensión. Cumplido, y es donde la separación del antecedente es más nítida. Las trece operaciones están declaradas con `definir-operacion` (`protocolo/operacion.lisp`), que rechaza en tiempo de expansión cualquiera que no reciba la arquitectura como primer parámetro, y la prueba negativa lo comprueba contra la forma exacta de `compile-excel-formula`. El fallo de 2026 no es que no se haya repetido: es que no se puede repetir.

5. Implementar arquitecturas de naturaleza distinta. Cumplido en tres de cuatro. Texto plano, Excel y web están implementadas; I3 comprueba sobre el `.asd` que ninguna depende de otra ni del lenguaje, e I4 que el plano de libro no se escapa de la arquitectura que lo fabrica. SQL no está.

6. Definir el mecanismo de degradación y el informe de conformidad. Cumplido. Las cuatro respuestas están implementadas en `protocolo/carencia.lisp`, con la respuesta

por defecto en rechazar, y la política vive fuera de la arquitectura en ***politica***. Los seis informes se producen solos. La tabla completa de tres políticas por cuatro arquitecturas no se ejecutó: se ejecutó la permisiva sobre tres arquitecturas, más una prueba del modo estricto.

7. Validar contra el corpus y contra un tercero. Cumplido a medias, y es importante decir en qué mitad. El punto (b) está cumplido y es el resultado más fuerte del trabajo: las fórmulas se evalúan, con verificación reproducible por LibreOffice, ochenta y dos celdas sin una sola discrepancia. El punto (a), la equivalencia semántica con el generador de Python, no está. El punto (c), el experimento con el tutor, no se ha hecho.

9.1. Las tres contribuciones

Primera: subir la representación intermedia por encima del backend. En 2026 el generador compila hasta fórmulas de Excel antes de entregar, y la independencia del backend se afirma pero no existe. Aquí la representación intermedia está separada por invariantes que fallan solos, y el direccionamiento vive dentro de la arquitectura que lo necesita: el plano de libro, con sus **\$A\$4** y sus **OFFSET**, es una estructura interna de **situacion/excel** que ningún otro sistema importa.

Segunda: definir y hacer cumplir el mecanismo de extensión. Un protocolo de trece operaciones que despachan sobre la arquitectura, una jerarquía de doce capacidades componibles que permite especializar por propiedad en vez de por destino, y un mecanismo de degradación con cuatro respuestas y un informe que las registra. Lo que lo convierte en contribución y no en intención es que el sistema no admite la forma incorrecta. La idea de componer por propiedades la describe ([Arcia Corcho 2017](#)) y la atribuye a trabajos anteriores de la facultad; lo nuevo aquí es implementarla y usarla para decidir qué se degrada.

Tercera: acertar el núcleo semántico y demostrarlo con evidencia reproducible. El núcleo no es «una tabla»: son datos, derivaciones vivas, marcado con significado y entrada validada. La evidencia de que la separación es real es que la misma descripción produce tres artefactos de naturaleza distinta, y que la inversión de **con-crecimiento** —nativa en la web, emulada en la hoja de cálculo— aparece sola en los informes sin que nadie la haya puesto ahí. La evidencia de que lo producido además funciona es el *round-trip* por LibreOffice.

Capítulo 10

Recomendaciones

Completar el corpus. Describir el libro de gestión del departamento y las partes del generador de horarios que hoy quedan fuera. Es el trabajo más barato y el que más sube el valor de la afirmación «el lenguaje cubre el dominio». Conviene que la tercera descripción se escriba *antes* de tocar el lenguaje, para que sirva de prueba y no de confirmación.

Implementar la arquitectura SQL. No por completar un número, sino porque es la única de las cuatro previstas que tiene `con-derivacion-por-consulta` y no tiene `con-navegacion`. Una capacidad que ninguna arquitectura estresa es una capacidad cuya definición nadie ha puesto a prueba. Es además la arquitectura barata para ilustrar la degradación, porque rechazará cosas que las otras tres cumplen, y hoy la columna de rechazos está en cero en los seis informes.

Ejecutar el experimento con el tutor. Entregar el protocolo, el juego de conformidad y la documentación, y que implemente la arquitectura de Emacs sin acceso al núcleo y sin preguntar. Es la única prueba que responde la segunda mitad de la pregunta científica, y conviene diseñarla antes de ejecutarla: registrar qué se entrega, qué preguntas hace, cuántas veces necesita mirar el código de otra arquitectura, y si en algún momento tiene que modificar algo que no sea suyo. Un solo sujeto no es una muestra, pero es infinitamente más de lo que hay hoy. Los dos resultados son publicables: si lo logra, el mecanismo funciona; si no, el trabajo documenta dónde se rompió.

Completar la suite de conformidad. Añadir un caso mínimo por capacidad y reforzar I5 para que compruebe esa cobertura en vez de comprobar que alguna arquitectura declare cada capacidad. Conviene hacerlo antes del experimento anterior, porque forma parte de lo que hay que entregar.

Implementar I6 e I7. I6 —serializar las situaciones ya resueltas y buscar en esa serialización el vocabulario prohibido de I1— cierra el hueco por donde se coló el antecedente. I7 —comparar semánticamente el libro generado con el que produce hoy el generador de Python— es la prueba de equivalencia con el artefacto real, y la que convertiría el sistema en algo que se puede poner en producción y no solo defender.

Una sintaxis externa para quien no quiera escribir paréntesis. El lenguaje es interno

en Common Lisp ([Hudak 1996](#)): esa decisión ahorró los analizadores léxico y sintáctico y convirtió las macros en el compilador ([Graham 1993](#); [Seibel 2005](#)). El costo es que el usuario escribe sintaxis de Lisp, ventaja para el usuario real de este trabajo y barrera para cualquier otro. Una sintaxis externa que produzca el mismo árbol del núcleo es trabajo futuro razonable, con herramientas maduras ([Eysholdt y Behrens 2010](#); [Steinberg et al. 2008](#)), y no obliga a tocar nada por debajo: el núcleo, el protocolo y las tres arquitecturas seguirían intactos, que es precisamente lo que el diseño pretendía demostrar.

Medición cuantitativa. Medir con método declarado de antemano: líneas de descripción frente a líneas de generador para la misma cobertura, tiempo de incorporar una arquitectura nueva, defectos encontrados en un periodo de uso real. Sin normalizar por cobertura, cualquier comparación de tamaño es propaganda ([Peyton Jones, Blackwell, y Burnett 2003](#)). Exige describir el corpus completo primero, y por eso va la última.

Capítulo 11

El alcance del lenguaje

El capítulo anterior demostró que la misma descripción se materializa en tres arquitecturas y que las tres coinciden con el evaluador de referencia. Es una demostración de consistencia interna. No dice nada sobre la extensión del territorio.

Este capítulo contesta la pregunta que ninguna de las tres tesis de la línea se hace: **¿el lenguaje está limitado al corpus del que salió, o describe una clase más amplia de problemas?** ([Arcia Corcho 2017](#)) construye sobre ADOL* y GEMURAL y nunca sale de la planificación docente. ([Cápiro Colomar 2025](#)) describe su generador y no pregunta qué más podría generar. ([Morales Lazo 2026](#)) propone una taxonomía de construcciones tabulares y la ilustra únicamente con los documentos que ya tenía delante. En los tres casos, el dominio del trabajo y el conjunto de ejemplos del trabajo son el mismo conjunto, y eso hace que la pregunta del alcance no se pueda ni formular.

Todo lo que sigue se obtuvo ejecutando el sistema. El experimento se hizo el 2026-09-21 y sus informes completos están en `documentacion/tesis/alcance/`, uno por dominio. Las construcciones que salieron de él están en el código, y las pruebas que las cierran están en `situacion/pruebas/alcance.lisp`. Cuando algo es interpretación y no medición, se dice.

11.1. La pregunta y por qué no se puede contestar razonando

La afirmación tentadora es “el lenguaje describe cualquier situación tabular”. Esa afirmación es indefendible, y conviene ver por qué antes de intentar sustituirla.

Primero, porque “situación tabular” no es una clase definida. No hay una caracterización matemática de lo tabular que permita demostrar pertenencia. Cualquier cosa se puede poner en una tabla si se le hace suficiente violencia; el problema no es meterla, es que al meterla siga significando lo mismo.

Segundo, porque un lenguaje específico de dominio se define precisamente por su dominio, y el análisis de dominio es una actividad empírica, no deductiva ([Mernik, Heering, y Sloane 2005](#); [Czarnecki y Eisenecker 2000](#)). La literatura de ingeniería de dominios es consistente en esto: el alcance de un DSL se establece delimitando un conjunto de sistemas, no razonando

sobre un universal (Kelly y Tolvanen 2008; Voelter 2013). Un lenguaje que se justifica con “todo lo que sea X” y no declara qué es X está declarando que no hizo análisis de dominio.

Tercero, y es lo decisivo para este trabajo: el corpus del que salió el lenguaje es también su oráculo de validación. Las construcciones del álgebra se justificaron una por una señalando dónde aparecen en los tres libros de la facultad. Esa disciplina evitó la especulación, pero tiene un efecto de segundo orden incómodo: **un lenguaje validado contra el corpus que lo generó no puede sorprenderse a sí mismo**. La batería de pruebas comprueba lo que ya se sabe. Por construcción no puede encontrar lo que falta.

Queda entonces una sola cosa honesta que hacer: intentarlo con dominios que no tengan nada que ver, ejecutarlos de verdad, y reportar lo que pase. No demuestra el universal. Sí produce una frontera medida, que es lo que se puede defender.

11.2. El método

Se eligieron cinco dominios ajenos a la planificación académica. Para cada uno se escribió una descripción en el lenguaje y se ejecutó la cadena completa: análisis estático, evaluador de referencia, y materialización en las tres arquitecturas, con verificación del libro por LibreOffice y de la página ejecutando el JavaScript emitido.

Dominio	Archivo	Líneas
Control de inventario de un almacén	exploracion/inventario.lisp	480
Evaluación de un curso	exploracion/notas.lisp	245
Presupuesto y ejecución de un proyecto	exploracion/presupuesto.lisp	376
Torneo con eliminatorias	exploracion/torneo.lisp	731
Cuadrante de turnos de un equipo	exploracion/turnos.lisp	497

Dos de los cinco se eligieron **esperando que se rompieran**. El torneo es un árbol: la dependencia entre filas es intra-colección, transitiva y de profundidad arbitraria, y nada de eso aparece en el corpus. El cuadrante es una matriz, que es exactamente la forma de la hoja de grupo del generador de horarios —la parte del corpus que durante todo el análisis se había clasificado como “no encaja en el modelo”. Elegir casos con la expectativa de que fallen es lo contrario de elegir casos que luzcan bien, y es la única forma de que un resultado positivo signifique algo.

Interesa separar este método de la batería de pruebas del capítulo anterior, porque son dos formas distintas de validación y responden preguntas distintas.

Una batería de pruebas comprueba lo que ya se sabe. Cada caso codifica una expectativa que alguien escribió porque ya la tenía en la cabeza. Si la expectativa es incorrecta, la prueba pasa y el defecto sigue ahí. Las 34 pruebas y 464 comprobaciones del sistema no

encontraron ninguno de los tres fallos que se relatan más abajo, y no por descuido: los fallos estaban en construcciones que el corpus declara pero no ejercita, y la batería se escribió mirando el corpus.

Describir un dominio nuevo comprueba lo que no se sabe. El dominio impone sus preguntas antes de que nadie decida cuáles son razonables. Un almacén pregunta por existencias a fecha de corte; un curso pregunta por la media ponderada; un torneo pregunta quién pasa a la siguiente ronda. Ninguna de esas preguntas la habría escrito quien diseñó el álgebra, porque el corpus no se las hace. Es la diferencia entre verificar y explorar, y para un trabajo cuyo aporte es un lenguaje, la segunda es la que dice si el lenguaje existe como lenguaje o solo como generador con paréntesis.

El criterio de veredicto fue de tres valores:

- **CABE:** el dominio se expresa sin deformarlo.
- **CABE-A-MEDIAS:** se expresa, con rodeos identificables y documentados.
- **CABE-FORZADO:** se expresa solo a costa de meter en el documento tablas o columnas que en el problema no existen.
- **NO-CABE:** no se expresa.

11.3. Los cinco dominios

Dominio	Veredicto
Control de inventario de un almacén	CABE-FORZADO
Evaluación de un curso	CABE-A-MEDIAS
Presupuesto y ejecución de un proyecto	CABE-A-MEDIAS
Torneo con eliminatorias	CABE-A-MEDIAS
Cuadrante de turnos de un equipo	CABE-A-MEDIAS

Ninguno dio NO-CABE, y ninguno dio CABE limpio. **El lenguaje describe más de lo que se diseñó para describir, y menos de lo que haría falta para decir que describe cualquier situación tabular.** Ésa es la respuesta medida.

11.3.1. Control de inventario de un almacén — CABE-FORZADO

Se expresaron artículos con clave y stock mínimo, un libro de movimientos que crece, existencias como resta de dos agregados con filtro compuesto, marcas de bajo mínimo y agotado con concordancia de plural, imputación validada contra el listado de artículos, y autocompletado del nombre por búsqueda de clave.

La sorpresa positiva fue el kardex. El saldo por artículo en el momento de cada movimiento —la pregunta central de un almacén— **sí es expresable**, con un agregado auto-referente sobre la misma colección:

```
(campo saldo :rol derivado
  (suma firmada de movimientos
    :donde (y (= articulo (de fila articulo))
      (<= dia (de fila dia))))))
```

El veredicto es CABE-FORZADO y no CABE-A-MEDIAS por una razón concreta: la deformación se ve en el artefacto, no solo en la descripción. “Entrada o salida” son dos palabras del dominio y hubo que declararlas como una colección de una columna y dos filas, que en el almacén no existe y que aparece como una pestaña más del libro generado. Y “una fila por proveedor” no se podía decir: toda colección era `:origen :declarada`, así que había que enumerar los proveedores a mano y mantener esa lista sincronizada con lo que escribiera el operario.

El dominio dejó una lección de forma más general. Los tres libros de la facultad tienen todas sus tablas dadas de antemano. Un almacén tiene una tabla dada y una que crece sin forma conocida, y sus preguntas interesantes son *agrupamientos de la que crece*. Es la diferencia entre un lenguaje de documentos y un lenguaje de registros.

11.3.2. Evaluación de un curso — CABE-A-MEDIAS

Se expresaron tres colecciones con clave compuesta, el peso traído por clave con su guarda de ausencia, la media ponderada, el umbral como parámetro, la marca de suspenso con explicación en palabras, y cuatro vistas. La hoja recalculada por LibreOffice dio 88, 52.75, 63.5 y 53.5, exactamente lo que dijeron el evaluador de referencia y el JavaScript de la página.

Los rodeos fueron dos, y los dos instructivos. El primero: **no se puede sumar una expresión, solo un campo**. La media ponderada es literalmente sumar `nota * peso` cruzando dos claves, y el nodo **agregado** guarda un nombre de campo, no una expresión. Hubo que materializar el producto como columna derivada intermedia. Cabe, y cabe bien, pero el documento acaba con dos columnas que el profesor no pidió.

El segundo: **no hay dónde poner un valor del curso entero**. “Cuántos estudiantes aprobaron” no es un valor de ninguna fila, y hubo que fabricar una colección de una sola fila que en la hoja aparece como una pestaña con una tabla de una fila y cuatro columnas.

Este dominio también destapó algo que no es limitación del lenguaje sino defecto del sistema: al dividir, el evaluador devolvía racionales exactos de Common Lisp y los imprimía como 211/4, lo que además producía JSON inválido y tumbaba la cadena de verificación antes de empezar. El corpus no divide nunca, y por eso el lenguaje no había tenido que decidir qué significa `:tipo numero`.

11.3.3. Presupuesto y ejecución de un proyecto — CABE-A-MEDIAS

Partidas y gastos, imputación validada sin bloquear, gasto por partida, lo que queda, el porcentaje ejecutado con guarda de división por cero, y dos marcas con severidad distinta y umbral parametrizado.

El resultado que no se esperaba: **la jerarquía se expresa y hace falta recursión, y funcionó sin añadir nada al lenguaje**. Un campo derivado que se suma a sí mismo en las filas hijas:

```
(campo gastado :rol derivado :tipo numero
  (+ (de fila gasto-directo)
    (suma gastado de partidas :donde (= padre (de fila codigo)))))
```

El análisis lo acepta porque **comprobar-ciclos** solo mira dependencias dentro de la misma fila y ésta pasa por un agregado; el evaluador lo resuelve a tres niveles por memorización perezosa. Y la hoja de cálculo también: la fórmula emitida es un **SUMIF** cuyo rango de suma contiene la propia celda, y LibreOffice la resuelve sin marcar referencia circular, porque detecta ciclos por reentrada en tiempo de evaluación y no por solape estático de rangos. Es un éxito medido, pero un éxito que depende de cómo Calc granula su grafo de dependencias, y no se pudo comprobar con Excel de Microsoft. Queda como riesgo abierto, no como resultado.

Lo que no cupo: el total general —el número que el jefe mira primero— no tiene dónde vivir, porque el modelo solo tiene campos, parámetros y marcas. Repetirlo en cada fila imprime 18600 seis veces. Y el lenguaje no sabe que **padre** apunta a **codigo** de su propia colección, así que con datos cíclicos el análisis pasa limpio y el error llega en tiempo de evaluación con datos reales, que es el peor momento posible.

11.3.4. Torneo con eliminatorias — CABE-A-MEDIAS

Era uno de los dos casos elegidos esperando que se rompiera, y dio el hallazgo más interesante del experimento.

Un árbol de eliminatorias sí se puede tabular. El cuadro se describe como una colección de partidos con un campo que dice de qué partido viene cada equipo, y la arista del árbol es un campo fijo:

```
(campo equipo-a :rol derivado :etiqueta "Equipo A"
  (si (= (de fila viene-a) 0)
    (de fila sembrado-a)
    (el ganador de (la-fila-de partidos
      :donde (= id (de fila viene-a)))
      :si-no "")))
```

Se lee en voz alta: “el ganador del partido del que viene A, y si no lo hay, nada”. La búsqueda es la misma que ya servía para el **VLOOKUP** del corpus.

Funcionó a cuatro niveles y después a seis: un cuadro de 64 equipos, 63 partidos, **693 celdas evaluadas por LibreOffice, cero discrepancias**. Contra la hipótesis inicial, la referencia circular no se produce: el **VLOOKUP** emitido abarca la propia celda y Calc lo resuelve igual. La cascada también corrió sin límite estructural en el evaluador de referencia.

Lo que no se puede es **calcular la forma del árbol**. Con la numeración canónica de un cuadro, el partido padre es la parte entera de $i/2$. El álgebra tiene las cuatro operaciones, las comparaciones y los conectores, y nada más: no hay parte entera ni resto. Consecuencia

concreta: cada arista hay que escribirla a mano como dato. 30 números para 16 equipos, 126 para 64. **El lenguaje no puede decir cómo es el cuadro; solo puede listarlo.** Es la diferencia entre describir una estructura y transcribirla, y es el límite honesto del veredicto.

Tampoco se puede dibujar. Un cuadro de eliminatorias es sobre todo una figura, y lo que se lee de él es qué partido alimenta a cuál. El modelo de vista sabe partir y agrupar filas; no tiene forma de expresar que dos filas están conectadas.

11.3.5. Cuadrante de turnos de un equipo — CABA-A-MEDIAS

El otro caso elegido para que fallara, porque es la forma de la hoja de grupo. Se probaron dos representaciones: (a) una fila por celda del cuadrante, y (b) una fila por día con una columna por turno.

Las tres reglas del dominio se expresan, y se expresan bien, en las dos. La más interesante es “quien hace la noche no entra de mañana al día siguiente”. En (b) sale con (de (anterior fila) noche) y se lee exactamente como la frase del dominio. En (a) sale sumando uno al día, que es lo que la frase *significa*:

```
(existe otra :en asignaciones
  :donde (y (= (de otra turno) "manana")
             (= (de otra dia) (+ (de fila dia) 1))
             (= (de otra persona) (de fila persona))))
```

El resultado que importa: **la representación que no usa vecino es la que sobrevive a las tres arquitecturas.** La que lo usa depende de que el orden declarado signifique algo que el lenguaje no garantiza, y es precisamente el PREVIOUS-OF posicional que (Morales Lazo 2026) tiene y que este análisis existe para impedir.

Lo que no cupo fue la **forma**: el cuadrante como rejilla día × turno no existía en el lenguaje. La representación (b) parece una matriz solo porque uno escribe tres columnas a mano; en cuanto los turnos dejan de ser tres fijos, deja de haber cuadrante. Ése fue el hallazgo central del dominio, y es el que se resolvió después. Se trata aparte, porque es la parte más importante del capítulo.

11.4. Lo que el experimento rompió

Esta sección es la justificación metodológica de todo el ejercicio. Intentar describir cosas nuevas encontró **tres fallos reales en el sistema**, y ninguno de los tres lo habían encontrado las pruebas.

1. La arquitectura web recalculaba en una sola pasada. `recalcularTodo()` en `web/recursos.lisp` recorría cada colección una vez, en el orden del arreglo, con los derivados calculados con `avidez`. Con derivaciones que dependen de otras filas de la misma colección —el ganador de un partido pasa al siguiente, el saldo de un renglón se arrastra del anterior— **cada render hacía converger exactamente un nivel de la cadena.** Medido con el cuadro de eliminatorias:

```

pasada 1: 1:-/- 2:-/- 3:-/- 4:-/- ... 8:Alemania/Escocia ...
pasada 2: 1:-/- 2:-/- 3:-/- 4:Alemania/Suiza 5:Espana/Italia ...
pasada 3: 1:-/- 2:Alemania/Espana 3:Países Bajos/Inglaterra ...
pasada 4: 1:Espana/Países Bajos ... (converge)

```

La página no avisaba de nada: devolvía cadena vacía donde debía ir un equipo. Y el informe de conformidad, que es la evidencia central de este trabajo, decía **“5 nativas, 0 emuladas, 0 degradadas, 0 rechazadas”**. Un verde falso. Es exactamente la clase de fallo que este trabajo le reprocha a (Morales Lazo 2026): el documento afirma algo que no cumple, y lo afirma en el sitio donde el lector confía. Un informe de conformidad que solo sabe mentir bonito es peor que no tener informe.

Corregido. `web/recursos.lisp` recalcula hasta punto fijo, comparando una huella del estado entre pasadas, con un tope de 32 para que una dependencia circular no cuelgue el navegador. Si se alcanza el tope, `RECALCULO_INCOMPLETO` queda en verdadero y **la página lo dice**. El comentario del código lo deja escrito: callar sería volver al verde falso.

2. `(:orden dia turno)` descartaba el segundo campo en silencio. `compile-coleccion` hacía `(setf orden (normalizar (second f)))` y tiraba el resto. La descripción se leía como “ordenado por día y turno” y significaba “ordenado por día”.

3. `(de (anterior fila :donde ...) campo)` descartaba el `:donde` en silencio. `compile-acceso` leía `(first origen)` y `(second origen)` y tiraba lo demás. El saldo corriente de un kardex es “lo del renglón anterior *del mismo artículo*”; escrito así, el sistema compilaba sin protestar, pasaba el análisis, y daba números mal.

Las tres son del mismo género. No son caídas ni excepciones. Son descripciones que se leen como correctas y significan otra cosa. Es el género de fallo que la literatura de hojas de cálculo lleva décadas documentando como el más costoso y el más difícil de detectar, precisamente porque el artefacto se ve bien (Panko 1998; Hermans, Pinzger, y Deursen 2012; Abraham y Erwig 2006). Y es el género que este trabajo eligió como enemigo declarado.

Las dos últimas se cerraron rechazando la forma con un mensaje que dice si falta una construcción o si no encaja en el modelo:

```

(:orden dia turno) declara mas de un campo, y el orden solo admite uno.
Si de verdad hace falta ordenar por varios, no es un problema de sintaxis:
es que al lenguaje le falta el orden compuesto.

```

```

ANTERIOR y SIGUIENTE recorren la coleccion entera en el orden que declara,
y no saben particionar por ningun campo. Si hace falta "el anterior del
mismo grupo", al lenguaje le falta esa construccion; no es un problema de
sintaxis.

```

Esa distinción —falta una construcción / no encaja en el modelo— es la misma que organiza el resto del capítulo, y ponerla en el mensaje de error significa que el lenguaje sabe decir dónde está su propia frontera. La prueba `alcance-nada-se-descarta-en-silencio` comprueba que las dos formas se rechazan.

Tres fallos en una noche, encontrados por un método que la batería no podía ejecutar. Es el argumento a favor del método, y vale más que cualquiera de los tres arreglos.

11.5. Los nueve huecos y cómo se cerraron

Los cinco informes coincidieron en una lista de carencias. Cada una se clasificó como **falta una construcción** (el modelo la admite, solo hay que escribirla) o **no encaja en el modelo** (hay que cambiar una decisión estructural). La clasificación importa porque determina el costo, y el costo determina si se hace.

`situacion/pruebas/alcance.lisp` tiene una prueba por hueco, y cada una dice de cuál de los cinco dominios salió. Las nueve pasan.

#	Hueco	Clasificación	Qué se hizo
1	Enumeración literal como dominio	Falta una construcción	Nodo <code>enumeracion</code> , <code>syntax</code> (<code>uno-de "entrada" "salida"</code>)
2	Fechas comparables por su valor	Falta una construcción	<code>fecha-como-numero</code> en el núcleo, día juliano
3	Horas comparables por su valor	Falta una construcción	<code>hora-como-numero</code> , minutos desde medianoche
4	Agregados con varios criterios en la hoja	Arquitectura incompleta	<code>COUNTIFS</code> y <code>SUMIFS</code> en <code>excel/formula.lisp</code>
5	Agrupación: una fila por valor distinto	No encaja en el modelo	(<code>:una-fila-por campo :de coleccion</code>)
6	Resumen de una sola fila	No encaja en el modelo	(<code>:una-sola-fila</code>)
7	Concordancia con el sustantivo de los datos	Falta una construcción	<code>singular</code> y <code>plural</code> pasan a ser expresiones
8	Relación uno a muchos	Falta una construcción	Nodo <code>relacionadas</code> , (<code>las-filas-de ... :donde ...</code>)
9	La tabla cruzada	No encaja en el modelo	Tres ramuras en el nodo <code>vista</code>

Cuatro observaciones sobre esa tabla.

El hueco 4 no era del lenguaje. La condición más normal de todas —dos igualdades

unidas por y— tumbaba la materialización entera con un **error** de Lisp crudo, fuera del mecanismo de carencias: sin informe, sin **:rechaza**, sin capacidad que nombrar. El emisor de Excel implementaba exactamente las formas del corpus y ni una más. Hoy **criterios-de** descompone la condición en pares de campo y criterio, reconoce además “está escrito” y “está vacío”, y emite la variante en plural. Y cuando no puede, no emite nada: *“No se emite nada antes que emitir una fórmula que parezca correcta y no lo sea.”* El hueco enseñó algo más general: el corpus, como oráculo, mide la cobertura de la sintaxis declarada mucho peor de lo que parece.

Los huecos 2 y 3 son coerción, no un tipo de fecha. `nucleo/evaluador.lisp` lo dice explícitamente: no hay aritmética de calendario, y sumar un día a una fecha sigue sin poder decirse. Lo que se ganó es que “existencias a fecha de corte” y “turnos antes de las diez” se puedan escribir, que son las dos preguntas más normales de media docena de dominios.

Los huecos 5 y 6 comparten mecanismo. Los dos son la ranura **origen** del nodo `coleccion`, que el modelo ya documentaba y que `compilar-coleccion` nunca llenaba: escribía siempre `:origen :declarada`. Era una forma prevista y no existente. Hoy `filas-derivadas` produce una fila por valor distinto —en el orden de aparición, sin vacíos y sin repetir— o exactamente una fila sin campos escritos. El almacén dejó de necesitar un listado de proveedores mantenido a mano; el presupuesto dejó de necesitar una fila fantasma suministrada en los datos de ejecución, que si se olvidaba hacía desaparecer el total general en silencio.

El hueco 9 merece su propia sección.

Vale la pena medir el costo de una de estas extensiones, porque es el argumento sobre el mecanismo. Añadir el nodo `enumeracion` —el hueco 1— tocó exactamente seis archivos: `nucleo/expresion.lisp` (el nodo), `nucleo/paquete.lisp` (su exportación), `lenguaje/expresiones.lisp` (la sintaxis), `nucleo/evaluador.lisp` (la semántica de referencia), y `excel/formula.lisp` y `web/expresiones.lisp` (un método por arquitectura que emite código). La arquitectura de texto no necesitó nada, porque congela valores ya calculados por el evaluador. Seis archivos, ninguno de ellos el protocolo, y el crecimiento es lineal en el número de arquitecturas.

11.6. La tabla cruzada: de caro a barato

Ésta es la parte más importante del capítulo, y es un razonamiento más que un resultado.

El punto de partida. La hoja de grupo del generador de horarios —`horarios/hoja_grupo.py`— no es una lista de filas: es una rejilla de día por turno. Durante todo el análisis del dominio se clasificó como “no encaja en el modelo”, y por eso se dejó fuera del corpus descrito. Era la parte del propio problema de la facultad que el lenguaje no alcanzaba. El informe del cuadrante de turnos lo confirmó desde el otro lado: lo caro no era dibujar la rejilla, era que **el número de columnas depende del contenido y no de la declaración**, y todo el direccionamiento de la arquitectura de hoja de cálculo parte de que cada campo tiene su letra, fijada al planificar. Un eje que sale de los datos mueve el plan, el direccionamiento, los rangos de todo agregado y el alcance de toda marca.

La salida. Fue darse cuenta de que **la matriz nunca fue una estructura de datos**. Los datos son, y siempre fueron, una colección de filas: día, turno, asignatura, aula. Lo que es matriz es la **presentación**. Y la presentación es una vista.

Esa distinción no es un truco para esquivar el problema. Es exactamente el mismo criterio que el lenguaje aplica en todas partes: **la intención se declara, el mecanismo lo inventa cada arquitectura**. Una marca declara un significado y una severidad, nunca un color. Una colección declara que crece, nunca cuántas filas se reservan. Una vista declara que hay un eje de filas, un eje de columnas y algo en el cruce, nunca cómo se dibuja. El comentario de `nucleo/modelo.lisp` lo deja escrito, y es la frase que resume el capítulo:

LA TABLA CRUZADA ES UNA VISTA, NO UNA ESTRUCTURA DE DATOS.

[...] Meter matrices en el modelo de datos habría tocado el núcleo, el análisis y las tres arquitecturas; como vista, son tres ranuras y un método por destino.

Ése es el cambio de costo. De “toca el núcleo y las tres arquitecturas” a “tres ranuras en el nodo de vista y un método por destino”: `eje-de-filas`, `eje-de-columnas`, `lo-que-se-muestra`, y un predicado `cruzada-p` que es verdadero cuando las tres están.

La declaración real. Con eso, la parte del corpus que no cabía cabe. `situacion/corpus/horario-del-grupo.lisp` declara la rejilla como lo que es —una fila por casilla— y después declara cómo se ve:

```
(coleccion casillas
  (:clave dia turno)
  (:orden turno)
  (campo dia :rol fijo :etiqueta "Dia")
  (campo turno :rol fijo :etiqueta "Turno")
  (campo asignatura :rol entrada
    :dominio (los abrev de asignaturas) :al-violar advertir)
  ...)

(vista rejilla :de casillas :entrada t
  :etiqueta "Horario semanal"
  :filas turno :columnas dia :muestra asignatura)
```

Ni una coordenada. Ni una letra de columna. La vista dice qué va en cada eje y qué aparece en el cruce, y nada más.

Lo que hace cada arquitectura con esa misma declaración.

Texto plano dibuja la rejilla alineada. Es lo único que sabe hacer bien, y lo declara como capacidad nativa. La salida real:

Horario semanal

Turno	lunes	martes	miercoles
-----	-----	-----	-----
T1	AL	AL	AL
T2	L	IP	AL
T3		AM	

La página la recalcula al vuelo. distintosDe lee los ejes de los datos cada vez que dibuja, así que **si mañana aparece un valor nuevo, aparece una columna nueva sola**. No cuesta nada, y por eso la web declara `con-tabla-cruzada` como nativa.

La hoja de cálculo la *emula*. Fija los dos ejes al generar, leyendo los valores que hay, y se inventa una columna de clave compuesta en la hoja de origen. La razón es concreta: `BUSCARV` busca por un criterio, y un cruce necesita dos. La columna auxiliar concatena los dos ejes y se busca con `INDICE` y `COINCIDIR`, que es la pareja portable entre Excel y LibreOffice Calc (*ECMA-376: Office Open XML File Formats* 2016; Sestoft 2014). Lo emitido de verdad para el horario del grupo:

```
Casillas!H4      =$B4&"|"&$A4
Horario semanal!B4
    =IFERROR(INDEX(Casillas!$C$4:$C$12,
                  MATCH($A4&"|"&$B$3,Casillas!$H$4:$H$12,0)), "")
```

La columna H de la hoja Casillas aparece con el encabezado (`clave`). Es maquinaria que el lenguaje no nombra: la inventa esa arquitectura porque no tiene búsqueda por clave compuesta. En una página no haría ninguna falta.

El renglón real del informe de conformidad de Excel, con su límite declarado:

```
emula    con-tabla-cruzada    los dos ejes se leen de los datos y se fijan
        AL GENERAR, y se anade a la hoja de origen una columna de clave
        compuesta para poder buscar por dos criterios con INDICE y
        COINCIDIR, que es la pareja portable entre Excel y LibreOffice
        Calc.
        Limite: si manana aparece un valor nuevo en cualquiera de los
        dos ejes, no aparece una fila ni una columna para el; hay que
        volver a generar el libro.
```

Ese límite es el resultado, no una disculpa. La arquitectura con rejilla es la que peor lleva la rejilla, y lo dice. Es otra instancia de la inversión de capacidades que el capítulo anterior documenta.

Los informes completos del caso nuevo:

Arquitectura	Nativas	Emuladas	Degradadas	Rechazadas
texto	4	1	4	0
excel	8	1	0	0
web	9	0	0	0

Y el resultado medido. `horario-del-grupo.xlsx`: **15 celdas de la rejilla evaluadas por LibreOffice coinciden con el evaluador de referencia**. La página del mismo caso da 105 valores coincidentes. El caso entró en `./demostrar.sh` junto a los dos anteriores, y las seis verificaciones de ejecución —tres libros por LibreOffice, tres páginas ejecutando el JavaScript emitido— dan cero discrepancias.

11.7. Qué sigue sin caber, y por qué se deja fuera

Lo que sigue no cabe. Se enumera con su clasificación porque una frontera que no se declara no es una frontera.

La recursión y el cierre transitivo. “Los subordinados de alguien, recursivamente”, “todas las subpartidas a cualquier profundidad”. *No encaja en el modelo*. El caso del presupuesto funcionó a tres niveles por accidente afortunado: el evaluador memoriza perezosamente y Calc granula su grafo de dependencias de una forma que lo tolera. Pero el lenguaje no sabe que **padre** apunta a **codigo** de su propia colección, así que no puede exigir aciclicidad ni planificar un orden de cálculo, y con datos cíclicos el error aparece en tiempo de evaluación en vez de en el análisis.

Los grafos de dependencias. Un cronograma con precedencias, una red de tareas. *No encaja en el modelo*. Es la generalización del anterior, y arrastra lo mismo: un orden de cómputo que no es el orden de lectura. El núcleo prohibió las coordenadas a propósito para no dejar entrar el modelo de la rejilla, y con ellas se fue también la noción de **precedencia**, que no es una coordenada. Es la tensión más interesante del diseño y no está resuelta.

Los documentos de forma libre. Un informe con párrafos, un acta con narrativa. *No encaja en el modelo*, y además por decisión. El lenguaje describe situaciones tabulares. Un documento de prosa no es una situación tabular, y aceptarlo sería cambiar el dominio, no extender el lenguaje.

Cuantificar sobre campos y no solo sobre filas. “Ningún turno del día puede quedar vacío” es una sola frase del dominio, y en la representación (b) del cuadrante son tres. *No encaja en el modelo*. **existe**, **cuantas**, **los** y **la-fila-de** recorren filas; no hay ligador de columnas. **comparten** es lo único que recibe una lista de campos, y es fija en tiempo de escritura: es la versión congelada del cuantificador, no el cuantificador.

Una relación declarada entre colecciones. Algo como **:pertenece-a**, que diga que las asignaciones son la sub-tabla de los días. *Falta una construcción*, y de las que más se echan de menos: sin ella, dos colecciones solo se juntan repitiendo la igualdad dentro de cada expresión, y ninguna arquitectura puede saber que una es el detalle de la otra para anidarlas.

Y la razón de fondo para dejarlas fuera. El riesgo más común de un trabajo así no es que el lenguaje se quede corto. Es que **crezca sin control**. Es el fracaso clásico del diseño de lenguajes específicos de dominio: la presión de cada caso nuevo empuja hacia añadir una construcción más, hasta que el lenguaje deja de tener un dominio y pasa a ser un lenguaje de propósito general con peor sintaxis (Mernik, Heering, y Sloane 2005; Fowler 2010). Aquí ese riesgo tiene una forma cuantificable: **cada construcción nueva multiplica el trabajo por el número de arquitecturas**. Seis archivos hoy con tres destinos; ocho con cuatro. Una construcción que no esté justificada por un caso real del dominio es deuda que se paga en cada arquitectura futura, incluida la que escriba un tercero.

Por eso los nueve huecos que se cerraron salieron todos de dominios ejecutados, y los cinco que quedan se documentan en vez de implementarse.

11.8. Cómo enunciar el alcance en una defensa

Hay dos formulaciones posibles, y la diferencia entre ellas es la diferencia entre una afirmación que no se sostiene y una que sí.

La indefendible: *“El lenguaje describe cualquier situación tabular.”*

Se cae con una sola pregunta del tribunal. No hay definición de “cualquier situación tabular”, así que la afirmación no es ni verdadera ni falsa: es inverificable. Y el trabajo tiene cinco informes que muestran justo lo contrario: cinco dominios, cinco veredictos que no son CABE limpio, y catorce carencias identificadas, de las cuales cinco siguen abiertas. Afirmar el universal sería además incoherente con el propio método, que consistió en buscar dónde se rompía.

La defendible: *“El lenguaje describe una clase de situaciones tabulares que está caracterizada: colecciones de filas homogéneas con campos fijos, de entrada y derivados; derivación por fila con acceso a otras colecciones por clave, a la propia colección por agregado, y al vecino en un orden declarado; restricción de dominio sobre la entrada; marcas por fila con significado y severidad; presentación como lista o como tabla cruzada. Cinco dominios ajenos al corpus se describieron y se ejecutaron dentro de esa clase. Lo que queda fuera está enumerado, clasificado y documentado.”*

Ésa se defiende, porque cada cláusula corresponde a un nodo del núcleo, a una capacidad del protocolo o a una prueba que pasa. Y tiene tres partes que se sostienen por separado.

Primero: el lenguaje describe una clase caracterizada. 24 nodos en el árbol del núcleo, 15 capacidades componibles, 13 operaciones en el protocolo. No es una promesa: es un inventario.

Segundo: la clase está caracterizada. Y lo está por los dos lados. Por dentro con las construcciones que existen; por fuera con los cinco huecos que quedan abiertos, cada uno clasificado como “falta una construcción” o “no encaja en el modelo”. El lenguaje incluso sabe decirlo en tiempo de compilación: los mensajes de rechazo de (`:orden a b`) y de (`anterior fila :donde ...`) distinguen las dos cosas. Un lenguaje que puede nombrar lo que no sabe hacer es un lenguaje que hizo análisis de dominio (Czarnecki y Eisenecker 2000; Kelly y Tolvanen 2008).

Tercero, y es lo que de verdad se tiene: la generalidad no vive en el vocabulario, vive en el mecanismo de extensión. El aporte que sobrevive no es la lista de construcciones —esa lista crecerá— sino que añadir una tenga un costo medido y acotado. Está medido: **seis archivos** —el nodo, su exportación, la sintaxis, la semántica de referencia y un método por arquitectura que emite código—, ninguno de ellos el protocolo, y lineal en el número de arquitecturas. Ese mecanismo es el protocolo de metaobjetos aplicado a la generación de documentos (Kiczales, Rivières, y Bobrow 1991), construido con macros que hacen de compilador (Graham 1993; Seibel 2005) sobre un lenguaje interno (Hudak 1996). Es lo que ninguna de las tres tesis previas tiene, y es lo que hace que la frontera medida en este capítulo sea una frontera de hoy y no una frontera del diseño.

La tesis que de verdad se tiene, entonces, no es “el lenguaje lo describe todo”. Es: **el lenguaje describe una clase que se sabe cuál es, se midió dónde termina, y extenderla cuesta seis archivos.** Es menos vistoso y es infinitamente más defendible.

Capítulo 12

Apéndice: el sistema después del experimento de alcance

12.1. Por qué existe este apéndice

Los capítulos 4, 5 y 6 describen el sistema tal como estaba el día en que se escribieron. Después se hizo el experimento de alcance que documenta el capítulo 8: cinco dominios ajenos al corpus, descritos en el lenguaje y ejecutados contra las tres arquitecturas. El experimento encontró huecos. El sistema creció en una sesión para cerrar los que eran baratos, y los capítulos quedaron desfasados en números y en afirmaciones.

Reescribirlos habría producido un documento coherente y sin rastro. El rastro es justamente lo que un trabajo así tiene que poder enseñar: qué se creyó terminado, qué lo desmintió y qué costó corregirlo. Se dejan los capítulos como están y se pone aquí la errata.

Todos los números de este apéndice se midieron el 21 de septiembre de 2026 con `wc -l`, `grep` y la ejecución de `./ejecutar-pruebas.sh` y `./demostrar.sh`. Ninguno se copió de otro documento.

12.2. Recuentos

Magnitud	Lo que dicen los capítulos	Medido
Líneas de Lisp	5 392 en 39 archivos (cap. 6)	6 736 en 46 archivos
Líneas de Python	387	387
Líneas de JavaScript	109	109
Operaciones del protocolo	13	13
Operaciones sin ningún método	4 (cap. 5)	0
Capacidades componibles	12	15

Magnitud	Lo que dicen los capítulos	Medido
Clases de nodo del árbol	15 del álgebra (cap. 4 y 6)	24 <code>defnodo</code> : 18 en <code>nucleo/expresion.lisp</code> , 6 en <code>nucleo/modelo.lisp</code>
Formas de expresión escritas	16 (cap. 4)	18
Descripciones del corpus	2	3
Pruebas	no se dice	34
Comprobaciones	no se dice	464, cero fallidas
Verificaciones de ejecución	4 (cap. 6)	6, cero discrepancias
Archivo mayor	397 líneas, el evaluador (cap. 5)	510, el mismo archivo

De las 6 736 líneas de Lisp (mas 2 329 del directorio de exploracion, que no es parte de ningun sistema), 6 736 están en los 41 archivos del sistema y 2 329 en `situacion/exploracion/`, que son las cinco descripciones del experimento de alcance y no forman parte de ningún sistema ASDF.

Por sistema, contra la tabla del capítulo 6:

Sistema	Cap. 6	Medido
<code>situacion/nucleo</code>	6 / 918	6 / 1 097
<code>situacion/lenguaje</code>	3 / 488	3 / 540
<code>situacion/analisis</code>	2 / 322	2 / 361
<code>situacion/protocolo</code>	7 / 741	7 / 799
<code>situacion/texto</code>	2 / 209	2 / 312
<code>situacion/excel</code>	5 / 834	5 / 1 161
<code>situacion/web</code>	4 / 672	4 / 853
<code>situacion/corpus</code>	3 / 241	4 / 402
<code>situacion/pruebas</code>	6 / 862	7 / 1 103
<code>situacion/demostracion</code>	1 / 105	1 / 108

Dos erratas menores del mismo tipo. El capítulo 6 dice «diez sistemas ASDF»; `situacion.asd` declara once `defsystem`. Y el capítulo 5 usa las 397 líneas del evaluador como señal del criterio C6; hoy son 510, y la señal debe volver a leerse.

Las seis verificaciones de ejecución, con su salida literal:

<code>plan-del-grupo.xlsx</code>	30 celdas evaluadas por LibreOffice
<code>defensas-de-tesis.xlsx</code>	52 celdas evaluadas por LibreOffice
<code>horario-del-grupo.xlsx</code>	15 celdas evaluadas por LibreOffice
<code>plan-del-grupo.html</code>	48 valores calculados por el JavaScript emitido
<code>defensas-de-tesis.html</code>	60 valores calculados por el JavaScript emitido
<code>horario-del-grupo.html</code>	105 valores calculados por el JavaScript emitido

El caso nuevo es `corpus/horario-del-grupo.lisp`, la hoja de grupo del generador de horarios, que durante todo el análisis se clasificó como «no encaja en el modelo». Sus tres informes de conformidad, medidos: texto, 4 nativas, 1 emulada, 4 degradadas, 0 rechazadas; Excel, 8 nativas, 1 emulada, 0 degradadas; web, 9 nativas y nada más.

12.3. Construcciones del álgebra que los capítulos no recogen

La tabla del capítulo 4 lista dieciséis formas y quince clases de nodo, y declara la lista «cerrada y completa». Hoy `lenguaje/expresiones.lisp`, en `compilar-forma-compuesta`, admite dos formas más, y `nucleo/expresion.lisp` tiene dos `defnodo` más.

(uno-de "a" "b" ...), **también** (alguno-de ...) — **nodo enumeracion**. Un conjunto escrito a mano, para usar como `:dominio` de un campo de entrada. Existe porque sin él «entrada o salida» obliga a declarar una colección de una columna y dos filas que en el dominio no existe, y que acaba apareciendo como una pestaña más del libro generado. El nodo no tiene subexpresiones.

(las-filas-de COLECCION :donde CONDICION) — **nodo relacionadas**. Todas las filas de otra colección que se relacionan con ésta: el detalle de carga de un profesor, los movimientos de un artículo. Lo compila `compilar-relacionadas`, que exige el `:donde` y liga la variable candidata anónima, igual que `la-fila-de`. El capítulo 4 afirma que esta forma «**no está implementada**» y que «ninguna descripción puede pedirla hoy». Las dos frases ya no son ciertas: `pedir-por-expresion` (`protocolo/requerimientos.lisp`) traduce el nodo en una petición de `con-relacion-uno-a-muchos`, la web la cumple, texto la cumple escribiendo las filas separadas por comas y Excel responde `:rechaza` con la nota que dice que la emulación está diseñada y no implementada.

(plural CANTIDAD SINGULAR PLURAL) con las dos ramas como expresiones. El capítulo 4 la escribe como (plural N "sing" "plur"), con cadenas. En `compilar-forma-compuesta` las tres ramas pasan por `compilar-expresion`, así que el sustantivo puede venir de los datos. Es lo que permite «Faltan 5 resmas», donde la unidad de medida es un campo y no una palabra del programador.

Hay además un cambio de alcance que no añade forma pero sí semántica: `+variable-candidata+` y todo identificador de una descripción se internan ahora en el paquete `situacion.nombres` y no en el del núcleo. Se explica más abajo.

12.4. Capacidades nuevas

El capítulo 5 lista doce capacidades y el capítulo 6 repite el número. En `protocolo/capacidades.lisp` hay quince. Las tres nuevas, con el motivo que declara su propia documentación:

`con-busqueda-por-clave`. Traer un dato de otra colección a partir de una clave. Es la construcción central del trabajo y hasta ahora era invisible en el informe de conformidad, porque las tres arquitecturas la cumplen y nadie la pedía. Que salga como cumplida no es

ruido: un informe que sólo lista lo que falla no dice qué se comprobó, y una arquitectura de exportación plana no la cumpliría.

con-tabla-cruzada. Poner los valores de un campo en el eje de columnas. La dificultad no es dibujarla: es que el número de columnas depende del contenido y no de la declaración. En una página eso no cuesta nada; en una hoja de cálculo mueve el direccionamiento entero, y por eso allí es una emulación con límite declarado.

con-agrupacion. Que las filas de una colección salgan de otra: «una fila por proveedor», «una fila por partida». El conjunto de filas no se declara, se calcula. Ninguna de las tres arquitecturas la cumple de forma nativa —las tres la emulan fijando las filas al generar— y el invariante I5 la lleva en su lista de huérfanas justificadas para que eso sea un acto deliberado y no un descuido.

Los conjuntos por arquitectura también cambiaron, y siguen sin estar anidados, que es el argumento de la inversión de capacidades del capítulo 6: texto pasa de 1 a 4 (**con-tabla-cruzada**, **con-marcado-textual**, **con-busqueda-por-clave**, **con-orden-declarado**), Excel de 8 a 9 y la web de 10 a 12. Que la arquitectura elegida por no poder casi nada cumpla hoy la tabla cruzada de forma nativa, y la hoja de cálculo tenga que emularla, refuerza el resultado en vez de debilitarlo.

12.5. La vista cruzada

El nodo `vista` de `nucleo/modelo.lisp` tiene tres ranuras que el capítulo 4 no describe: `eje-de-filas`, `eje-de-columnas` y `lo-que-se-muestra`. Se declaran con `:filas`, `:columnas` y `:muestra` en `compilar-vista` (`lenguaje/macros.lisp`), y la función `cruzada-p` es cierta cuando las tres están puestas.

```
(vista rejilla :de casillas :entrada t
              :etiqueta "Horario semanal"
              :filas turno :columnas dia :muestra asignatura)
```

El razonamiento está en el capítulo 8 y aquí sólo se registra su consecuencia: la matriz nunca fue una estructura de datos. Los datos siguen siendo una colección de filas —día, turno, asignatura, aula— y lo que es matriz es la presentación. Declararlo en la vista y no en la colección es el mismo criterio que el resto del lenguaje aplica: la intención se declara, el mecanismo lo inventa cada arquitectura. Es también lo que abarató el problema. Meter matrices en el modelo de datos habría tocado el núcleo, el análisis y las tres arquitecturas; como vista son tres ranuras y un método por destino.

Las otras dos ranuras de `vista`, `secciones` y `agrupacion`, siguen como estaban y se listan abajo.

12.6. Lo que se corrigió en el código

Tres fallos los encontró el experimento de alcance. Dos más los encontraron invariantes escritos a raíz de él.

1. La página recalculaba en una sola pasada. Con derivaciones que dependen de otras filas de la misma colección —el ganador de un partido pasa al siguiente— cada `render` hacía converger exactamente un nivel de la cadena y la página enseñaba valores incompletos sin avisar. El informe de conformidad decía «5 nativas, 0 degradadas»: un verde falso, que es la clase de fallo que este trabajo le reprocha a (Morales Lazo 2026). Corregido en `web/recursos.lisp`, `recalcularTodo`: se recalcula hasta punto fijo con tope de 32 pasadas, y si el tope se alcanza `RECALCULO_INCOMPLETO` queda en cierto y la página lo dice.

2. `(:orden dia turno)` descartaba el segundo campo en silencio. `compile-coleccion` tomaba el primero y tiraba el resto. Ahora señala un error que distingue los dos casos: si de verdad hace falta ordenar por varios campos, no es un problema de sintaxis, es que al lenguaje le falta el orden compuesto.

3. `(de (anterior fila :donde ...) campo)` descartaba el `:donde` en silencio, y daba números mal en una descripción que se leía como correcta. `compile-acceso` rechaza ahora cualquier argumento de más en el vecino, con el mismo tipo de mensaje. Las dos últimas son del mismo género y es el género que más importa: una descripción que parece correcta y significa otra cosa (Panko 1998; Hermans, Pinzger, y Deursen 2012).

4. Cuatro operaciones del protocolo que ninguna arquitectura implementaba. El capítulo 5 lo declara honestamente: `planificar-vista`, `emitir-marca`, `emitir-entrada` y `emitir-vista` estaban declaradas, documentadas y registradas, y lo que nombran se materializaba con código interno de cada backend. El protocolo prometía más de lo que cumplía. Se escribió el invariante I6 (`pruebas/conformidad.lisp`, `i6-ninguna-operacion-del-protocolo-esta-muerta`), que recorre el registro de `definir-operacion` y falla si alguna genérica no tiene métodos. Hoy las tres arquitecturas especializan las cuatro, y el capítulo 5 debe leerse sabiendo que su párrafo sobre «contrato declarado sin uso» ya no describe el código.

5. Los nombres de usuario se internaban en el paquete del núcleo. `nombrar` (`nucleo/nodo.lisp`) internaba en `situacion.nucleo` todo identificador escrito por el autor de una descripción, como dice el capítulo 4. El efecto era que el invariante I1 —ningún símbolo del núcleo puede nombrar una dirección— estaba juzgando nombres que no son del sistema: bastaba llamar `rejilla` a una vista para que I1 mordiera sin que el núcleo hubiera nombrado ninguna rejilla. Ahora hay un paquete aparte, `situacion.nombres`, sin `:use` y sin exportar nada, y `nombrar` interna allí. I1 vuelve a juzgar sólo el vocabulario del sistema.

12.7. Lo que sigue sin estar

Comprobado contra el código, no contra la documentación.

- **La arquitectura SQL.** `situacion.asd` no declara ningún sistema para ella. Con ella, `con-derivacion-por-consulta` no la cumple ninguna arquitectura: las tres la degradan diciendo que sus derivaciones son vivas, que es más de lo que se pide.
- **con-agrupacion sin dueño nativo.** Las tres la emulan fijando las filas al generar. Está en la lista de huérfanas justificadas de I5.

- **:secciones y :agrupada-por.** El análisis comprueba que los campos existan (`comprobar-vista`); ninguna arquitectura las lee. `grep` sobre `texto/`, `excel/` y `web/` da cero usos de las dos ranuras.
- **:opcional.** Se guarda en el nodo `campo` y lo consulta sólo `compile-campo` para guardarlo. Sigue siendo documentación.
- **El tipo declarado.** `:tipo` se normaliza a palabra clave y no se valida: el análisis no comprueba que sea uno de los seis previstos. `grep -n tipo analisis/comprobacion.lisp` no devuelve nada.
- **El parámetro editable.** `emitir-expresion` sobre `ref-parametro` resuelve el valor como literal en tiempo de emisión, en Excel y en la web (`excel/formula.lisp:56`, `web/expresiones.lisp:48`). Un `parametro ... :rol entrada` no produce ningún control en ningún destino. Y ninguna de las tres descripciones del corpus declara un parámetro: `grep -n parametro corpus/*.lisp` no devuelve nada, de modo que la forma sigue sin oráculo.
- **El agregado sobre una expresión.** `compile-agregado-de-campo` exige un campo. La media ponderada —el cálculo más común de un documento académico— sigue necesitando una columna auxiliar que el usuario no pidió.
- **El redondeo, el promedio y el valor absoluto.** El `ecase` del evaluador (`nucleo/evaluador.lisp:393`) tiene cinco operaciones: `:cuantas`, `:suma`, `:minimo`, `:maximo`, `:distintas`. No hay nodo unario de redondeo, y sin él cualquier porcentaje realista rompe la conformidad entre destinos.
- **La degradación con `restart-case`.** El documento de diseño la propone; el código usa una genérica más una política externa. El capítulo 5 ya lo declara y sigue siendo así.

Ninguna de estas ausencias invalida lo que los capítulos 4 a 6 afirman. Cambian los números, y cambian dos afirmaciones concretas: la lista del álgebra no estaba cerrada, y el contrato del protocolo prometía más de lo que cumplía. Las dos se descubrieron por intentar describir cosas nuevas y por escribir invariantes que mordieran, que es el método que este trabajo defiende frente al de la línea anterior ([Arcia Corcho 2017](#); [Morales Lazo 2026](#)).

Referencias bibliográficas

- Abraham, Robin, y Martin Erwig. 2006. «Inferring templates from spreadsheets». En *Proceedings of the 28th International Conference on Software Engineering (ICSE '06)*, 182-91. ACM. <https://doi.org/10.1145/1134285.1134312>.
- American National Standard for Information Systems. Programming Language Common Lisp*. 1994. ANSI.
- Arcia Corcho, Yasmany. 2017. «Automatizar la creación de Generadores Multilenguajes». Trabajo de diploma, La Habana, Cuba: Facultad de Matemática y Computación, Universidad de La Habana.
- Cápiro Colomar, Ricardo Antonio. 2025. «LDMAG: Lenguaje para el diseño de microaplicaciones de gestión». Trabajo de diploma, La Habana, Cuba: Facultad de Matemática y Computación, Universidad de La Habana.
- Czarnecki, Krzysztof, y Ulrich W. Eisenecker. 2000. *Generative Programming: Methods, Tools, and Applications*. Addison-Wesley.
- ECMA-376: Office Open XML File Formats*. 2016. 5.^a ed. ECMA International.
- Eysholdt, Moritz, y Heiko Behrens. 2010. «Xtext: implement your language faster than the quick and dirty way». En *Proceedings of the ACM International Conference Companion on Object Oriented Programming Systems Languages and Applications (OOPSLA '10)*, 307-9. ACM. <https://doi.org/10.1145/1869542.1869625>.
- Fowler, Martin. 2010. *Domain-Specific Languages*. The Addison-Wesley Signature Series. Addison-Wesley.
- Graham, Paul. 1993. *On Lisp: Advanced Techniques for Common Lisp*. Prentice Hall.
- Hermans, Felienne, Martin Pinzger, y Arie van Deursen. 2012. «Detecting and visualizing inter-worksheet smells in spreadsheets». En *Proceedings of the 34th International Conference on Software Engineering (ICSE '12)*, 441-51. IEEE. <https://doi.org/10.1109/ICSE.2012.6227171>.
- Hudak, Paul. 1996. «Building domain-specific embedded languages». *ACM Computing Surveys* 28 (4es). <https://doi.org/10.1145/242224.242477>.
- Keene, Sonya E. 1989. *Object-Oriented Programming in Common Lisp: A Programmer's Guide to CLOS*. Addison-Wesley.
- Kelly, Steven, y Juha-Pekka Tolvanen. 2008. *Domain-Specific Modeling: Enabling Full Code Generation*. Wiley-IEEE Computer Society Press.
- Kiczales, Gregor, Jim des Rivières, y Daniel G. Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT Press.
- Ko, Andrew J., Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin

- Erwig, Chris Scaffidi, et al. 2011. «The state of the art in end-user software engineering». *ACM Computing Surveys* 43 (3): 1-44. <https://doi.org/10.1145/1922649.1922658>.
- Mernik, Marjan, Jan Heering, y Anthony M. Sloane. 2005. «When and how to develop domain-specific languages». *ACM Computing Surveys* 37 (4): 316-44. <https://doi.org/10.1145/1118890.1118892>.
- Morales Lazo, José Ernesto. 2026. «Un lenguaje de dominio específico para la descripción declarativa de estructuras tabulares». Trabajo de diploma, La Habana, Cuba: Facultad de Matemática y Computación, Universidad de La Habana.
- Panko, Raymond R. 1998. «What we know about spreadsheet errors». *Journal of End User Computing* 10 (2): 15-21.
- Peyton Jones, Simon, Alan Blackwell, y Margaret Burnett. 2003. «A user-centred approach to functions in Excel». En *Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming (ICFP '03)*, 165-76. ACM. <https://doi.org/10.1145/944705.944721>.
- Seibel, Peter. 2005. *Practical Common Lisp*. Apress.
- Sestoft, Peter. 2014. *Spreadsheet Implementation Technology: Basics and Extensions*. MIT Press.
- Steele, Guy L. 1990. *Common Lisp: The Language*. 2.^a ed. Digital Press.
- Steinberg, Dave, Frank Budinsky, Marcelo Paternostro, y Ed Merks. 2008. *EMF: Eclipse Modeling Framework*. 2.^a ed. Addison-Wesley.
- Voelter, Markus. 2013. *DSL Engineering: Designing, Implementing and Using Domain-Specific Languages*. CreateSpace / dslbook.org.